



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Πολυτεχνική Σχολή
Τμήμα Μηχανικών Η/Υ & Πληροφορικής

Διπλωματική Εργασία

ΔΗΜΙΟΥΡΓΙΑ ΙΣΤΟΤΟΠΟΥ ΓΙΑ ΕΝΑΛΛΑΚΤΙΚΕΣ - ΔΙΑΔΡΑΣΤΙΚΕΣ
ΔΡΑΣΤΗΡΙΟΤΗΤΕΣ ΜΕ ΕΦΑΡΜΟΓΕΣ ΑΙ

Πούπουζα Βασιλική
Α.Μ. 1080449

Επιβλέπων
Σιούτας Σπυρίδων, Καθηγητής

Πάτρα, 2026



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

Τριμελής Επιτροπή Αξιολόγησης

Σιούτας Σπυρίδων, Καθηγητής

Μακρής Χρήστος, Αναπληρωτής Καθηγητής

Δημήτρης Τσώλης, Αναπληρωτής Καθηγητής



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

Copyright, all rights reserved Πούπουζα Βασιλική, 2026

Με την επιφύλαξη παντός δικαιώματος.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ' ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα.

Το περιεχόμενο αυτής της εργασίας δεν απηχεί απαραίτητα τις απόψεις του τμήματος, του Επιβλέποντα ή της επιτροπής που την ενέκρινε.

Υπεύθυνη Δήλωση

Βεβαιώνω ότι είμαι συγγραφέας αυτής της διπλωματικής εργασίας, και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στη διπλωματική εργασία. Επίσης έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επίσης βεβαιώνω ότι αυτή η διπλωματική εργασία προετοιμάστηκε από εμένα προσωπικά ειδικά για τις απαιτήσεις του προγράμματος σπουδών του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής του Πανεπιστημίου Πατρών.

(Υπογραφή)

.....

Πούπουζα Βασιλική

Περίληψη

Η παρούσα διπλωματική εργασία πραγματεύεται τον σχεδιασμό, την υλοποίηση και την ανάπτυξη (deployment) της διαδικτυακής εφαρμογής "Pyxis", ενός καινοτόμου συστήματος που στοχεύει στη διευκόλυνση και αυτοματοποίηση της λήψης ομαδικών αποφάσεων για δραστηριότητες και εξόδους. Στη σύγχρονη εποχή της υπερπληροφορίας, η οργάνωση μιας κοινής εμπειρίας μέσω παραδοσιακών εφαρμογών ανταλλαγής μηνυμάτων (group chats) ή ατομικών μηχανών αναζήτησης καταλήγει συχνά να είναι μια διαδικασία χρονοβόρα, πολύπλοκη και αναποτελεσματική, οδηγώντας αρκετές φορές σε αδιέξοδο ή δυσαρέσκεια των μελών μιας ομάδας.

Για την επίλυση αυτού του προβλήματος, η εφαρμογή Pyxis προτείνει μια ολοκληρωμένη αρχιτεκτονική που συνδυάζει σύγχρονους τεχνολογικούς πυλώνες. Αρχικά, παρέχει τη δυνατότητα δημιουργίας ψηφιακών δωματίων (Lobbies), όπου οι χρήστες μπορούν να συνδεθούν με ασφάλεια σε ένα κοινό, ιδιωτικό περιβάλλον. Δεύτερον, ενσωματώνει ισχυρά στοιχεία παιχνιδοποίησης μέσω ενός πολυτροπικού μηχανισμού σάρωσης καρτών (Swiping), επιτρέποντας την ταχύτατη και ταυτόχρονη ψηφοφορία επί των διαθέσιμων επιλογών. Τρίτον, αναπτύσσει μια δυναμική Σελίδα Προτάσεων (Suggestions Page) που αποτελεί τον ευφυή πυρήνα του συστήματος. Μέσω της αξιοποίησης του Μεγάλου Γλωσσικού Μοντέλου LLaMA 3.3 και της ανάλυσης δεδομένων σε πραγματικό χρόνο, η σελίδα αυτή λειτουργεί ως ένας έξυπνος διαμεσολαβητής, ικανός να κατανοεί σύνθετα αιτήματα σε φυσική γλώσσα και να παράγει εξατομικευμένες προτάσεις με επίγνωση πλαισίου (context-awareness).

Από τεχνικής σκοπιάς, το σύστημα σχεδιάστηκε ακολουθώντας την αρχιτεκτονική Πελάτη-Εξυπηρετητή (Client-Server). Η διεπαφή χρήστη (Frontend) αναπτύχθηκε με τη βιβλιοθήκη React.js, ενσωματώνοντας διαδραστικούς χάρτες, αναλυτικά γραφήματα στατιστικών και έναν τοπικό ευρετικό αλγόριθμο βαθμολόγησης (Heuristic Scoring). Η επιχειρηματική λογική (Backend) υλοποιήθηκε στο περιβάλλον Node.js με χρήση σχεσιακής βάσης δεδομένων (MySQL) και προτύπων RESTful API. Μια από τις σημαντικότερες καινοτομίες αποτελεί η "επίγνωση πλαισίου" (context-awareness) του μοντέλου AI, το οποίο λαμβάνει δεδομένα καιρού σε πραγματικό χρόνο (OpenWeather API) για να αναπροσαρμόζει δυναμικά τις προτάσεις του. Παράλληλα, η ενσωμάτωση του πρωτοκόλλου WebSockets (Socket.IO) εξασφαλίζει την αμφίδρομη επικοινωνία, υποστηρίζοντας ζωντανή συνομιλία (Live Chat) και ακαριαίο συγχρονισμό των αποτελεσμάτων ομοφωνίας (Match). Το τελικό προϊόν

αναπτύχθηκε επιτυχώς σε περιβάλλον νέφους (cloud deployment), αποτελώντας μια πλήρως λειτουργική και επεκτάσιμη πλατφόρμα.

Λέξεις-Κλειδιά: Διαδικτυακή Εφαρμογή, Ομαδική Λήψη Αποφάσεων, Συστήματα Συστάσεων για Ομάδες, Τεχνητή Νοημοσύνη, Παιχνιδοποίηση, Client-Server, WebSockets, React.js, Node.js.

Abstract

This diploma thesis presents the design, implementation, and deployment of "Pyxis", an innovative web application aimed at facilitating and automating group decision-making for activities and outings. In the modern era of information overload, organizing a shared experience through traditional messaging applications or individual search engines often becomes a time-consuming, complex, and inefficient process, frequently leading to deadlocks or dissatisfaction among group members.

To solve this problem, Pyxis proposes a comprehensive architecture combining modern technological pillars. First, it enables the creation of digital rooms (Lobbies), where users can securely connect in a shared, private environment. Second, it integrates strong gamification elements through a multi-modal card-swiping mechanism, allowing for rapid and simultaneous voting on available options. Third, the Suggestions Page constitutes the intelligent core of the system. By leveraging the LLaMA 3.3 Large Language Model and real-time data analysis, this page acts as an intelligent mediator capable of understanding natural language requests and dynamically generating highly personalized recommendations with context-awareness.

From a technical perspective, the system was designed following the Client-Server architecture. The user interface (Frontend) was developed using the React.js library, incorporating interactive maps, statistical analytics charts, and a local heuristic scoring algorithm. The business logic (Backend) was implemented in Node.js, utilizing a relational database (MySQL) and RESTful API standards. A major innovation is the AI's "context-awareness," which retrieves real-time weather data (OpenWeather API) to dynamically adjust its recommendations. Furthermore, the integration of WebSockets (Socket.IO) ensures bidirectional communication, supporting Live Chat and instant result synchronization (Match) among users. The final product was successfully deployed in a cloud environment, resulting in a fully functional, user-friendly, and scalable social platform.

Keywords: Web Application, Group Recommender Systems, Artificial Intelligence, Gamification, Client-Server, WebSockets, React.js, Node.js.

Πίνακας Περιεχομένων

Περίληψη.....	3
Abstract	5
Πίνακας Εικόνων.....	10
Κατάλογος Ακρωνυμίων	11
Κεφάλαιο 1: Εισαγωγή.....	12
1.1 Γενικό Πλαίσιο και Κίνητρο	12
1.2 Αντικείμενο και Στόχοι της Εργασίας	13
1.3 Η Συνεισφορά της Εργασίας	14
1.4 Δομή της Διπλωματικής Εργασίας	15
Κεφάλαιο 2: Βιβλιογραφική Ανασκόπηση και Σχετικές Εργασίες	16
2.1 Εισαγωγή.....	16
2.2 Συστήματα Συστάσεων (Recommendation Systems).....	16
2.3 Συστήματα Συστάσεων για Ομάδες (Group Recommendation Systems)	17
2.4 Μηχανισμοί Αλληλεπίδρασης και Παιχνιδοποίηση (Gamification & Swiping)	18
2.5 Η Συμβολή της Τεχνητής Νοημοσύνης (AI) και των Chatbots	19
2.6 Υφιστάμενες Εφαρμογές και η Καινοτομία του Προτεινόμενου Συστήματος.....	20
Κεφάλαιο 3: Θεωρητικό Υπόβαθρο και Τεχνολογίες Υλοποίησης	22
3.1 Εισαγωγή.....	22
3.2 Αρχιτεκτονική Πελάτη-Εξυπηρετητή (Client-Server Architecture)	22
3.3 Τεχνολογίες Διεπαφής Χρήστη (Frontend) - React.js και Βιβλιοθήκες Επέκτασης	23
3.3.1 Πειραματική Αξιολόγηση Εργαλείων Διεπαφής (Swiping)	25
3.4 Περιβάλλον Backend - Node.js και Express.js	25
3.5 Σχεσιακά Συστήματα Διαχείρισης Βάσεων Δεδομένων (RDBMS) - MySQL	26
3.6 Επικοινωνία σε Πραγματικό Χρόνο και WebSockets (Socket.IO).....	26
3.7 Μηχανισμοί Ασφάλειας, Ταυτοποίησης και Κρυπτογράφησης (JWT & bcrypt).....	27
3.8 Μεγάλα Γλωσσικά Μοντέλα (LLMs) και το Μοντέλο LLaMA 3.3	28
3.8.1 Πειραματική Αξιολόγηση Εναλλακτικών Μοντέλων AI (OpenAI & Google Gemini)	28
3.9 Αρχιτεκτονική RESTful API.....	29
3.9.1 Δοκιμές και Αποσφαλμάτωση API (API Testing & Debugging).....	31
Κεφάλαιο 4: Σχεδιασμός και Αρχιτεκτονική Συστήματος.....	32

4.1 Εισαγωγή.....	32
4.2 Αρχιτεκτονική Σχεδίαση (System Architecture).....	32
4.3 Σχεδιασμός και Μοντελοποίηση Βάσης Δεδομένων (Database Schema).....	33
4.4 Ροές Δεδομένων και Λειτουργικά Διαγράμματα	35
4.5 Ενσωμάτωση Τεχνητής Νοημοσύνης και Μηχανική Υποδείξεων (Prompt Engineering)	37
4.5.1 Δομή και Πλαίσιο Υποδείξεων (Prompt Structuring)	37
4.5.2 Διαχείριση Ιστορικού Συνομιλίας (Context Window Management)	37
4.5.3 Μηχανική Υποδείξεων για Παραγωγή Δραστηριοτήτων.....	37
4.5.4 Βελτιστοποίηση Κόστους και Ταχύτητας (Pre-filtering)	37
4.5.5 Επίγνωση Πλαισίου (Context-Awareness) μέσω OpenWeather API	38
4.5.6 Ευρετικός Αλγόριθμος Βαθμολόγησης Πελάτη (Frontend Heuristic Scoring)	38
4.6 Δυναμικές Ροές Εργασίας και ο Αλγόριθμος Συμφωνίας (Matching Algorithm)	39
4.6.1 Λογική Δωματίων (Lobby Session Lifecycle)	39
4.6.2 Υλοποίηση του Αλγορίθμου Συμφωνίας σε Πραγματικό Χρόνο.....	39
Κεφάλαιο 5: Υλοποίηση, Διεπαφή Χρήστη (UI) και Ανάπτυξη (Deployment).....	40
5.1 Εισαγωγή.....	40
5.2 Υλοποίηση Διεπαφής Χρήστη (User Interface / User Experience).....	40
5.2.1 Σύστημα Ταυτοποίησης και Προστατευμένες Διαδρομές (Login / Registration & Protected Routes)	41
5.2.2 Αρχική Σελίδα	42
5.2.3 Δημιουργία και Είσοδος σε Ψηφιακά Δωμάτια (Lobbies).....	42
5.2.4 Περιβάλλον Διαδραστικού Βοηθού και Δυναμικοί Σύνδεσμοι (AI Chatbot & Regex Link Parsing).....	43
5.2.5 Μηχανισμός Σάρωσης και Ψηφοφορίας (Swiping)	44
5.2.6 Οθόνη Επιτυχούς Συμφωνίας (Match Screen)	45
5.2.7 Εξερεύνηση και Πολυκριτηριακή Αναζήτηση Δραστηριοτήτων.....	46
5.2.8 Διαδραστικός Χάρτης Δραστηριοτήτων (Interactive Map).....	47
5.2.9 Προφίλ Χρήστη, Στατιστικά και Προσωπικές Επιλογές (Profile, Recharts & Personal Choice)	48
5.2.10 Μηχανισμός Τυχαίας Επιλογής (Surprise Dice)	49
5.2.11 Δυναμική Προσαρμογή Διεπαφής (Time-based Theming)	50
5.2.12 Κοινωνικός Διαμοιρασμός και Ενσωμάτωση AI (Social & AI Insights)	51
5.2.13 Ανακάλυψη Ομάδων (Lobby Radar) και Πρόσβαση (PIN Access)	52
5.2.14 Ζωντανή Συνομιλία (Live Chat) σε Πραγματικό Χρόνο.....	53
5.2.15 Διαβαθμισμένα Αποτελέσματα Ομοφωνίας (Match Results)	54

5.2.16 Έξυπνη Διαχείριση Πολυμέσων (Smart Image Fallbacks).....	55
5.3 Διάθεση της Εφαρμογής στο Νέφος (Cloud Deployment).....	56
5.4 Σύνοψη.....	57
Κεφάλαιο 6: Συμπεράσματα και Μελλοντικές Επεκτάσεις	57
6.1 Σύνοψη και Συμπεράσματα.....	57
6.2 Μελλοντικές Επεκτάσεις	58
7. Βιβλιογραφία.....	60
Επιστημονικά Άρθρα, Βιβλία και Έρευνες.....	60
Τεχνολογίες, Πλαίσια Ανάπτυξης και Εργαλεία.....	61
Πηγές Πολυμέσων και Φωτογραφικού Υλικού	62
Παράρτημα Α: Σημαντικά Αποσπάσματα Πηγαίου Κώδικα	64
Ενότητα Α.1: Υλοποίηση Τεχνητής Νοημοσύνης (Backend - ai.js).....	64
Απόσπασμα 1: Αλγόριθμος Καθαρισμού και Κανονικοποίησης Γεωγραφικών Δεδομένων	64
Απόσπασμα 2: Παραμετροποίηση Μοντέλου (System Prompt Engineering) για το AI Suggestion.....	65
Απόσπασμα 3: Ασφαλής Σύνδεση με το Groq SDK και Επιβολή Δομημένης Εξόδου (Structured JSON).....	65
Απόσπασμα 4: Περιορισμός Ψευδαισθήσεων (Hallucination Control) στη Λειτουργία του Chatbot	66
Ενότητα Α.2: Επιχειρησιακή Λογική και SQL Queries (Backend - group.js)	66
Απόσπασμα 5: Καταγραφή Ψήφων με Αποτροπή Διπλοεγγραφών (Upsert Logic)	66
Απόσπασμα 6: Αλγόριθμος Real-Time Υπολογισμού Απόλυτης Ομοφωνίας (Match) .	67
Απόσπασμα 7: Σύνθετο SQL Query για τον Υπολογισμό των Match Results (Ranking)	67
Ενότητα Α.3: Σύνδεση WebSockets και Mobile Gestures (Frontend - GroupSwipePage.jsx)	68
Απόσπασμα 8: Διαχείριση Real-Time Γεγονότων μέσω Socket.IO Client.....	68
Απόσπασμα 9: Μαθηματικός Υπολογισμός Κίνησης για Custom Swipe Χειρονομίες..	69
Ενότητα Α.4: Διαδραστική Χαρτογράφηση και Γεωμετρία (Frontend - Map.js).....	69
Απόσπασμα 10: Αλγόριθμος Πρόληψης Επικάλυψης Pins (Marker Displacement Logic)	69
Ενότητα Α.5: Ασφάλεια και Αρχιτεκτονική Υποδομή	70
Απόσπασμα 11: Προστασία Διαδρομών (Middleware Αυθεντικοποίησης)	70
Απόσπασμα 12: Κρυπτογράφηση Κωδικών (bcrypt).....	71
Απόσπασμα 13: Κεντρική Διαχείριση Σύνδεσης Βάσης (Database Connection Pooling)	72

Απόσπασμα 14: Προστασία από SQL Injection	73
Απόσπασμα 15: Συνολική Διαχείριση Σφαλμάτων (Error Handling).....	73

Πίνακας Εικόνων

ΕΙΚΟΝΑ 3.1: ΑΡΧΙΤΕΚΤΟΝΙΚΟ ΔΙΑΓΡΑΜΜΑ ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ ΡΥΧΙΣ ΚΑΙ ΤΩΝ ΕΞΩΤΕΡΙΚΩΝ ΥΠΗΡΕΣΙΩΝ (3-Tier ARCHITECTURE).	23
ΕΙΚΟΝΑ 4.1: ΔΙΑΓΡΑΜΜΑ ΟΝΤΟΤΗΤΩΝ - ΣΥΣΧΕΤΙΣΕΩΝ (ER DIAGRAM) ΤΗΣ ΣΧΕΣΙΑΚΗΣ ΒΑΣΗΣ ΔΕΔΟΜΕΝΩΝ MYSQL ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ ΡΥΧΙΣ.	34
ΕΙΚΟΝΑ 4.2: ΔΙΑΓΡΑΜΜΑ ΡΟΗΣ ΕΡΓΑΣΙΩΝ (WORKFLOW DIAGRAM) ΚΑΤΑ ΤΗ ΔΙΑΡΚΕΙΑ ΜΙΑΣ ΟΜΑΔΙΚΗΣ ΣΥΝΕΔΡΙΑΣ ΣΤΟ ΣΥΣΤΗΜΑ ΡΥΧΙΣ.	36
ΕΙΚΟΝΑ 5.1: ΣΤΙΓΜΙΟΤΥΠΟ ΟΘΟΝΗΣ ΑΠΟ ΤΗ ΣΕΛΙΔΑ ΣΥΝΔΕΣΗΣ (LOGIN) ΚΑΙ ΑΥΘΕΝΤΙΚΟΠΟΙΗΣΗΣ ΤΗΣ ΕΦΑΡΜΟΓΗΣ ΡΥΧΙΣ.	41
ΕΙΚΟΝΑ 5.2: ΣΤΙΓΜΙΟΤΥΠΟ ΤΗΣ ΚΕΝΤΡΙΚΗΣ ΟΘΟΝΗΣ (HOME PAGE) ΤΗΣ ΕΦΑΡΜΟΓΗΣ, ΟΠΟΥ ΑΠΕΙΚΟΝΙΖΟΝΤΑΙ ΟΙ ΠΡΟΣΩΠΟΠΟΙΗΜΕΝΕΣ ΠΡΟΤΑΣΕΙΣ ΤΟΥ ΕΥΡΕΤΙΚΟΥ ΑΛΓΟΡΙΘΜΟΥ ΒΑΣΕΙ ΤΩΝ ΠΡΟΤΙΜΗΣΕΩΝ ΤΟΥ ΧΡΗΣΤΗ.	42
ΕΙΚΟΝΑ 5.3: ΣΤΙΓΜΙΟΤΥΠΟ ΟΘΟΝΗΣ ΑΠΟ ΤΟ ΠΕΡΙΒΑΛΛΟΝ ΤΟΥ LOBBY ΚΑΙ ΤΟ ΠΕΔΙΟ ΕΙΣΑΓΩΓΗΣ ΤΟΥ ΕΞΑΨΗΦΙΟΥ ΚΩΔΙΚΟΥ.	43
ΕΙΚΟΝΑ 5.4: ΣΤΙΓΜΙΟΤΥΠΟ ΟΘΟΝΗΣ ΑΠΟ ΤΗ ΔΙΕΠΑΦΗ ΣΥΝΟΜΙΛΙΑΣ ΜΕ ΤΟ ΕΝΣΩΜΑΤΩΜΕΝΟ ΑΙ CHATBOT.	44
ΕΙΚΟΝΑ 5.5: ΣΤΙΓΜΙΟΤΥΠΟ ΟΘΟΝΗΣ ΑΠΟ ΤΗΝ ΚΑΡΤΑ ΔΡΑΣΤΗΡΙΟΤΗΤΑΣ ΚΑΙ ΤΟΝ ΜΗΧΑΝΙΣΜΟ ΣΑΡΩΣΗΣ (SWIPING).	45
ΕΙΚΟΝΑ 5.6: ΣΤΙΓΜΙΟΤΥΠΟ ΟΘΟΝΗΣ ΑΠΟ ΤΗΝ ΕΜΦΑΝΙΣΗ ΕΠΙΤΥΧΟΥΣ ΣΥΜΦΩΝΙΑΣ (MATCH) ΣΤΟ ΣΥΝΟΛΟ ΤΗΣ ΟΜΑΔΑΣ.	46
ΕΙΚΟΝΑ 5.7: Η ΚΕΝΤΡΙΚΗ ΣΕΛΙΔΑ ΕΞΕΡΕΥΝΗΣΗΣ ΔΡΑΣΤΗΡΙΟΤΗΤΩΝ ΚΑΙ ΔΥΝΑΜΙΚΩΝ ΠΡΟΤΑΣΕΩΝ.	47
ΕΙΚΟΝΑ 5.8: ΑΠΕΙΚΟΝΙΣΗ ΤΟΥ ΔΙΑΔΡΑΣΤΙΚΟΥ ΧΑΡΤΗ ΔΡΑΣΤΗΡΙΟΤΗΤΩΝ ΜΕ ΔΥΝΑΜΙΚΗ ΟΜΑΔΟΠΟΙΗΣΗ.	48
ΕΙΚΟΝΑ 5.9: ΠΡΟΒΟΛΗ ΤΟΥ ΠΡΟΦΙΛ ΧΡΗΣΤΗ ΚΑΙ ΟΠΤΙΚΟΠΟΙΗΣΗ ΣΤΑΤΙΣΤΙΚΩΝ ΜΕΣΩ RECHARTS.	49
ΕΙΚΟΝΑ 5.10: ΜΗΧΑΝΙΣΜΟΣ ΤΥΧΑΙΑΣ ΕΠΙΛΟΓΗΣ (SURPRISE DICE).	50
ΕΙΚΟΝΑ 5.11: ΤΟ ΣΚΟΤΕΙΝΟ ΘΕΜΑ (DARK MODE) ΤΗΣ ΔΙΕΠΑΦΗΣ.	51
ΕΙΚΟΝΑ 5.12: ΛΕΠΤΟΜΕΡΕΙΕΣ ΔΡΑΣΤΗΡΙΟΤΗΤΑΣ, ΚΟΙΝΩΝΙΚΟΣ ΔΙΑΜΟΙΡΑΣΜΟΣ ΚΑΙ ΑΙ ΡΥΧΙΣ INSIGHT.	52
ΕΙΚΟΝΑ 5.13: ΑΝΑΚΑΛΥΨΗ ΕΝΕΡΓΩΝ ΟΜΑΔΩΝ ΜΕΣΩ ΤΟΥ LOBBY RADAR.	53
ΕΙΚΟΝΑ 5.14: ΖΩΝΤΑΝΗ ΑΝΤΑΛΛΑΓΗ ΜΗΝΥΜΑΤΩΝ ΤΗΣ ΟΜΑΔΑΣ.	54
ΕΙΚΟΝΑ 5.15: ΠΡΟΒΟΛΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ ΟΜΟΦΩΝΙΑΣ Η ΠΛΕΙΟΨΗΦΙΑΣ.	55
ΕΙΚΟΝΑ 5.16: ΕΦΑΡΜΟΓΗ ΤΟΥ ΑΛΓΟΡΙΘΜΟΥ ΕΞΥΠΙΝΗΣ ΑΝΑΠΛΗΡΩΣΗΣ ΕΙΚΟΝΩΝ (SMART FALLBACKS).	56

Κατάλογος Ακρωνυμίων

- **AI (Artificial Intelligence):** Τεχνητή Νοημοσύνη
- **API (Application Programming Interface):** Διεπαφή Προγραμματισμού Εφαρμογών
- **CRUD (Create, Read, Update, Delete):** Δημιουργία, Ανάγνωση, Ενημέρωση, Διαγραφή (Βασικές λειτουργίες βάσεων δεδομένων)
- **DOM (Document Object Model):** Μοντέλο Αντικειμένων Εγγράφου
- **HCI (Human-Computer Interaction):** Αλληλεπίδραση Ανθρώπου-Υπολογιστή
- **HTTP (Hypertext Transfer Protocol):** Πρωτόκολλο Μεταφοράς Υπερκειμένου
- **JSON (JavaScript Object Notation):** Μορφή Ανταλλαγής Δεδομένων
- **JWT (JSON Web Token):** Πρότυπο ασφαλούς ανταλλαγής πληροφοριών και ταυτοποίησης
- **LLM (Large Language Model):** Μεγάλο Γλωσσικό Μοντέλο
- **MVP (Minimum Viable Product):** Ελάχιστο Βιώσιμο Προϊόν
- **NLP (Natural Language Processing):** Επεξεργασία Φυσικής Γλώσσας
- **RDBMS (Relational Database Management System):** Σχεσιακό Σύστημα Διαχείρισης Βάσεων Δεδομένων
- **REST (Representational State Transfer):** Αρχιτεκτονικό Πρότυπο για Web Services
- **SPA (Single Page Application):** Μονοσέλιδη Εφαρμογή
- **SQL (Structured Query Language):** Δομημένη Γλώσσα Ερωτημάτων
- **UI (User Interface):** Διεπαφή Χρήστη
- **UX (User Experience):** Εμπειρία Χρήστη

Κεφάλαιο 1: Εισαγωγή

1.1 Γενικό Πλαίσιο και Κίνητρο

Η ραγδαία εξέλιξη του διαδικτύου και η διείσδυση των φορητών συσκευών στην καθημερινότητα έχουν μεταβάλει ριζικά τον τρόπο με τον οποίο οι χρήστες αναζητούν και επιλέγουν εναλλακτικές λύσεις ψυχαγωγίας, εστίασης και δραστηριοτήτων. Οι σύγχρονες πλατφόρμες και μηχανές αναζήτησης προσφέρουν έναν τεράστιο όγκο δεδομένων, γεγονός που συχνά οδηγεί στο φαινόμενο της «υπερφόρτωσης πληροφοριών» (information overload). Παρότι η εξατομικευμένη αναζήτηση έχει βελτιστοποιηθεί σε μεγάλο βαθμό μέσω των Συστημάτων Συστάσεων, η ομαδική λήψη αποφάσεων παραμένει μια περίπλοκη και συχνά αναποτελεσματική διαδικασία.

Το κύριο κίνητρο για την εκπόνηση της παρούσας διπλωματικής εργασίας πηγάζει από μια γνώριμη καθημερινή δυσκολία: την προσπάθεια μιας ομάδας ατόμων να καταλήξει σε κοινή απόφαση για μια δραστηριότητα ή έξοδο. Η διαδικασία αυτή συνήθως διεξάγεται μέσω εφαρμογών ομαδικών συνομιλιών, όπου οι προτάσεις χάνονται μέσα σε ένα πλήθος μηνυμάτων. Ως αποτέλεσμα, η λήψη αποφάσεων καθυστερεί, ενώ συχνά επικρατούν οι προτιμήσεις των πιο ενεργών μελών, δημιουργώντας αίσθημα δυσαρέσκειας στα υπόλοιπα μέλη. Διαπιστώθηκε, επομένως, η επιτακτική ανάγκη για την ανάπτυξη ενός κεντρικού, δομημένου και αμερόληπτου ψηφιακού εργαλείου, το οποίο θα αυτοματοποιεί και θα διευκολύνει την επίτευξη ομαδικής συναίνεσης. Αφετηρία και κεντρικός πυρήνας αυτού του οράματος αποτέλεσε η δημιουργία μιας δυναμικής Σελίδας Προτάσεων (Suggestions Page). Αντί η ομάδα να αναλώνεται σε χειροκίνητες αναζητήσεις, η σελίδα αυτή σχεδιάστηκε για να λειτουργεί ως ένας "ευφυής διαμεσολαβητής" ο οποίος, με την αρωγή της Τεχνητής Νοημοσύνης, θα συνθέτει και θα προτείνει αυτόματα τις ιδανικές επιλογές, απαλλάσσοντας τους χρήστες από τον γνωσιακό φόρτο της αναζήτησης.

1.2 Αντικείμενο και Στόχοι της Εργασίας

Αντικείμενο της παρούσας διπλωματικής εργασίας αποτελεί ο σχεδιασμός και η εξ ολοκλήρου ανάπτυξη μιας διαδικτυακής εφαρμογής, με την ονομασία "Pyxis", η οποία λειτουργεί ως ένας ευφυής διαμεσολαβητής για τη λήψη ομαδικών αποφάσεων.

Οι επιμέρους στόχοι που τέθηκαν και επιτεύχθηκαν κατά τη διάρκεια της υλοποίησης είναι οι εξής:

- **Ανάπτυξη φιλικού περιβάλλοντος χρήστη (UI/UX):** Σχεδιασμός μιας μοντέρνας, διαδραστικής και αποκριτικής διεπαφής (μέσω της βιβλιοθήκης React.js). Η διεπαφή ενσωματώνει προηγμένες τεχνικές, όπως δυναμική αλλαγή θεμάτων βάσει χρόνου (time-based theming) και πολυτροπικό χειρισμό, επιτρέποντας την απρόσκοπτη πλοήγηση από οποιαδήποτε συσκευή.
- **Υλοποίηση Ομαδικών Περιβαλλόντων (Lobbies) & Live Chat:** Δημιουργία ενός ασφαλούς συστήματος υποδομής (Backend με Node.js, Express και MySQL). Οι χρήστες δύνανται να δημιουργούν κλειστές ομάδες (Private Squads) μέσω μοναδικών κωδικών PIN ή να ανακαλύπτουν ανοιχτές παρέες στην πόλη τους (Public Lobbies). Η εμπειρία ενισχύεται με την ενσωμάτωση ζωντανής συνομιλίας (Live Chat) σε πραγματικό χρόνο μέσω WebSockets (Socket.IO).
- **Ενσωμάτωση Μηχανισμών Παιχνιδοποίησης:** Αξιοποίηση της λογικής της σάρωσης καρτών (Swiping) ως καινοτόμου εργαλείου ψηφοφορίας, μετατρέποντας τη διαδικασία της επιλογής σε μια διαδραστική εμπειρία. Παράλληλα, παρέχονται εναλλακτικοί διασκεδαστικοί μηχανισμοί, όπως η τυχαία επιλογή δραστηριότητας ("Surprise Dice").
- **Αξιοποίηση Τεχνητής Νοημοσύνης (AI) με Επίγνωση Πλαισίου:** Διασύνδεση της πλατφόρμας με το Μεγάλο Γλωσσικό Μοντέλο LLaMA 3.3. Η Τεχνητή Νοημοσύνη δεν λειτουργεί απλώς ως Chatbot, αλλά διαθέτει επίγνωση πλαισίου (context-awareness), αντλώντας ζωντανά δεδομένα καιρού (OpenWeather API) για να παράγει δυναμικές, απόλυτα στοχευμένες και πρωτότυπες προτάσεις βάσει των συνθηκών της εκάστοτε στιγμής.

1.3 Η Συνεισφορά της Εργασίας

Η επιστημονική και πρακτική συνεισφορά της παρούσας εργασίας έγκειται στον επιτυχή συγκερασμό και την οργανική ενοποίηση τεσσάρων διακριτών τεχνολογικών τομέων: των **Συστημάτων Συστάσεων (Recommendation Systems)**, της **Παιχνιδοποίησης (Gamification)**, της **Επικοινωνίας σε Πραγματικό Χρόνο (Real-Time WebSockets)** και της **Επεξεργασίας Φυσικής Γλώσσας μέσω Τεχνητής Νοημοσύνης (Generative AI / Chatbots)**, υπό ένα ενιαίο και λειτουργικό πρίσμα.

Σε αντίθεση με τις υπάρχουσες πλατφόρμες της αγοράς, οι οποίες επικεντρώνονται αποκλειστικά είτε στην παθητική παροχή μεμονωμένων προτάσεων (π.χ. TripAdvisor) είτε σε διαπροσωπικές γνωριμίες (π.χ. εφαρμογές γνωριμιών), το Pyxis εισάγει μια νέα, υβριδική κατηγορία ψηφιακού εργαλείου (**Collaborative Group Decision Enabler**). Η καινοτομία και η κύρια συνεισφορά της εργασίας εστιάζεται στα εξής σημεία:

- **Αρχιτεκτονική AI με Επίγνωση Πλαισίου (Context-Aware AI Architecture):** Η υλοποίηση υπερβαίνει τη συμβατική χρήση στατικών προτροπών (prompts), εισάγοντας ένα δυναμικό σύστημα που αντλεί δεδομένα καιρού σε πραγματικό χρόνο (OpenWeather API). Αυτό επιτρέπει στο μοντέλο LLaMA 3.3 να προσαρμόζει σημασιολογικά τις προτάσεις του στις επικρατούσες περιβαλλοντικές συνθήκες, αυξάνοντας κατακόρυφα την πρακτική αξία των συστάσεων.
- **Υβριδικό Μοντέλο Βαθμολόγησης (Hybrid Edge & Cloud Computing):** Σχεδιάστηκε ένας μηχανισμός διπλού φιλτραρίσματος. Το Backend εκτελεί προ-φιλτράρισμα βάσει προϋπολογισμού και τοποθεσίας για τη μείωση της κατανάλωσης tokens, ενώ το Frontend εκτελεί έναν τοπικό ευρετικό αλγόριθμο βαθμολόγησης (Heuristic Scoring) βάσει των προτιμήσεων του προφίλ του χρήστη. Η προσέγγιση αυτή βελτιστοποιεί τους υπολογιστικούς πόρους και εκμηδενίζει την υστέρηση (latency).
- **Συγχρονισμένη Κοινωνική Διάδραση (Real-Time Social Discovery):** Μέσω της τεχνολογίας WebSockets (Socket.IO), η εφαρμογή μετατρέπεται από ένα στατικό εργαλείο ψηφοφορίας σε μια ζωντανή πλατφόρμα. Η ενσωμάτωση δωματίων συνομιλίας (Live Chat) ανά δραστηριότητα, η παρακολούθηση της κατάστασης σύνδεσης των μελών (online status) και ο ακαριαίος συγχρονισμός της ομοφωνίας (Match) επιλύουν το πρόβλημα της ομαδικής αδράνειας αλγοριθμικά.

- **Προηγμένη Εμπειρία Χρήστη (Advanced UI/UX Engineering):** Εισάγονται καινοτόμα στοιχεία, όπως η αυτόματη αλλαγή χρωματικών θεμάτων βάσει της ώρας (time-based theming), η δυναμική δρομολόγησης μέσω AI με ανάλυση κρυφών συνδέσμων (regex parsing) και η διασφάλιση της ακεραιότητας των δεδομένων με διαδραστικούς μηχανισμούς επιβεβαίωσης (custom toasts).

1.4 Δομή της Διπλωματικής Εργασίας

Η δομή της παρούσας διπλωματικής εργασίας διαμορφώνεται ως εξής:

- **Κεφάλαιο 1:** Γίνεται εισαγωγή στο υπό εξέταση θέμα, παρατίθενται τα κίνητρα, οι στόχοι και αναλύεται η επιστημονική και πρακτική συνεισφορά του έργου.
- **Κεφάλαιο 2:** Πραγματοποιείται ανασκόπηση της σχετικής βιβλιογραφίας γύρω από τα ατομικά και ομαδικά συστήματα συστάσεων, την παιχνιδοποίηση και τους συνομιλιακούς πράκτορες, καθώς και επισκόπηση των υφιστάμενων συστημάτων της αγοράς.
- **Κεφάλαιο 3:** Παρουσιάζεται το θεωρητικό υπόβαθρο και αναλύονται διεξοδικά οι τεχνολογίες υλοποίησης που υιοθετήθηκαν, συμπεριλαμβανομένων της React.js, του Node.js/Express.js, της MySQL, του πρωτοκόλλου WebSockets (Socket.IO), της ασφάλειας (JWT & bcrypt), του Groq API (LLaMA 3.3) και των εργαλείων Postman και Navicat.
- **Κεφάλαιο 4:** Περιγράφεται η μεθοδολογία σχεδιασμού της πολυεπίπεδης αρχιτεκτονικής, η μοντελοποίηση της βάσης δεδομένων, οι μηχανισμοί Prompt Engineering (pre-filtering, sliding window) και η ροή του προτεινόμενου αλγορίθμου ομοφωνίας (matching algorithm) σε πραγματικό χρόνο.
- **Κεφάλαιο 5:** Αναλύεται η πρακτική υλοποίηση του συστήματος και η διεπαφή χρήστη (UI/UX), παρουσιάζοντας λειτουργίες όπως το Lobby Radar, το Live Chat, το Time-based Theming, τα Match Results και το Surprise Dice, ενώ περιγράφεται και η διαδικασία ανάπτυξης και φιλοξενίας στο υπολογιστικό νέφος (Render Cloud Deployment).

- **Κεφάλαιο 6:** Συνοψίζεται το έργο, εξάγονται τα τελικά συμπεράσματα γύρω από τα τεχνικά επιτεύγματα της υλοποίησης και προτείνονται συγκεκριμένες κατευθύνσεις για μελλοντικές επεκτάσεις του συστήματος.
- **Κεφάλαιο 7:** Παρατίθεται η συγκεντρωτική ακαδημαϊκή βιβλιογραφία, οι πηγές των τεχνολογικών εργαλείων και το παράρτημα αδειοδότησης των πολυμέσων.

Κεφάλαιο 2: Βιβλιογραφική Ανασκόπηση και Σχετικές Εργασίες

2.1 Εισαγωγή

Στο παρόν κεφάλαιο παρατίθεται το θεωρητικό υπόβαθρο και η ανασκόπηση της σχετικής βιβλιογραφίας γύρω από τους επιστημονικούς τομείς που συνθέτουν την προτεινόμενη εφαρμογή. Δεδομένου ότι ο πυρήνας του συστήματος βασίζεται στην παροχή ευφών προτάσεων (suggestions), η έρευνα εστιάζει πρωτίστως στα **Συστήματα Συστάσεων (Recommendation Systems)**, εξετάζοντάς τα τόσο σε ατομικό όσο και σε ομαδικό επίπεδο (Group Recommendation Systems). Επιπροσθέτως, αναλύεται η αξιοποίηση της Τεχνητής Νοημοσύνης για τη βελτιστοποίηση της αναζήτησης μέσω της επίγνωσης πλαισίου (context-awareness), καθώς και οι σύγχρονοι μηχανισμοί διεπαφής χρήστη (όπως ο πολυτροπικός χειρισμός και το swiping) που επιταχύνουν τη διαδικασία λήψης αποφάσεων. Τέλος, πραγματοποιείται μια συνοπτική επισκόπηση των υφιστάμενων εμπορικών εφαρμογών, προκειμένου να αναδειχθούν τα λειτουργικά κενά τα οποία καλύπτει η παρούσα Full-Stack υλοποίηση.

2.2 Συστήματα Συστάσεων (Recommendation Systems)

Η ραγδαία αύξηση της διαθέσιμης πληροφορίας στον παγκόσμιο ιστό έχει καταστήσει επιτακτική τη χρήση συστημάτων ικανών να διηθούν (φιλτράρουν) τον τεράστιο όγκο δεδομένων, αναδεικνύοντας το περιεχόμενο που είναι υψίστης σημασίας για τον εκάστοτε

χρήστη. Τα Συστήματα Συστάσεων (ΣΣ) αποτελούν υποκατηγορία των συστημάτων φιλτραρίσματος πληροφορίας και εφαρμόζονται ευρέως σε πληθώρα τομέων, από το ηλεκτρονικό εμπόριο έως την εύρεση τοποθεσιών και δραστηριοτήτων [5].

Στη σύγχρονη βιβλιογραφία, οι προσεγγίσεις των Συστημάτων Συστάσεων διακρίνονται παραδοσιακά σε τρεις κύριες κατηγορίες:

- **Φιλτράρισμα βάσει περιεχομένου (Content-based Filtering):** Το σύστημα προτείνει αντικείμενα (π.χ. δραστηριότητες, εστιατόρια) με παρόμοια χαρακτηριστικά σε σχέση με εκείνα που έχει επιλέξει ή αξιολογήσει θετικά ο χρήστης στο παρελθόν. Η μεθοδολογία αυτή βασίζεται στα ενδογενή χαρακτηριστικά (features) των αντικειμένων, όπως η τοποθεσία, η κατηγορία δραστηριότητας και το κόστος.
- **Συνεργατικό Φιλτράρισμα (Collaborative Filtering):** Η συγκεκριμένη μέθοδος εδράζεται στη συμπεριφορά του ευρύτερου συνόλου των χρηστών. Εάν δύο χρήστες παρουσιάζουν ισχυρή συσχέτιση στο ιστορικό των επιλογών τους, το σύστημα συνάγει συμπεράσματα και προτείνει στον έναν χρήστη επιλογές που έχουν ήδη ικανοποιήσει τον άλλον [5].
- **Υβριδικά Συστήματα (Hybrid Systems):** Πρόκειται για συστήματα τα οποία συνενώνουν τις προαναφερθείσες μεθόδους, με στόχο την εξάλειψη των εγγενών μειονεκτημάτων τους. Χαρακτηριστικό παράδειγμα αποτελεί η αντιμετώπιση του προβλήματος της "ψυχρής εκκίνησης" (cold start problem), το οποίο παρατηρείται όταν ένας νέος χρήστης δεν διαθέτει επαρκή δεδομένα στο προφίλ του.

Στο πλαίσιο της εφαρμογής Pyxis, η παραγωγή προτάσεων εξελίσσει το παραδοσιακό στατικό φιλτράρισμα, μετατοπίζοντας μέρος της υπολογιστικής ισχύος στην πλευρά του πελάτη (Edge Computing). Αυτό επιτυγχάνεται μέσω της ανάπτυξης ενός τοπικού ευρετικού αλγορίθμου βαθμολόγησης (Heuristic Scoring) ο οποίος αξιολογεί άμεσα στον browser τις δραστηριότητες βάσει των δυναμικών προτιμήσεων του προφίλ του χρήστη.

2.3 Συστήματα Συστάσεων για Ομάδες (Group Recommendation Systems)

Ενώ τα παραδοσιακά Συστήματα Συστάσεων εστιάζουν στη βελτιστοποίηση της εμπειρίας ενός μεμονωμένου χρήστη, πολλά σύγχρονα σενάρια εφαρμογής –όπως η επιλογή ενός εστιατορίου ή μιας κοινής ψυχαγωγικής δραστηριότητας– προϋποθέτουν την ικανοποίηση των

προτιμήσεων μιας ολόκληρης ομάδας. Η ανάγκη αυτή οδήγησε στην επιστημονική ανάπτυξη των Συστημάτων Συστάσεων για Ομάδες (Group Recommendation Systems - GRS)[2], [3]

Η θεμελιώδης πρόκληση σε ένα σύστημα GRS, η οποία στην εφαρμογή Pyxis αντιμετωπίζεται μέσω της λειτουργίας των ψηφιακών δωματίων (Lobbies), έγκειται στον βέλτιστο συνδυασμό των ατομικών προτιμήσεων των μελών, με σκοπό την παραγωγή μιας αποδεκτής κοινής λίστας προτάσεων[4].

Στη βιβλιογραφία διακρίνονται δύο κυρίαρχες κλασικές στρατηγικές:

Συνένωση Προφίλ (Profile Aggregation): Το σύστημα υπολογίζει ένα "εικονικό προφίλ" που αντιπροσωπεύει τη μέση συνισταμένη των προτιμήσεων ολόκληρης της ομάδας. Στη συνέχεια, παράγει προτάσεις βασισμένες αποκλειστικά σε αυτό το ενοποιημένο προφίλ.

Συνένωση Προτάσεων (Recommendation Aggregation): Το σύστημα εξάγει αρχικά ατομικές προτάσεις για κάθε μέλος ξεχωριστά. Έπειτα, εφαρμόζει στρατηγικές σύνθεσης, όπως η μέθοδος του «μέσου όρου» ή η μέθοδος «ελαχιστοποίησης της δυσaréσκειας» (least misery strategy), προκειμένου να συνδυάσει τις ατομικές λίστες σε μια τελική, ομαδική λίστα επιλογών[1].

Το Pyxis εισάγει μια τρίτη, σύγχρονη εναλλακτική, η οποία ορίζεται ως **Δυναμική Συνδρομητική Συναίνεση** (Dynamic Conversational Consensus). Αντί για στατικό συνδυασμό σταθερών πινάκων, το σύστημα επιτρέπει στην ομάδα να συνδιαμορφώσει τις προτάσεις μέσω αμφίδρομης επικοινωνίας και WebSockets, καθιστώντας τον σχεδιασμό μηχανισμών συναίνεσης (consensus) μια ζωντανή ροή εργασίας στο πεδίο της Αλληλεπίδρασης Ανθρώπου-Υπολογιστή (Human-Computer Interaction - HCI).

2.4 Μηχανισμοί Αλληλεπίδρασης και Παιχνιδοποίηση (Gamification & Swiping)

Η εμπειρία χρήστη (User Experience - UX) διαδραματίζει καθοριστικό ρόλο στην υιοθέτηση και την εμπορική επιτυχία μιας σύγχρονης διαδικτυακής εφαρμογής. Προκειμένου να μετριαστεί η κόπωση του χρήστη από την υπερφόρτωση πληροφοριών, πληθώρα συστημάτων ενσωματώνει στοιχεία παιχνιδιού (Gamification) στις διεπαφές τους [11]. Ο στόχος της παιχνιδοποίησης είναι η αύξηση της δέσμευσης, γεγονός που επιβεβαιώνεται από εμπειρικές μελέτες [12].

Ένας από τους πλέον διαδεδομένους μηχανισμούς διεπαφής είναι η λογική της γρήγορης ταξινόμησης μέσω σάρωσης (Swiping). Παρότι ο εν λόγω μηχανισμός έγινε ευρέως γνωστός μέσω εφαρμογών γνωριμιών (dating apps) [10], πλέον αξιοποιείται εκτενώς σε ετερόκλητα περιβάλλοντα (π.χ. ηλεκτρονικό εμπόριο, εύρεση εργασίας) χάρη στα ακόλουθα πλεονεκτήματα:

- **Μείωση Γνωσιακού Φόρτου (Cognitive Load):** Ο χρήστης καλείται να λάβει μια απλή δυαδική απόφαση (Θετική/Αρνητική) για ένα αντικείμενο τη φορά, αποφεύγοντας την ταυτόχρονη επεξεργασία εκτενών λιστών.
- **Ταχύτητα και Αμεσότητα:** Η λήψη αποφάσεων καθίσταται διαισθητική, μετασχηματίζοντας τη διαδικασία της αναζήτησης σε μια διασκεδαστική και αβίαστη εμπειρία.

Στο πλαίσιο των Συστημάτων Συστάσεων για ομάδες, όπως υλοποιείται στα Lobbies της εφαρμογής Pyxis, ο μηχανισμός swiping λειτουργεί ως ένα άτυπο αλλά εξαιρετικά αποτελεσματικό σύστημα ψηφοφορίας. Η τελική απόφαση (Match) επιτυγχάνεται αποκλειστικά όταν όλα τα μέλη της ομάδας αλληλεπιδράσουν θετικά με την ίδια πρόταση, επιλύοντας αλγοριθμικά το πρόβλημα της επίτευξης συναίνεσης.

2.5 Η Συμβολή της Τεχνητής Νοημοσύνης (AI) και των Chatbots

Οι πρόσφατες αλματώδεις εξελίξεις στον τομέα της Επεξεργασίας Φυσικής Γλώσσας (Natural Language Processing - NLP) και των Μεγάλων Γλωσσικών Μοντέλων (Large Language Models - LLMs) [8], [9] έχουν καταστήσει εφικτή την ενσωμάτωση ευφύων Συνομιλιακών Πρακτόρων (Conversational Agents ή Chatbots) στα σύγχρονα πληροφοριακά συστήματα [7].

Η εφαρμογή της Τεχνητής Νοημοσύνης για την παραγωγή προτάσεων πλεονεκτεί έναντι των παραδοσιακών, στατικών αλγορίθμων [6], καθώς επιτρέπει τη δυναμική και αμφίδρομη αλληλεπίδραση. Ειδικότερα:

- **Εξαγωγή Προθέσεων (Intent Extraction):** Ένα AI chatbot έχει την ικανότητα να αναλύει και να κατανοεί περίπλοκες απαιτήσεις των χρηστών μέσω ρεαλιστικού διαλόγου σε φυσική γλώσσα (π.χ. «Επιθυμούμε μια οικονομική δραστηριότητα στο κέντρο της πόλης για 4 άτομα»).

- **Προσαρμοστικότητα σε Πραγματικό Χρόνο:** Σε περίπτωση που η ομάδα απορρίπτει τις αρχικές συστάσεις, το μοντέλο AI δύναται να αναπροσαρμόσει άμεσα τα κριτήρια αναζήτησης και να συνθέσει νέες, εναλλακτικές λύσεις.

Η σύγχρονη βιβλιογραφία αναδεικνύει τη σημασία της **επίγνωσης πλαισίου (context-aware NLP)**. Στο Pyxis, το μοντέλο LLaMA 3.3 δεν βασίζεται μόνο στο κείμενο εισόδου, αλλά εμπλουτίζεται δυναμικά με περιβαλλοντικές μεταβλητές (πραγματικός καιρός μέσω OpenWeather API) και περιορίζεται αυστηρά σε δομημένη μορφή εξόδου (structured JSON parsing), γεφυρώνοντας το χάσμα μεταξύ της ελεύθερης συζήτησης και της προγραμματιστικής διαχείρισης δεδομένων.

2.6 Υφιστάμενες Εφαρμογές και η Καινοτομία του Προτεινόμενου Συστήματος

Μια κριτική επισκόπηση της τρέχουσας αγοράς καταδεικνύει την κυριαρχία εδραιωμένων πλατφορμών αναζήτησης εμπειριών (όπως τα TripAdvisor, Yelp και Meetup). Αν και οι εν λόγω πλατφόρμες διαθέτουν ογκώδεις βάσεις δεδομένων και προηγμένους αλγορίθμους συστάσεων, συχνά υστερούν ως προς την υποστήριξη της ομαδικής δυναμικής. Ο χρήστης συνήθως εκτελεί την αναζήτηση ατομικά και, κατόπιν, αναγκάζεται να μοιραστεί χειροκίνητα τις επιλογές του (μέσω εξωτερικών εφαρμογών ανταλλαγής μηνυμάτων) με τα υπόλοιπα μέλη, προκειμένου να καταλήξουν σε συμφωνία. Από την άλλη πλευρά, οι εφαρμογές που αξιοποιούν τον γρήγορο μηχανισμό swiping και matching προσανατολίζονται σχεδόν αποκλειστικά σε διαπροσωπικές γνωριμίες, παραβλέποντας παντελώς τον τομέα των ομαδικών δραστηριοτήτων αναψυχής.

Η Συνεισφορά του Συστήματος Pyxis: Η παρούσα διπλωματική εργασία έρχεται να καλύψει το προαναφερθέν βιβλιογραφικό και πρακτικό κενό, προτείνοντας μια ολοκληρωμένη Full-Stack λύση η οποία συνδυάζει οργανικά τέσσερις διακριτούς τεχνικούς άξονες:

- Την παροχή υποδομής για τη δημιουργία κλειστών ή ανοιχτών ψηφιακών δωματίων (Lobbies / Lobby Radar), επιτρέποντας τη συγχρονισμένη αναζήτηση.
- Την ενσωμάτωση αμφίδρομης επικοινωνίας πραγματικού χρόνου (Socket.IO Live Chat) για την άμεση ανταλλαγή μηνυμάτων και online status μελών.
- Την υιοθέτηση του μηχανισμού Swiping με πολυτροπική είσοδο ως εργαλείου ομαδικής ψηφοφορίας για την άμεση ανάδειξη της επικρατέστερης επιλογής (Match).

- Την ενίσχυση της διαδικασίας μέσω Τεχνητής Νοημοσύνης (Context-Aware AI Chatbot), η οποία αναλύει τον καιρό και τις προτιμήσεις για την ακαριαία τροφοδότηση της ομάδας με δομημένα JSON δεδομένα.

Μέσω αυτής της καινοτόμου αρχιτεκτονικής, το σύστημα μετασχηματίζει τη χρονοβόρα και δυσχερή διαδικασία οργάνωσης μιας ομαδικής εξόδου σε μια σύγχρονη, άμεση και ευχάριστη ψηφιακή ροή εργασίας.

Κεφάλαιο 3: Θεωρητικό Υπόβαθρο και Τεχνολογίες Υλοποίησης

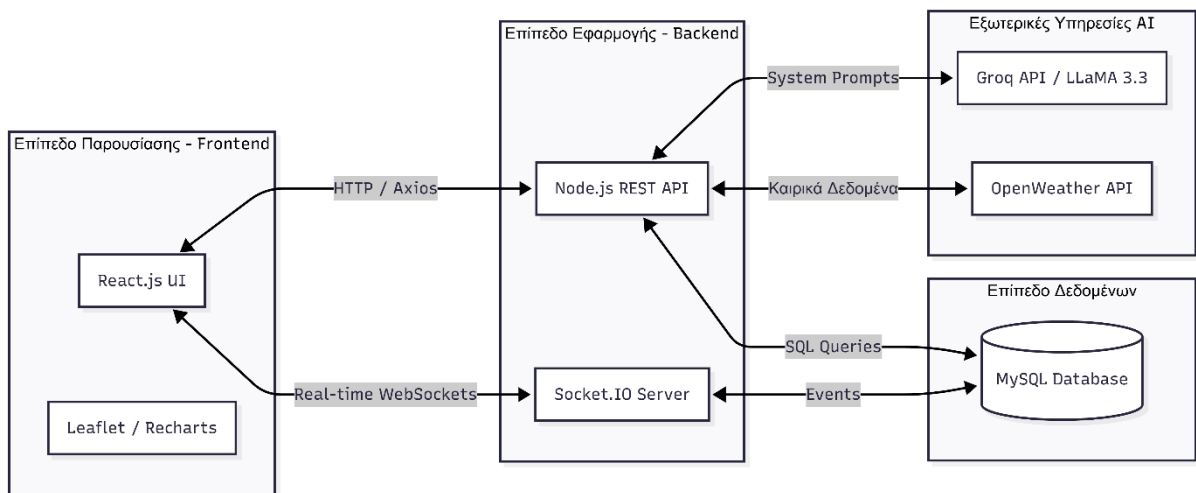
3.1 Εισαγωγή

Η επιτυχής υλοποίηση ενός σύγχρονου πληροφοριακού συστήματος, το οποίο απαιτεί συγχρονισμό σε πραγματικό χρόνο, διαχείριση σχεσιακών δεδομένων και ενσωμάτωση μοντέλων Τεχνητής Νοημοσύνης, εξαρτάται άμεσα από την επιλογή των κατάλληλων τεχνολογιών. Στο παρόν κεφάλαιο αναλύεται το θεωρητικό υπόβαθρο και επεξηγούνται οι τεχνολογικοί πυλώνες στους οποίους βασίστηκε η ανάπτυξη της εφαρμογής Pyxis. Δίνεται έμφαση στα πλεονεκτήματα κάθε τεχνολογίας, καθώς και στον τρόπο με τον οποίο αυτές συνεργάζονται για τη δημιουργία ενός κλιμακώσιμου, ασφαλούς και αποδοτικού συστήματος.

3.2 Αρχιτεκτονική Πελάτη-Εξυπηρετητή (Client-Server Architecture)

Η εφαρμογή Pyxis ακολουθεί το κλασικό αρχιτεκτονικό πρότυπο Πελάτη-Εξυπηρετητή (Client-Server). Το πρότυπο αυτό βασίζεται στον διαχωρισμό των αρμοδιοτήτων (Separation of Concerns) μεταξύ του επιπέδου παρουσίασης (Frontend/Client) και του επιπέδου επιχειρηματικής λογικής και δεδομένων (Backend/Server) [17].

Ο Πελάτης είναι υπεύθυνος αποκλειστικά για τη διαχείριση της διεπαφής χρήστη, την καταγραφή των αλληλεπιδράσεων (π.χ. swiping) και την προβολή των αποτελεσμάτων. Ο Εξυπηρετητής αναλαμβάνει την επεξεργασία των αιτημάτων, την επικοινωνία με τη βάση δεδομένων, την αυθεντικοποίηση των χρηστών και τη διασύνδεση με εξωτερικές υπηρεσίες AI. Η επικοινωνία μεταξύ των δύο επιπέδων πραγματοποιείται ασύγχρονα μέσω του πρωτοκόλλου HTTP (REST API) και μόνιμων αμφίδρομων συνδέσεων (WebSockets), διασφαλίζοντας τη χαμηλή υστέρηση (latency) και τη βέλτιστη αξιοποίηση των πόρων του συστήματος [16].



Εικόνα 3.1: Αρχιτεκτονικό διάγραμμα του συστήματος Pyxis και των εξωτερικών υπηρεσιών (3-Tier Architecture).

3.3 Τεχνολογίες Διεπαφής Χρήστη (Frontend) - React.js και Βιβλιοθήκες Επέκτασης

Για την ανάπτυξη της εφαρμογής στην πλευρά του πελάτη επιλέχθηκε η βιβλιοθήκη React.js, η οποία αναπτύσσεται από τη Meta και αποτελεί ένα από τα πιο διαδεδομένα εργαλεία για τη δημιουργία Μονοσέλιδων Εφαρμογών (Single Page Applications - SPAs) [22]. Η επιλογή αυτή βασίστηκε σε τρία θεμελιώδη χαρακτηριστικά της βιβλιοθήκης:

- **Συστατικά Στοιχεία (Component-Based Architecture):** Η διεπαφή χρήστη διασπάται σε αυτόνομα, επαναχρησιμοποιήσιμα κομμάτια κώδικα (Components), γεγονός που διευκολύνει τη συντήρηση, τον έλεγχο και την επέκταση της εφαρμογής (π.χ. ξεχωριστά συστατικά για το Chatbot, την κάρτα δραστηριότητας και το μενού πλοήγησης).
- **Εικονικό DOM (Virtual DOM):** Το React.js δεν ενημερώνει απευθείας το πραγματικό DOM του φυλλομετρητή σε κάθε αλλαγή κατάστασης. Αντίθετα, δημιουργεί μια ελαφριά αναπαράστασή του στη μνήμη, υπολογίζει τις ελάχιστες απαιτούμενες αλλαγές μέσω ενός αλγορίθμου διαφοροποίησης (reconciliation) και ενημερώνει επιλεκτικά το UI [21]. Η διεργασία αυτή είναι κρίσιμη για την ομαλή ροή της λειτουργίας swiping, όπου οι κάρτες πρέπει να εναλλάσσονται ακαριαία χωρίς καθυστερήσεις στην οθόνη.

- **Δηλωτικός Προγραμματισμός (Declarative UI):** Ο κώδικας περιγράφει πώς πρέπει να μοιάζει η διεπαφή σε μια συγκεκριμένη χρονική στιγμή βάσει των δεδομένων της (state), καθιστώντας τη συμπεριφορά της εφαρμογής πιο προβλέψιμη και εύκολη στην αποσφαλμάτωση.

Για την ασύγχρονη επικοινωνία (HTTP Requests) με το Backend (REST API) αξιοποιήθηκε η διαδεδομένη βιβλιοθήκη **Axios**, η οποία διευκολύνει την αποτελεσματική, γρήγορη και ασφαλή ανταλλαγή δεδομένων μεταξύ της διεπαφής και του εξυπηρετητή.

Πέραν του πυρήνα της React, για την υλοποίηση των εξειδικευμένων και διαδραστικών λειτουργιών της εφαρμογής Pyxis, ενσωματώθηκαν οι εξής κομβικές βιβλιοθήκες:

- **Μηχανισμός Swiping & Οπτικά Εφέ (Animations):** Η διαδραστική λειτουργία της σάρωσης καρτών (swiping) υλοποιήθηκε μέσω **ιδιοκατασκευασμένου (native) κώδικα JavaScript [15]**. Ο μηχανισμός διαχειρίζεται απευθείας τα γεγονότα αφής και μετακίνησης (onTouchStart, onTouchEnd, onMouseDown, onMouseUp), υπολογίζοντας το διάνυσμα της μετατόπισης στον οριζόντιο άξονα προκειμένου να αναγνωρίσει την πρόθεση ψήφου του χρήστη. Παράλληλα, για την ομαλή μετάβαση των στοιχείων διεπαφής (όπως η εμφάνιση της οθόνης επιτυχούς συμφωνίας - Match) αξιοποιήθηκαν CSS Keyframe Animations (popIn και fadeIn), εξασφαλίζοντας υψηλό ρυθμό ανανέωσης χωρίς τη χρήση περιττών εξωτερικών εξαρτήσεων.
- **Διαδραστικοί Χάρτες:** Προκειμένου οι χρήστες να μπορούν να οπτικοποιούν τις τοποθεσίες των προτεινόμενων δραστηριοτήτων, το σύστημα ενσωματώνει διαδραστικούς χάρτες με τη βοήθεια των βιβλιοθηκών leaflet, react-leaflet και react-leaflet-cluster για την ομαδοποίηση των σημείων ενδιαφέροντος [31].
- **Οπτικοποίηση Στατιστικών Δεδομένων:** Για την προβολή στατιστικών στοιχείων και ιστορικού (π.χ. συχνότητα επιλογών, ποσοστά matches) αξιοποιείται η βιβλιοθήκη recharts, η οποία παρέχει δυναμικά και παραμετροποιήσιμα γραφήματα [32].
- **Σχεδιασμός UI και Ειδοποιήσεις:** Η δομή και το αποκριτικό (responsive) στυλ της εφαρμογής βασίστηκε στο πλαίσιο Bootstrap 5. Για τη βελτίωση της Εμπειρίας Χρήστη (UX) κατά τις επιτυχείς ή αποτυχημένες ενέργειες (π.χ. έλεγχος διαπιστευτηρίων, σφάλματα), επιλέχθηκαν οι βιβλιοθήκες sweetalert2 και react-hot-toast που παρέχουν καλαίσθητες αναδυόμενες ειδοποιήσεις.

3.3.1 Πειραματική Αξιολόγηση Εργαλείων Διεπαφής (Swiping)

Αξίζει να σημειωθεί ότι κατά τα αρχικά στάδια της πρωτοτυποποίησης (prototyping) της εφαρμογής, πραγματοποιήθηκε εκτενής πειραματισμός και δοκιμή με την εξειδικευμένη βιβλιοθήκη react-tinder-card, η οποία αποτελεί διαδεδομένη λύση για την προσομοίωση διεπαφών τύπου Tinder. Παρότι η βιβλιοθήκη αυτή παρείχε βασική λειτουργικότητα, απορρίφθηκε τελικά κατά τη φάση της αρχιτεκτονικής αναθεώρησης για δύο κύριους λόγους. Αφενός, εμφάνιζε περιορισμούς στην ομαλή ενσωμάτωση του πολυτροπικού χειρισμού (multi-modal input) που απαιτούσε το Pyxis, όπως η ταυτόχρονη υποστήριξη "double-tap" και φυσικών πλήκτρων πλοήγησης (Arrow Keys). Αφετέρου, η ανάπτυξη custom (native) κώδικα επέτρεψε την πλήρη βελτιστοποίηση του μεγέθους της εφαρμογής (bundle size) στο Frontend, προσφέροντας απόλυτο έλεγχο στις ροές δεδομένων των ψήφων προς το Backend και εξαλείφοντας κάθε πιθανότητα ασυμβατότητας.

3.4 Περιβάλλον Backend - Node.js και Express.js

Η επιχειρηματική λογική της εφαρμογής Pyxis αναπτύχθηκε στο περιβάλλον εκτέλεσης Node.js, το οποίο επιτρέπει την εκτέλεση κώδικα JavaScript στην πλευρά του διακομιστή [14]. Το Node.js βασίζεται στη μηχανή V8 της Google και χαρακτηρίζεται από ένα ασύγχρονο, οδηγημένο από συμβάντα (event-driven) μοντέλο εισόδου/εξόδου (Non-blocking I/O). Μέσω του Μηχανισμού Βρόχου Συμβάντων (Event Loop), ο εξυπηρετητής μπορεί να διαχειρίζεται χιλιάδες ταυτόχρονες συνδέσεις χρηστών χωρίς τη δημιουργία ξεχωριστών νημάτων (threads) για κάθε αίτημα, καθιστώντας τον ιδανικό για εφαρμογές πραγματικού χρόνου.

Πάνω στο Node.js χρησιμοποιήθηκε το πλαίσιο (framework) Express.js, το οποίο παρέχει μια μινιμαλιστική και ευέλικτη υποδομή για τη δημιουργία διαδικτυακών εφαρμογών και APIs [23]. Το Express.js διευκολύνει τη διαχείριση της δρομολόγησης (routing) των αιτημάτων HTTP, την ενσωμάτωση ενδιάμεσου λογισμικού (middleware) για τον έλεγχο της ασφάλειας και την οργάνωση των επιμέρους λειτουργικών ενοτήτων του συστήματος.

3.5 Σχεσιακά Συστήματα Διαχείρισης Βάσεων Δεδομένων (RDBMS) - MySQL

Για την αποθήκευση και διαχείριση των μόνιμων δεδομένων της εφαρμογής (στοιχεία χρηστών, δωμάτια, ιστορικό, κριτικές) επιλέχθηκε η MySQL, ένα από τα πιο αξιόπιστα Σχεσιακά Συστήματα Διαχείρισης Βάσεων Δεδομένων (RDBMS). Η επιλογή της MySQL βασίστηκε στην ανάγκη για αυστηρή δομή δεδομένων και διατήρηση της ακεραιότητας των συσχετίσεων (Referential Integrity) μέσω ξένων κλειδιών (Foreign Keys) [19], [20].

Επιπλέον, η MySQL υποστηρίζει πλήρως τις ιδιότητες ACID (Atomicity, Consistency, Isolation, Durability), διασφαλίζοντας ότι οι συναλλαγές στη βάση (όπως η ταυτόχρονη καταγραφή swiping από πολλούς χρήστες στο ίδιο δωμάτιο) θα εκτελούνται με απόλυτη ασφάλεια και χωρίς αλλοίωση των δεδομένων. Για τη βελτιστοποίηση των επιδόσεων και τη μείωση του χρόνου απόκρισης, υιοθετήθηκε η τεχνική της Δεξαμενής Συνδέσεων (Connection Pooling), η οποία επαναχρησιμοποιεί ενεργές συνδέσεις με τη βάση, αποφεύγοντας το υπολογιστικό κόστος της δημιουργίας νέας σύνδεσης σε κάθε HTTP αίτημα.

Ειδικότερα, χρησιμοποιήθηκε η βιβλιοθήκη mysql2/promise, η οποία επιτρέπει την ασύγχρονη εκτέλεση ερωτημάτων (async/await), διευκολύνοντας τη διαχείριση της ροής του κώδικα. Επιπρόσθετα, για τη διασφάλιση της κρυπτογραφημένης μεταφοράς δεδομένων στο υπολογιστικό νέφος, έχει υλοποιηθεί δυναμική ενεργοποίηση πιστοποιητικών SSL όταν η εφαρμογή εκτελείται σε περιβάλλον παραγωγής (production).

3.6 Επικοινωνία σε Πραγματικό Χρόνο και WebSockets (Socket.IO)

Το παραδοσιακό πρωτόκολλο HTTP λειτουργεί αποκλειστικά με το μοντέλο Αιτήματος-Απόκρισης (Request-Response), όπου ο πελάτης πρέπει πάντα να ξεκινήσει την επικοινωνία. Για μια εφαρμογή όπως το Pyxis, όπου τα μέλη ενός δωματίου (Lobby) πρέπει να βλέπουν σε πραγματικό χρόνο ποιοι συνδέθηκαν, να συνομιλούν ζωντανά (Live Chat) ή να ενημερώνονται ακαριαία όταν επιτευχθεί μια συμφωνία (Match), η αποκλειστική χρήση του HTTP θα απαιτούσε συνεχή ερωτήματα (Short/Long Polling), επιβαρύνοντας υπερβολικά τον εξυπηρετητή και το δίκτυο.

Για την επίλυση αυτού του προβλήματος, υιοθετήθηκε το πρωτόκολλο WebSockets μέσω της δημοφιλούς βιβλιοθήκης **Socket.IO** [26]. Τα WebSockets επιτρέπουν τη δημιουργία μιας μόνιμης, αμφίδρομης (full-duplex) σύνδεσης χαμηλού επιπέδου πάνω από μία μοναδική σύνδεση TCP μεταξύ πελάτη και εξυπηρετητή. Η τεχνολογία αυτή βασίζεται σε

αρχιτεκτονική καθοδηγούμενη από γεγονότα (event-driven architecture), επιτρέποντας στο Backend να εκπέμπει (emit) ακαριαία ενημερώσεις χωρίς καθυστέρηση. Η Socket.IO παρέχει επιπλέον δυνατότητες υψηλού επιπέδου, όπως:

- **Δωμάτια (Rooms):** Επιτρέπει την οργάνωση των συνδέσεων σε αυτόνομα κανάλια. Κάθε Lobby της εφαρμογής αντιστοιχεί σε ένα ψηφιακό δωμάτιο της Socket.IO, διευκολύνοντας την ομαδική ανταλλαγή μηνυμάτων και τον συγχρονισμό της τρέχουσας κατάστασης των χρηστών.
- **Εκπομπή Συμβάντων (Broadcasting):** Όταν ένας χρήστης εκτελεί μια ενέργεια (π.χ. αποστολή μηνύματος ή καταχώρηση ψήφου), ο εξυπηρετητής ενημερώνει ακαριαία όλα τα υπόλοιπα μέλη του συγκεκριμένου δωματίου με ελάχιστο δικτυακό φόρτο.

3.7 Μηχανισμοί Ασφάλειας, Ταυτοποίησης και Κρυπτογράφησης (JWT & bcrypt)

Η ασφάλεια των προσωπικών δεδομένων και η διασφάλιση της πρόσβασης στις υπηρεσίες αποτέλεσαν προτεραιότητα κατά τον σχεδιασμό. Υλοποιήθηκαν δύο βασικοί μηχανισμοί:

- **Κρυπτογράφηση Κωδικών (bcrypt):** Οι κωδικοί πρόσβασης των χρηστών δεν αποθηκεύονται ποτέ σε μορφή απλού κειμένου στη βάση δεδομένων. Χρησιμοποιείται η βιβλιοθήκη bcrypt, η οποία εφαρμόζει έναν ισχυρό αλγόριθμο μονοδρομικής κρυπτογράφησης (hashing) σε συνδυασμό με την τεχνική προσθήκης τυχαίων χαρακτήρων (Salting) και επαναλαμβανόμενων κύκλων υπολογισμού (Key Stretching). Η προσέγγιση αυτή προστατεύει το σύστημα από επιθέσεις τύπου Brute Force και Rainbow Tables.
- **Αυθεντικοποίηση χωρίς Κατάσταση (JWT) και Ενδιάμεσο Λογισμικό (Middleware):** Για τη διατήρηση της σύνδεσης των χρηστών χρησιμοποιούνται JSON Web Tokens (JWT) [24]. Μετά την επιτυχή είσοδο (Login), ο εξυπηρετητής εκδίδει ένα ψηφιακά υπογεγραμμένο token που περιέχει την ταυτότητα του χρήστη και λήγει σε συγκεκριμένο χρονικό διάστημα (π.χ. 12 ώρες). Ο πελάτης αποθηκεύει το token και το αποστέλλει στην κεφαλίδα (Authorization Header) κάθε επόμενου αιτήματος.

Για την προστασία των ευαίσθητων τελικών σημείων (endpoints), αναπτύχθηκε ένα εξειδικευμένο ενδιάμεσο λογισμικό (**auth middleware**). Το λογισμικό αυτό αναχαιτίζει τα HTTP αιτήματα και επαληθεύει την εγκυρότητα της υπογραφής του token μέσω του

μυστικού κλειδιού (JWT_SECRET). Σε περίπτωση απουσίας του token επιστρέφει σφάλμα μη εξουσιοδοτημένης πρόσβασης (401 Unauthorized), ενώ αν το token είναι παραβιασμένο ή ληγμένο, απορρίπτει το αίτημα (403 Forbidden). Η προσέγγιση αυτή επιτρέπει στον εξυπηρετητή να παραμένει "χωρίς κατάσταση" (stateless), βελτιώνοντας την ταχύτητα και την κλιμακωσιμότητα του συστήματος.

3.8 Μεγάλα Γλωσσικά Μοντέλα (LLMs) και το Μοντέλο LLaMA 3.3

Η καινοτομία της εφαρμογής Pyxis βασίζεται στην ενσωμάτωση Generative AI για την παραγωγή εναλλακτικών δραστηριοτήτων. Χρησιμοποιείται το Μεγάλο Γλωσσικό Μοντέλο LLaMA 3.3 (Large Language Model Meta AI), το οποίο προσφέρει εξαιρετική ικανότητα κατανόησης και παραγωγής φυσικής γλώσσας, υψηλή συνεκτικότητα και προσαρμοστικότητα σε εξειδικευμένες οδηγίες (system prompts).

Η διασύνδεση με το μοντέλο πραγματοποιείται μέσω του Groq API [27]. Η Groq χρησιμοποιεί μια πρωτοποριακή αρχιτεκτονική επεξεργαστών LPU (Language Processing Units), η οποία επιτρέπει την εκτέλεση LLMs με εξαιρετικά υψηλή ταχύτητα παραγωγής tokens ανά δευτερόλεπτο. Η επιλογή αυτή εξασφαλίζει ότι το Chatbot της εφαρμογής απαντά σχεδόν ακαριαία στα αιτήματα της ομάδας, διατηρώντας τη διαδραστικότητα της εφαρμογής σε υψηλό επίπεδο. Επιπλέον, για τη βέλτιστη παροχή πληροφοριών στο μοντέλο, χρησιμοποιείται το OpenWeather API [28], προσφέροντας στην Τεχνητή Νοημοσύνη ζωντανή περιβαλλοντική επίγνωση.

3.8.1 Πειραματική Αξιολόγηση Εναλλακτικών Μοντέλων AI (OpenAI & Google Gemini)

Κατά τα αρχικά στάδια ανάπτυξης της εφαρμογής Pyxis, πραγματοποιήθηκε εκτενής πειραματισμός και συγκριτική αξιολόγηση διαφορετικών Μεγάλων Γλωσσικών Μοντέλων (LLMs), προκειμένου να επιλεγεί ο καταλληλότερος πυρήνας Τεχνητής Νοημοσύνης. Στο πλαίσιο αυτό, ενσωματώθηκαν και δοκιμάστηκαν μέσω των επίσημων βιβλιοθηκών τους τα εμπορικά APIs του OpenAI (ChatGPT) και της Google (Gemini). Παρά τις προηγμένες δυνατότητές τους, η πρακτική εφαρμογή τους στο συγκεκριμένο σύστημα ανέδειξε ορισμένους περιορισμούς που οδήγησαν στην τελική επιλογή του LLaMA 3.3 μέσω του Groq API.

Πιο συγκεκριμένα, η αξιολόγηση κατέδειξε τα εξής:

1. **Μοντέλα OpenAI (π.χ. GPT-3.5 / GPT-4):** Αν και τα μοντέλα της OpenAI επέδειξαν εξαιρετική αντιληπτική ικανότητα στην παραγωγή των προτάσεων, το κύριο μειονέκτημά τους για την εφαρμογή Pyxis ήταν η υστέρηση χρόνου (latency). Επειδή η εφαρμογή απαιτεί αλληλεπίδραση σε πραγματικό χρόνο εντός του ψηφιακού δωματίου (Lobby), ο χρόνος αναμονής (Time-To-First-Token) για την παραγωγή ολοκληρωμένων απαντήσεων δημιουργούσε καθυστερήσεις, υποβαθμίζοντας την εμπειρία χρήστη (UX) στη μετάβαση προς τη φάση της σάρωσης (swiping). Επιπλέον, το υπολογιστικό κόστος ανά token κρίθηκε μη βιώσιμο για μια εφαρμογή με δυνητικά υψηλό όγκο ταυτόχρονων μηνυμάτων.
2. **Google Gemini:** Η ενσωμάτωση του μοντέλου Gemini της Google προσέφερε ταχύτατες αποκρίσεις, ωστόσο παρουσίασε δυσκολίες ως προς την αυστηρή τυποποίηση της εξόδου. Η εφαρμογή Pyxis απαιτεί η απάντηση του μοντέλου να είναι αποκλειστικά σε καθαρή μορφή JSON, ώστε να μεταφραστεί άμεσα σε κάρτες δραστηριοτήτων. Το Gemini συχνά παρενέβαλλε επεξηγηματικό κείμενο (π.χ. μορφοποίηση Markdown ``json), γεγονός που απαιτούσε επιπλέον πολύπλοκους μηχανισμούς καθαρισμού (parsing) στο Backend, αυξάνοντας την πιθανότητα σφαλμάτων κατά την εκτέλεση του κώδικα.

Η Τελική Επιλογή (LLaMA 3.3 & Groq API): Λαμβάνοντας υπόψη τα παραπάνω, προκρίθηκε η χρήση του ανοιχτού κώδικα μοντέλου LLaMA 3.3, εκτελούμενου πάνω στην υποδομή της Groq. Η καινοτόμος αρχιτεκτονική LPU (Language Processing Units) της Groq προσφέρει ασυναγώνιστη ταχύτητα δημιουργίας λέξεων (έως και εκατοντάδες tokens ανά δευτερόλεπτο). Σε συνδυασμό με την ικανότητα του LLaMA να υπακούει αυστηρά στα System Prompts για παραγωγή αμιγούς JSON, η λύση αυτή εξασφάλισε μηδενικές καθυστερήσεις στην παραγωγή προτάσεων, καθιστώντας το σύστημα απόλυτα ανθεκτικό, ταχύτατο και έτοιμο για περιβάλλον παραγωγής.

3.9 Αρχιτεκτονική RESTful API

Για τη βασική ανταλλαγή δεδομένων, υιοθετήθηκαν οι αρχές της αρχιτεκτονικής REST (Representational State Transfer) [13]. Τα RESTful APIs οργανώνουν τις λειτουργίες γύρω από πόρους (Resources), οι οποίοι αναγνωρίζονται από μοναδικά URLs (Endpoints) [18]. Η διασύνδεση μεταξύ Πελάτη (React) και Εξυπηρετητή (Node.js) επιτυγχάνεται μέσω τυποποιημένων μεθόδων HTTP (GET, POST, PUT, DELETE).

Το Backend της εφαρμογής Pyxis υλοποιεί ένα πλήρως δομημένο API, το οποίο χωρίζεται λογικά στους παρακάτω ελεγκτές (routers):

- **Διαχείριση Χρηστών & Αυθεντικοποίηση (/api/auth, /api/user)**
 - POST /auth/login & POST /auth/register: Είσοδος και εγγραφή με χρήση bcrypt για κρυπτογράφηση κωδικών και έκδοση JSON Web Tokens (JWT).
 - GET /user/profile & PUT /user/profile: Ανάκτηση και δυναμική ενημέρωση των προτιμήσεων του χρήστη (τοποθεσία, προϋπολογισμός, ενδιαφέροντα).
- **Τεχνητή Νοημοσύνη (/api/ai)**
 - POST /ai/suggest: Τροφοδοτεί το LLM (LLaMA 3.3) με τα ενδιαφέροντα του χρήστη και τον τρέχοντα καιρό, επιστρέφοντας βαθμολογημένες προτάσεις (AI Score).
 - POST /ai/chatbot: Διαχειρίζεται την ελεύθερη συνομιλία του χρήστη με τον ψηφιακό βοηθό.
- **Ομαδικές Συνεδρίες & Swiping (/api/group)**
 - POST /group/create: Δημιουργία ψηφιακού δωματίου και παραγωγή μοναδικού 6-ψήφιου PIN πρόσβασης.
 - POST /group/vote: Καταγραφή της ψήφου (like/dislike) σε πραγματικό χρόνο και εκτέλεση του αλγορίθμου συμφωνίας (Match Algorithm).
 - GET /group/activities/:sessionId: Δυναμική ανάκτηση και αποστολή καρτών με τυχαία σειρά (ORDER BY RAND()) στα μέλη του Lobby, βάσει των κριτηρίων της ομάδας.
- **Ιστορικό & Αξιολογήσεις (/api/history, /api/favourites, /api/reviews)**
 - Πλήρες σύνολο μεθόδων (CRUD) για την καταγραφή των σελίδων που επισκέπτεται ο χρήστης, την αποθήκευση αγαπημένων δραστηριοτήτων και την υποβολή ή ανάγνωση κριτικών από την κοινότητα.
- **Διαχείριση Δραστηριοτήτων (/api/activities)**

- GET /activities & GET /activities/:id: Ανάκτηση ολόκληρου του καταλόγου των διαθέσιμων δραστηριοτήτων ή προβολή λεπτομερειών (περιγραφή, κόστος, τοποθεσία) για μία συγκεκριμένη δραστηριότητα.
- **Διαχείριση Ψηφιακών Δωματίων και Μηνυμάτων (/api/lobby & index.js)**
 - POST /lobby/join: Διαδικασία εισόδου ενός χρήστη σε ένα ενεργό δωμάτιο.
 - GET /lobby/messages/:activityId: Ανάκτηση του ιστορικού συνομιλίας (chat) για το συγκεκριμένο δωμάτιο, ώστε οι νέοι χρήστες να βλέπουν τι έχει ήδη συζητηθεί πριν μουν.

3.9.1 Δοκιμές και Αποσφαλμάτωση API (API Testing & Debugging)

Για τον έλεγχο και την αποσφαλμάτωση των τελικών σημείων (endpoints) του API κατά τη διάρκεια της ανάπτυξης, αξιοποιήθηκε εκτενώς η πλατφόρμα **Postman [29]**. Αποτελώντας το βιομηχανικό πρότυπο για δοκιμές API, το Postman επέτρεψε την προσομοίωση πολύπλοκων HTTP αιτημάτων (GET, POST, DELETE) προς τον εξυπηρετητή (Backend) ανεξάρτητα από το Frontend. Μέσω αυτού του εργαλείου, επαληθεύτηκε η ορθή λειτουργία της δρομολόγησης, η ορθότητα των παραγόμενων δομών JSON, καθώς και η ασφάλεια των δεδομένων (π.χ. έλεγχος των JWT tokens στις κεφαλίδες των αιτημάτων), διασφαλίζοντας τη στιβαρότητα της επιχειρηματικής λογικής προτού αυτή διασυνδεθεί με τη διεπαφή χρήστη της React.

Κεφάλαιο 4: Σχεδιασμός και Αρχιτεκτονική Συστήματος

4.1 Εισαγωγή

Στο κεφάλαιο αυτό παρουσιάζεται ο λεπτομερής σχεδιασμός της αρχιτεκτονικής του συστήματος Pyxis. Αναλύεται η δομή των επιμέρους επιπέδων, το σχήμα της βάσης δεδομένων, οι ροές των δεδομένων κατά τη διάρκεια μιας ομαδικής συνεδρίας, καθώς και η μεθοδολογία ενσωμάτωσης της Τεχνητής Νοημοσύνης. Τέλος, παρατίθεται η ανάλυση και ο ψευδοκώδικας του προτεινόμενου αλγορίθμου ομοφωνίας (Matching Algorithm), ο οποίος αποτελεί τον λειτουργικό πυρήνα της εφαρμογής.

4.2 Αρχιτεκτονική Σχεδίαση (System Architecture)

Το σύστημα Pyxis ακολουθεί μια πολυεπίπεδη αρχιτεκτονική τριών στρωμάτων (3-Tier Architecture), η οποία επεκτείνεται με την προσθήκη μιας εξωτερικής υπηρεσίας AI. Τα επίπεδα αυτά ορίζονται ως εξής:

1. **Επίπεδο Παρουσίασης (Presentation Tier / Client):** Υλοποιημένο με React.js [22]. Είναι υπεύθυνο για τη διεπαφή, τη διαχείριση της τοπικής κατάστασης (Component State) και την επικοινωνία σε πραγματικό χρόνο με τον WebSocket εξυπηρετητή.
2. **Επίπεδο Εφαρμογής (Application Tier / Server):** Υλοποιημένο με Node.js [14] και Express.js [23]. Περιλαμβάνει τους ελεγκτές (Controllers), τα middlewares ασφαλείας, τη διαχείριση των WebSockets (Socket.IO) [26] και τη λογική επικοινωνίας με το Groq API [27].
3. **Επίπεδο Δεδομένων (Data Tier / Database):** Η σχεσιακή βάση δεδομένων MySQL [19], [20], η οποία αναλαμβάνει την ασφαλή εγγραφή και ανάκτηση των πληροφοριών.

Η ροή εργασίας ξεκινά από το Επίπεδο Παρουσίασης, περνά από το Επίπεδο Εφαρμογής όπου ελέγχεται η εγκυρότητα και η ασφάλεια του αιτήματος, και καταλήγει στο Επίπεδο Δεδομένων ή στην εξωτερική υπηρεσία AI, επιστρέφοντας την αντίστοιχη πληροφορία στον χρήστη.

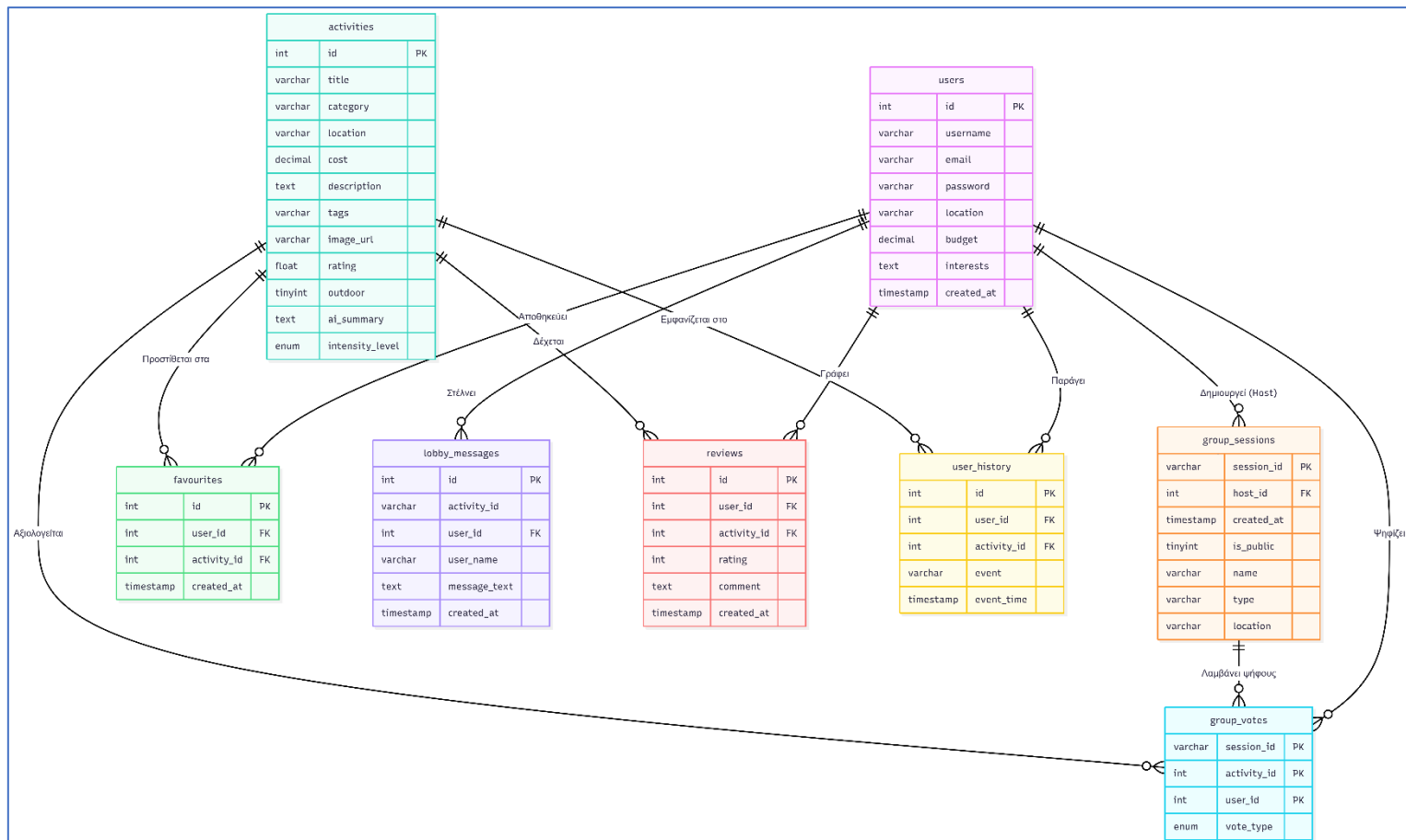
4.3 Σχεδιασμός και Μοντελοποίηση Βάσης Δεδομένων (Database Schema)

Η σχεσιακή βάση δεδομένων σχεδιάστηκε με γνώμονα την αποφυγή του πλεονασμού δεδομένων και τη διασφάλιση της ακεραιότητας των πληροφοριών. Το τελικό σχήμα της βάσης αποτελείται από τους εξής βασικούς πίνακες:

- **users:** Αποθηκεύει τα στοιχεία, το προφίλ και τις προτιμήσεις των χρηστών (*id, username, email, password, location, budget, interests, created_at*).
- **activities:** Αποθηκεύει τις προτεινόμενες δραστηριότητες και τα εκτενή χαρακτηριστικά τους (*id, title, category, location, cost, description, tags, image_url, rating, outdoor, ai_summary, intensity_level, discount_info*). Οι δραστηριότητες αυτές αποτελούν τον πυρήνα της επιλογής.
- **group_sessions:** Διαχειρίζεται τα ψηφιακά δωμάτια / ομάδες που δημιουργούν οι χρήστες (*session_id, host_id, created_at, is_public, name, type, location*).
- **group_votes:** Καταγράφει τις ψήφους (μηχανισμός swiping) των χρηστών για τις δραστηριότητες μέσα σε κάθε δωμάτιο (*session_id, activity_id, user_id, vote_type*). Το πεδίο *vote_type* λαμβάνει τις δυαδικές τιμές 'like' ή 'dislike'.
- **lobby_messages:** Αποθηκεύει το ιστορικό της συνομιλίας (chat) της ομάδας εντός του εκάστοτε δωματίου (*id, activity_id, user_id, user_name, message_text, created_at*), επιτρέποντας στα μέλη να βλέπουν τι έχει ειπωθεί.
- **favourites:** Καταγράφει τις δραστηριότητες που οι χρήστες έχουν αποθηκεύσει ως "αγαπημένες" (*id, user_id, activity_id, created_at*).
- **reviews:** Διαχειρίζεται τις κριτικές και τις βαθμολογίες (1-5) που αφήνουν οι χρήστες για τις δραστηριότητες (*id, user_id, activity_id, rating, comment, created_at*).
- **user_history:** Καταγράφει το ιστορικό αλληλεπίδρασης του χρήστη με την πλατφόρμα, όπως για παράδειγμα ποιες δραστηριότητες έχει προβάλει (*id, user_id, activity_id, event, event_time*).

Οι συσχετίσεις (Foreign Keys) μεταξύ των πινάκων εξασφαλίζουν ότι αν ένα Lobby διαγραφεί ή κλείσει, οι σχετικές εγγραφές μηνυμάτων και ψηφοφορίας ενημερώνονται ή απαλείφονται κατάλληλα (Cascade Operations), διατηρώντας τη βάση δεδομένων απολύτως καθαρή.

Διαχείριση Πολυμέσων και Χωρητικότητας: Αξίζει να σημειωθεί μια κρίσιμη αρχιτεκτονική επιλογή όσον αφορά τη διαχείριση των πολυμέσων του συστήματος. Προκειμένου να διατηρηθεί το μέγεθος της σχεσιακής βάσης δεδομένων σε χαμηλά επίπεδα και να βελτιστοποιηθεί ο χρόνος απόκρισης, τα αρχεία των εικόνων των δραστηριοτήτων δεν αποθηκεύονται τοπικά ως δυαδικά δεδομένα (BLOBs). Αντιθέτως, φιλοξενούνται σε εξωτερική υπηρεσία παροχής περιεχομένου (Image Hosting / CDN μέσω του ImgBB). Στον πίνακα activities αποθηκεύεται αποκλειστικά ο άμεσος σύνδεσμος (direct URL) στο αντίστοιχο πεδίο image_url. Η προσέγγιση αυτή ακολουθεί τις σύγχρονες βέλτιστες πρακτικές, καθώς αποφορτίζει τον κεντρικό διακομιστή (Node.js) από την εξυπηρέτηση βαριών στατικών αρχείων και εξασφαλίζει ταχύτατη φόρτωση των καρτών στη διεπαφή του χρήστη.

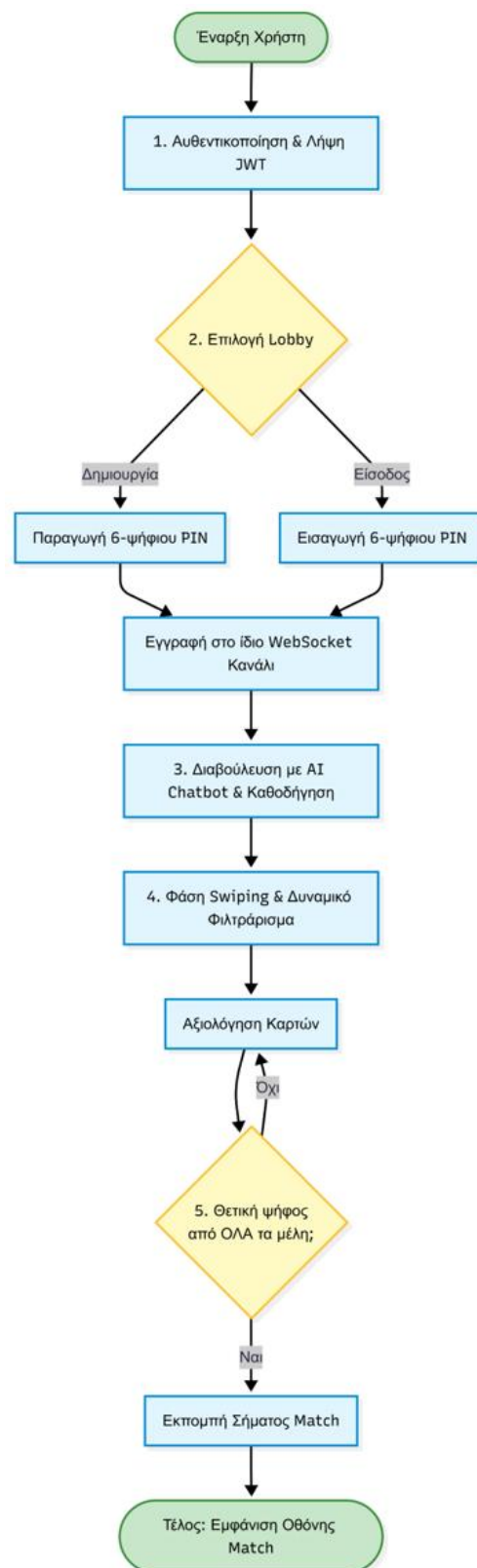


Εικόνα 4.1: Διάγραμμα Οντοτήτων - Συσχετίσεων (ER Diagram) της σχεσιακής βάσης δεδομένων MySQL του συστήματος Pyxis.

4.4 Ροές Δεδομένων και Λειτουργικά Διαγράμματα

Η λειτουργία της εφαρμογής Pyxis ακολουθεί μια αυστηρά καθορισμένη γραμμική ροή, η οποία περιλαμβάνει τις εξής φάσεις:

1. **Αυθεντικοποίηση & Σύνδεση:** Ο χρήστης εισέρχεται στο σύστημα και λαμβάνει το JSON Web Token (JWT) [24].
2. **Δημιουργία/Είσοδος σε Lobby:** Ένας χρήστης δημιουργεί ένα δωμάτιο, παράγεται ένας μοναδικός 6-ψήφιος κωδικός (room PIN), και οι φίλοι του συνδέονται εισάγοντας τον κωδικό αυτό. Όλοι οι συνδεδεμένοι χρήστες εγγράφονται στο ίδιο WebSocket κανάλι.
3. **Διαβούλευση & Παραγωγή Προτάσεων:** Η ομάδα αλληλεπιδρά με το AI Chatbot για να ανταλλάξει απόψεις. Το Chatbot, έχοντας πλήρη επίγνωση των διαθέσιμων δραστηριοτήτων της βάσης δεδομένων, καθοδηγεί τους χρήστες προτείνοντας στοχευμένες επιλογές.
4. **Φάση Swiping:** Μόλις οριστικοποιηθεί το πλαίσιο της εξόδου, η εφαρμογή μεταβαίνει στην οθόνη αξιολόγησης. Το Backend ανακτά δυναμικά από τον πίνακα activities όλες τις δραστηριότητες που αντιστοιχούν στην κατηγορία (category) και την τοποθεσία (location) του συγκεκριμένου Lobby, εξασφαλίζοντας ότι όλα τα μέλη θα ψηφίσουν ταυτόχρονα επί του ίδιου, εξατομικευμένου συνόλου καρτών.
5. **Επίλυση Συμφωνίας (Match):** Ο αλγόριθμος ελέγχει αν η συγκεκριμένη δραστηριότητα συγκέντρωσε θετική ψήφο από όλα τα ενεργά μέλη. Μόλις συμβεί αυτό, εκπέμπεται σήμα Match σε όλους τους χρήστες ταυτόχρονα.



Εικόνα 4.2: Διάγραμμα ροής εργασιών (Workflow Diagram) κατά τη διάρκεια μιας ομαδικής συνεδρίας στο σύστημα Pyxis.

4.5 Ενσωμάτωση Τεχνητής Νοημοσύνης και Μηχανική Υποδείξεων (Prompt Engineering)

Η επικοινωνία με το μοντέλο LLaMA 3.3 απαιτεί προσεκτικό σχεδιασμό, καθώς η έξοδος της Τεχνητής Νοημοσύνης πρέπει να είναι δομημένη με τέτοιο τρόπο ώστε να μπορεί να αναλυθεί (parsed) αυτόματα από τον κώδικα του Backend και να μετατραπεί σε κάρτες swiping.

4.5.1 Δομή και Πλαίσιο Υποδείξεων (Prompt Structuring)

Για την επίτευξη του στόχου αυτού, αναπτύχθηκε ένα αυστηρό "System Prompt" (Υπόδειξη Συστήματος). Το System Prompt καθορίζει τον ρόλο του μοντέλου ως «έμπειρου συμβούλου ομαδικής ψυχαγωγίας» και του επιβάλλει να απαντά αποκλειστικά σε μορφή έγκυρου αρχείου JSON. Οποιοδήποτε συμπληρωματικό κείμενο (π.χ. "Ορίστε οι προτάσεις μου:") απαγορεύεται ρητά, καθώς θα προκαλούσε σφάλμα κατά τη μετατροπή του κειμένου σε αντικείμενο JavaScript (JSON.parse).

4.5.2 Διαχείριση Ιστορικού Συνομιλίας (Context Window Management)

Προκειμένου το Chatbot να προσφέρει μια ρεαλιστική εμπειρία διαλόγου, το Backend διατηρεί στη μνήμη το ιστορικό των μηνυμάτων της τρέχουσας συνεδρίας του Lobby. Σε κάθε νέο μήνυμα ενός χρήστη, ολόκληρος ο πίνακας των προηγούμενων μηνυμάτων (μαζί με τα System και Assistant μηνύματα) αποστέλλεται εκ νέου στο Groq API. Για την αποφυγή της υπερχειλίσης του παραθύρου πλαισίου (Context Window) και τη μείωση του κόστους σε tokens, εφαρμόζεται ένας μηχανισμός ολισθαίνοντος παραθύρου (sliding window), ο οποίος διατηρεί δυναμικά μόνο τα τελευταία 10 μηνύματα της συνομιλίας.

4.5.3 Μηχανική Υποδείξεων για Παραγωγή Δραστηριοτήτων

Όταν ο χρήστης ζητά προτάσεις, το User Prompt εμπλουτίζεται δυναμικά από το Backend με μετα-δεδομένα. Το τελικό prompt που φτάνει στο LLaMA 3.3 επιβάλλει τη δημιουργία ενός πίνακα αντικειμένων JSON, όπου κάθε αντικείμενο πρέπει να περιλαμβάνει υποχρεωτικά τα εξής πεδία: title (τίτλος δραστηριότητας), description (σύντομη περιγραφή), category (κατηγορία) και estimated_cost (εκτιμώμενο κόστος).

4.5.4 Βελτιστοποίηση Κόστους και Ταχύτητας (Pre-filtering)

Μία από τις μεγαλύτερες προκλήσεις στη χρήση εμπορικών APIs Τεχνητής Νοημοσύνης είναι η διαχείριση του υπολογιστικού κόστους ανά Token και ο χρόνος απόκρισης. Προκειμένου να βελτιστοποιηθεί η απόδοση της εφαρμογής Pyxis, σχεδιάστηκε ένας μηχανισμός προ-φιλτραρίσματος (pre-filtering) στο επίπεδο του Backend. Πιο συγκεκριμένα, όταν ο χρήστης αναζητά προτάσεις, το σύστημα εκτελεί αρχικά κλασικά φίλτρα αφαίρεσης (όπως ο

περιορισμός βάσει του διαθέσιμου budget ή της τοποθεσίας) απευθείας στα δεδομένα της SQL βάσης. Μόνο το υποσύνολο των δραστηριοτήτων που ικανοποιούν τα σκληρά αυτά κριτήρια μετατρέπεται σε απλοποιημένη δομή JSON και αποστέλλεται στο μοντέλο LLaMA 3.3. Η προσέγγιση αυτή προστατεύει το Context Window του μοντέλου από άχρηστη πληροφορία, ελαχιστοποιεί την κατανάλωση tokens και εξασφαλίζει ακαριαίες απαντήσεις.

4.5.5 Επίγνωση Πλαισίου (Context-Awareness) μέσω OpenWeather API

Για να παραχθούν απόλυτα εξατομικευμένες και ρεαλιστικές προτάσεις από την Τεχνητή Νοημοσύνη, το σύστημα σχεδιάστηκε ώστε να διαθέτει "επίγνωση του πλαισίου" (context-awareness). Πιο συγκεκριμένα, στο επίπεδο της διεπαφής χρήστη (Frontend), υλοποιήθηκε διασύνδεση με το εξωτερικό API της υπηρεσίας OpenWeather [28]. Πριν την αποστολή του ερωτήματος στο Μεγάλο Γλωσσικό Μοντέλο (LLM), η εφαρμογή ανακτά σε πραγματικό χρόνο τις καιρικές συνθήκες (π.χ. βροχή, συννεφιά, καθαρός ουρανός) για την περιοχή ενδιαφέροντος. Αυτή η πληροφορία ενσωματώνεται δυναμικά στο prompt του AI. Με αυτόν τον μηχανισμό, το μοντέλο είναι σε θέση να προσαρμόσει την έξοδό του, δίνοντας για παράδειγμα αυστηρή προτεραιότητα σε δραστηριότητες εσωτερικού χώρου εάν στην πόλη του χρήστη βρέχει εκείνη τη στιγμή, αυξάνοντας ραγδαία την πρακτική αξία των προτάσεων.

4.5.6 Ευρετικός Αλγόριθμος Βαθμολόγησης Πελάτη (Frontend Heuristic Scoring)

Πέραν της βαριάς επεξεργασίας που αναλαμβάνει η Τεχνητή Νοημοσύνη, η εφαρμογή ενσωματώνει έναν ελαφρύ, ευρετικό αλγόριθμο συστάσεων (heuristic recommendation engine) που εκτελείται αποκλειστικά στην πλευρά του πελάτη (Frontend). Ο αλγόριθμος αυτός αναλύει το προφίλ του χρήστη και βαθμολογεί τοπικά τις διαθέσιμες δραστηριότητες (Scoring System) με βάση σταθμισμένα κριτήρια: ταύτιση τοποθεσίας (+40 πόντοι), σημασιολογική ταύτιση ενδιαφερόντων (+40 πόντοι), ικανοποίηση οικονομικού προϋπολογισμού (+20 πόντοι) και bonus δωρεάν δραστηριοτήτων (+10 πόντοι). Οι προτάσεις ταξινομούνται φθίνουσα και οι κορυφαίες εμφανίζονται στην ενότητα «Για Σένα». Η αρχιτεκτονική αυτή επιλογή (Edge Computing) αποφορτίζει τον κεντρικό εξυπηρετητή και μειώνει δραστικά τον χρόνο αναμονής του χρήστη.

4.6 Δυναμικές Ροές Εργασίας και ο Αλγόριθμος Συμφωνίας (Matching Algorithm)

4.6.1 Λογική Δωματίων (Lobby Session Lifecycle)

Ο κύκλος ζωής ενός δωματίου είναι δυναμικός και ελέγχεται από τις αλλαγές κατάστασης στη βάση δεδομένων. Η Socket.IO [26] διασφαλίζει ότι αν ένας χρήστης αποσυνδεθεί απροσδόκητα (λόγω δικτύου), το σύστημα τον αφαιρεί προσωρινά από τη λίστα των "ενεργών ψηφοφόρων", ώστε να μην παγώσει η διαδικασία λήψης απόφασης για τους υπόλοιπους. Μόλις ο χρήστης επανασυνδεθεί, ανακτά την τρέχουσα κατάσταση του δωματίου και συνεχίζει κανονικά τη διαδικασία.

4.6.2 Υλοποίηση του Αλγορίθμου Συμφωνίας σε Πραγματικό Χρόνο

Ο αλγόριθμος συμφωνίας εκτελείται στο Backend κάθε φορά που ένας χρήστης κάνει swipe 'like' σε μια κάρτα. Η βασική του ιδέα είναι να ελέγχει αν ο συνολικός αριθμός των θετικών ψήφων για τη συγκεκριμένη δραστηριότητα στο τρέχον δωμάτιο ισούται με τον αριθμό των συνολικών μελών που είναι εγγεγραμμένα στο δωμάτιο.

Ακολουθεί η αναπαράσταση της δομής του αλγορίθμου σε μορφή ψευδοκώδικα:

Διαδικασία API_Group_Vote(userId, sessionId, activityId, voteType)

```
// 1. Καταχώρηση της ψήφου στη Βάση Δεδομένων
Εκτέλεσε_Query("INSERT INTO group_votes (session_id, activity_id, user_id, vote_type)
VALUES (?, ?, ?, ?)", [sessionId, activityId, userId, voteType])

// 2. Αν η ψήφος είναι αρνητική (dislike), δεν απαιτείται έλεγχος για match
**Αν** voteType == "dislike" **τότε**
    **Επίστρεψε** "Vote Recorded"
**Τέλος_Αν**

// 3. Ανάκτηση συνολικού αριθμού των μελών του Lobby (από τη μνήμη του Node.js)
totalMembers = Μήκος(publicLobbies[sessionId])

// 4. Ανάκτηση του αριθμού των 'likes' που έχει λάβει η συγκεκριμένη δραστηριότητα
```



```
totalLikes = Εκτέλεσε_Query("SELECT COUNT(*) FROM group_votes WHERE  
session_id = ? AND activity_id = ? AND vote_type = 'like'", [sessionId, activityId])  
  
// 5. Έλεγχος επίτευξης ομοφωνίας (Match)  
**Αν** totalLikes == totalMembers **τότε**  
    // Εκπομπή του συμβάντος Match σε πραγματικό χρόνο  
    Εκπομπή_WebSocket_Σε_Δωμάτιο(sessionId, "match_found", activityId)  
**Τέλος_Αν**
```

Τέλος_Διαδικασίας

Κεφάλαιο 5: Υλοποίηση, Διεπαφή Χρήστη (UI) και Ανάπτυξη (Deployment)

5.1 Εισαγωγή

Στο παρόν κεφάλαιο παρουσιάζεται το τελικό πρακτικό αποτέλεσμα της διπλωματικής εργασίας. Πιο συγκεκριμένα, αναλύεται η υλοποίηση της Διεπαφής Χρήστη (User Interface - UI), περιγράφονται οι κύριες οθόνες με τις οποίες αλληλεπιδρά ο τελικός χρήστης, και παρατίθενται τα αντίστοιχα στιγμιότυπα οθόνης (screenshots) που αποδεικνύουν την ορθή λειτουργία του συστήματος. Επιπλέον, περιγράφεται η διαδικασία διάθεσης (deployment) της εφαρμογής σε περιβάλλον υπολογιστικού νέφους (cloud).

5.2 Υλοποίηση Διεπαφής Χρήστη (User Interface / User Experience)

Η σχεδίαση της διεπαφής (Frontend) πραγματοποιήθηκε με γνώμονα τη φιλικότητα προς τον χρήστη, την απλότητα και την υψηλή αισθητική. Βασική προτεραιότητα αποτέλεσε η υιοθέτηση ενός Αποκριτικού Σχεδιασμού (Responsive Design), ώστε η εφαρμογή να προσαρμόζεται δυναμικά και να προσφέρει άριστη εμπειρία πλοήγησης, ανεξαρτήτως αν ο χρήστης συνδέεται από ηλεκτρονικό υπολογιστή, tablet ή κινητό τηλέφωνο.

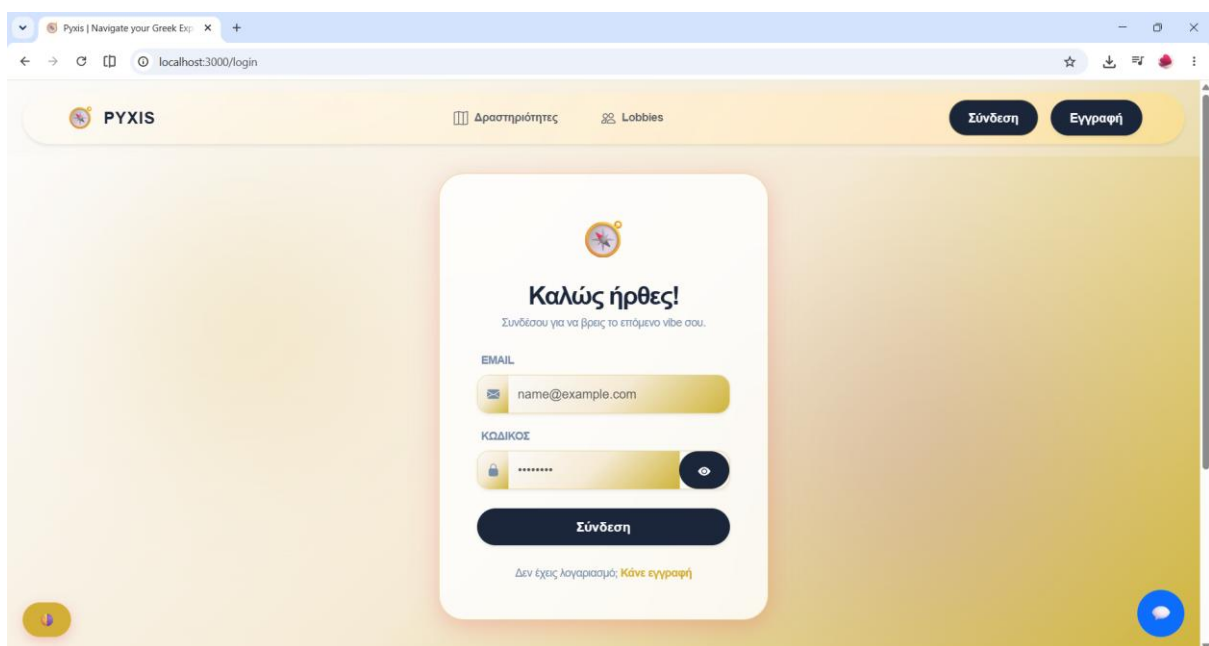
Για τον σχεδιασμό και τη δόμηση του περιβάλλοντος αξιοποιήθηκε το σύγχρονο πλαίσιο Bootstrap 5, εξασφαλίζοντας μια καθαρή και ομοιόμορφη αισθητική. Παράλληλα, για την άμεση και φιλική ενημέρωση του χρήστη (feedback) σχετικά με την επιτυχία ή αποτυχία των

ενεργειών του, ενσωματώθηκαν καλαίσθητες αναδυόμενες ειδοποιήσεις μέσω των βιβλιοθηκών SweetAlert2 και React Hot Toast, αναβαθμίζοντας σημαντικά τη συνολική Εμπειρία Χρήστη (UX).

5.2.1 Σύστημα Ταυτοποίησης και Προστατευμένες Διαδρομές (Login / Registration & Protected Routes)

Η πρώτη αλληλεπίδραση του χρήστη με την πλατφόρμα πραγματοποιείται μέσω των οθονών Εγγραφής και Σύνδεσης. Η φόρμα σύνδεσης είναι λιτή, ζητώντας μόνο τα απαραίτητα διαπιστευτήρια (email και κωδικό πρόσβασης). Σε περίπτωση λανθασμένης εισαγωγής στοιχείων, το σύστημα επιστρέφει κατάλληλα, φιλικά μηνύματα σφάλματος (error handling), ενώ η επιτυχής σύνδεση δρομολογεί αυτόματα τον χρήστη στην κεντρική οθόνη ελέγχου.

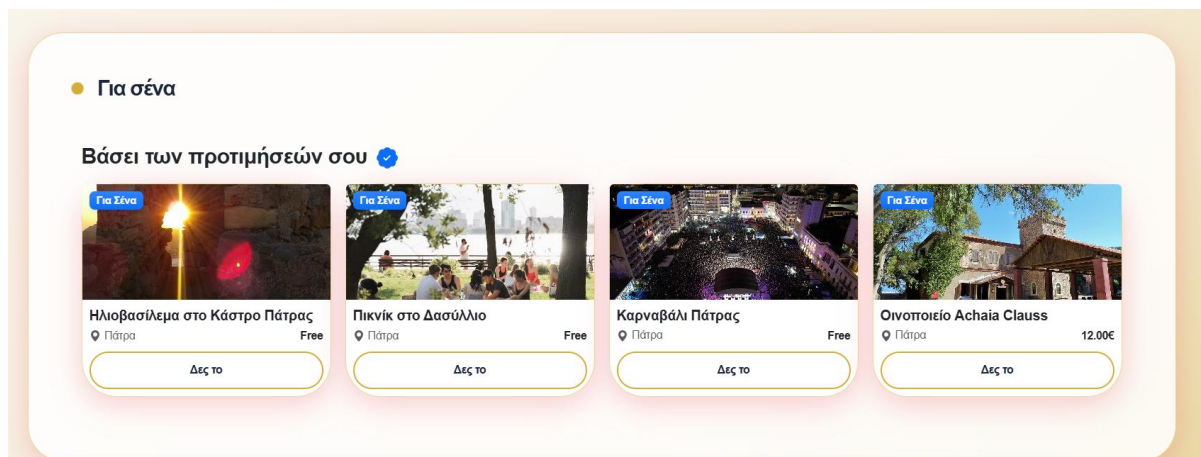
Η ασφάλεια επεκτείνεται και στο επίπεδο της διεπαφής μέσω της ανάπτυξης ειδικών συστατικών προστασίας (**Protected Routes**). Τα στοιχεία αυτά ελέγχουν διαρκώς την ύπαρξη του έγκυρου JWT Token [24] στο localStorage του φυλλομετρητή και αναχαιτίζουν την πλοήγηση μη ταυτοποιημένων χρηστών προς ιδιωτικές σελίδες (όπως οι προτάσεις ή το προφίλ), ανακατευθύνοντάς τους αυτόματα στην οθόνη σύνδεσης.



Εικόνα 5.1: Στιγμιότυπο οθόνης από τη σελίδα σύνδεσης (Login) και αυθεντικοποίησης της εφαρμογής Pyxis.

5.2.2 Αρχική Σελίδα

Παράλληλα, στην οθόνη αυτή ενσωματώνεται η οπτική διεπαφή του **Ευρετικού Αλγορίθμου** ("Βάσει των προτιμήσεών σου / Για Σένα"). Το σύστημα αναλύει τοπικά τα ενδιαφέροντα και το budget του χρήστη και προβάλλει αυτόματα προσωποποιημένες κάρτες προτάσεων.

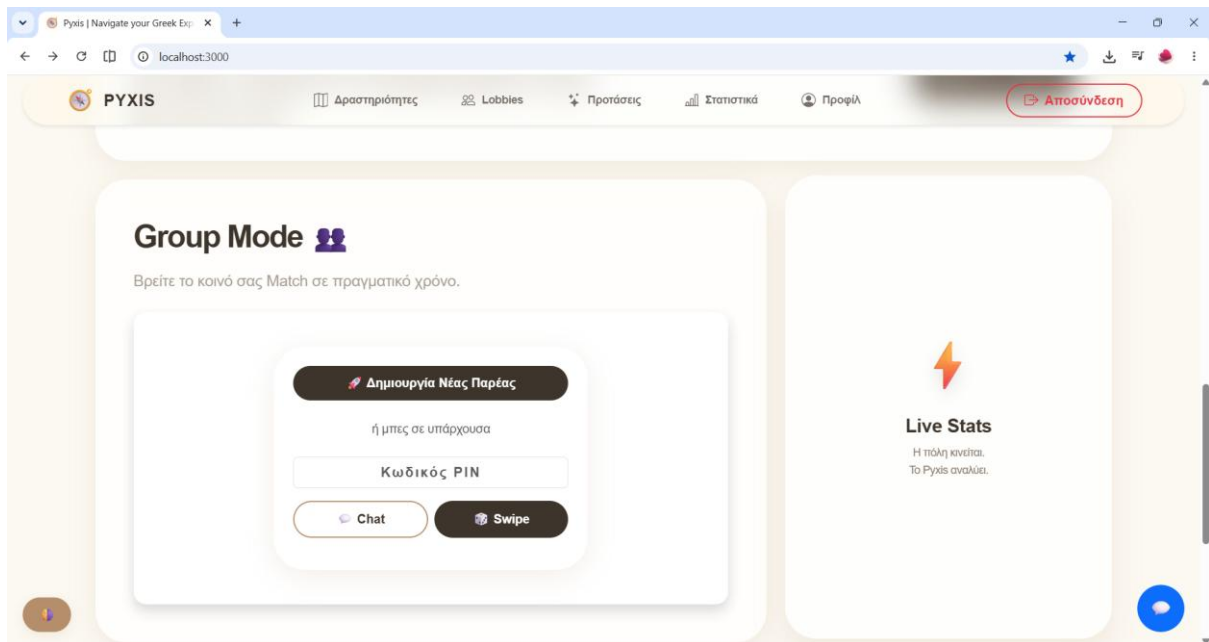


Εικόνα 5.2: Στιγμιότυπο της κεντρικής οθόνης (Home Page) της εφαρμογής, όπου απεικονίζονται οι προσωποποιημένες προτάσεις του Ευρετικού Αλγορίθμου βάσει των προτιμήσεων του χρήστη.

5.2.3 Δημιουργία και Είσοδος σε Ψηφιακά Δωμάτια (Lobbies)

Ο πυρήνας της κοινωνικής λειτουργικότητας του Pyxis είναι τα Ψηφιακά Δωμάτια (Lobbies). Στην κεντρική οθόνη, μέσω της ειδικής ενότητας "**Group Mode**", ο χρήστης διαθέτει δύο κύριες επιλογές: είτε να ξεκινήσει τη διαδικασία πατώντας "**Δημιουργία Νέας Παρέας**", είτε να εισαγάγει έναν υπάρχοντα εξαψήφιο κωδικό (PIN) στο αντίστοιχο πεδίο για να ενταχθεί σε μια ήδη σχηματισμένη ομάδα.

Για την περαιτέρω διευκόλυνση της εμπειρίας (UX), κάτω από το πεδίο του κωδικού παρέχονται άμεσα πλήκτρα πλοήγησης ("**Chat**" και "**Swipe**"), επιτρέποντας στα μέλη να μεταβούν ταχύτατα στην επιθυμητή λειτουργία του δωματίου. Μόλις ο χρήστης εισέλθει στο ψηφιακό περιβάλλον, η διεπαφή εμφανίζει σε πραγματικό χρόνο τα εικονίδια (avatars) των ενεργών συμμετεχόντων, αντλώντας τα δεδομένα δυναμικά.

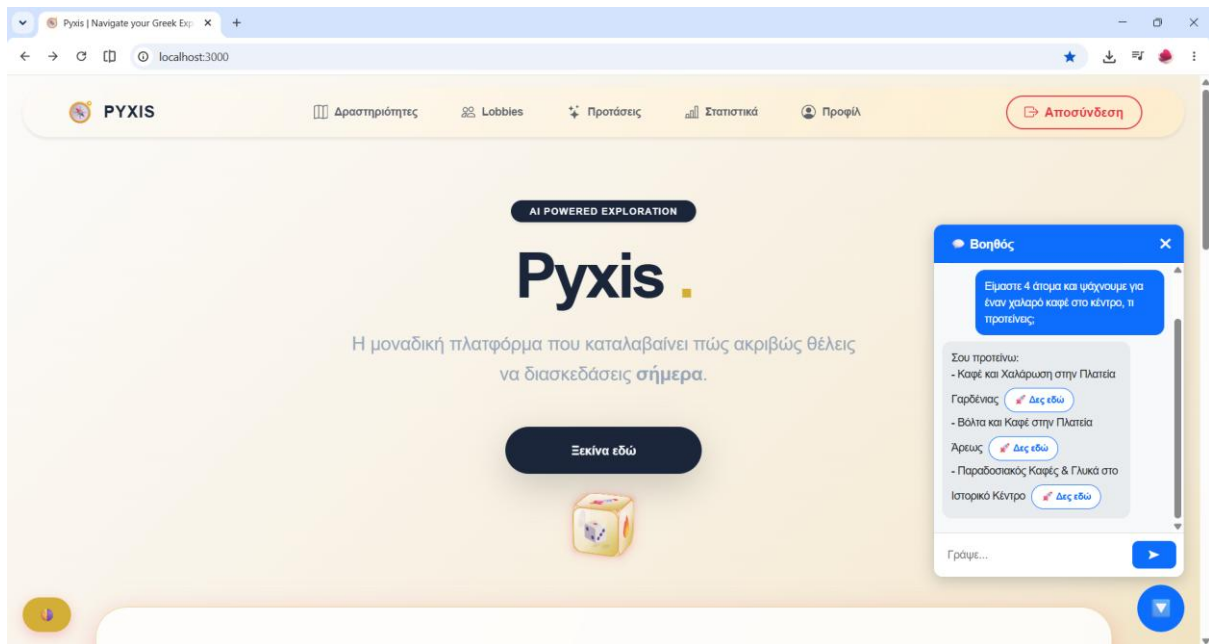


Εικόνα 5.3: Στιγμιότυπο οθόνης από το περιβάλλον του Lobby και το πεδίο εισαγωγής του εξαψήφιου κωδικού.

5.2.4 Περιβάλλον Διαδραστικού Βοηθού και Δυναμικοί Σύνδεσμοι (AI Chatbot & Regex Link Parsing)

Αφού η ομάδα συγκεντρωθεί στο δωμάτιο, παρέχεται πρόσβαση στο διαδραστικό AI Chatbot [7]. Η συγκεκριμένη οθόνη προσομοιώνει μια κλασική εφαρμογή ανταλλαγής μηνυμάτων. Κάθε χρήστης μπορεί να πληκτρολογήσει τις προτιμήσεις του σε φυσική γλώσσα (π.χ. «Ψάχνουμε για καφέ στο κέντρο»). Το μοντέλο LLaMA 3.3 [27] επεξεργάζεται τα μηνύματα και επιστρέφει δυναμικές προτάσεις απευθείας στο παράθυρο συνομιλίας.

Η καινοτομία της συγκεκριμένης οθόνης έγκειται στον μηχανισμό ανάλυσης απαντήσεων με χρήση Κανονικών Εκφράσεων (**Regex Link Parsing**). Όταν η Τεχνητή Νοημοσύνη προτείνει μια δραστηριότητα, επισυνάπτει στο κείμενο ένα κρυφό tag μορφής `{LINK:/activities/id}`. Το Frontend εντοπίζει αυτόματα αυτά τα tags και τα μετατρέπει σε διαδραστικά inline κουμπιά πλοήγησης (React Router Links), επιτρέποντας στους χρήστες να μεταβούν στη σελίδα της δραστηριότητας με ένα κλικ.

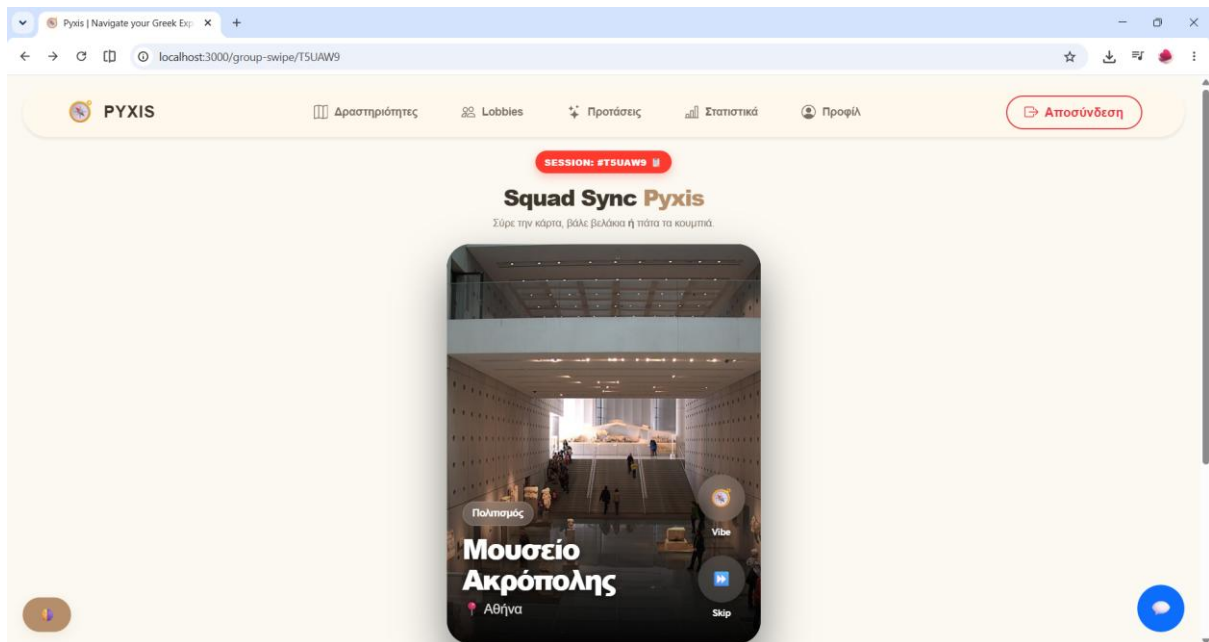


Εικόνα 5.4: Στιγμιότυπο οθόνης από τη διεπαφή συνομιλίας με το ενσωματωμένο AI Chatbot.

5.2.5 Μηχανισμός Σάρωσης και Ψηφοφορίας (Swiping)

Στην οθόνη αξιολόγησης (Swiping) εφαρμόζεται η μέθοδος της παιχνιδοποίησης [11]. Ο χρήστης καλείται να εγκρίνει ή να απορρίψει την εκάστοτε κάρτα δραστηριότητας. Έμφαση δόθηκε στην προσβασιμότητα και τη βελτιστοποίηση της εμπειρίας για κινητές συσκευές (Mobile-first UX).

Η διεπαφή υποστηρίζει πολυτροπικό χειρισμό (multi-modal input) βασισμένο σε **ιδιοκατασκευασμένο (native) κώδικα JavaScript [15]** για την ανίχνευση κίνησης, αποφεύγοντας βαριές εξωτερικές βιβλιοθήκες [10]. Ο χρήστης μπορεί να αξιολογήσει είτε χρησιμοποιώντας τα γραφικά πλήκτρα της οθόνης, είτε κάνοντας φυσική σάρωση (Touch & Drag events) υπολογίζοντας τη διαφορά της απόστασης μετατόπισης, είτε εκτελώντας διπλό χτύπημα στην οθόνη (Double Tap) για άμεση θετική ψήφο, είτε κάνοντας χρήση των φυσικών πλήκτρων πλοήγησης του πληκτρολογίου (Arrow Keys) σε περιβάλλον υπολογιστή.

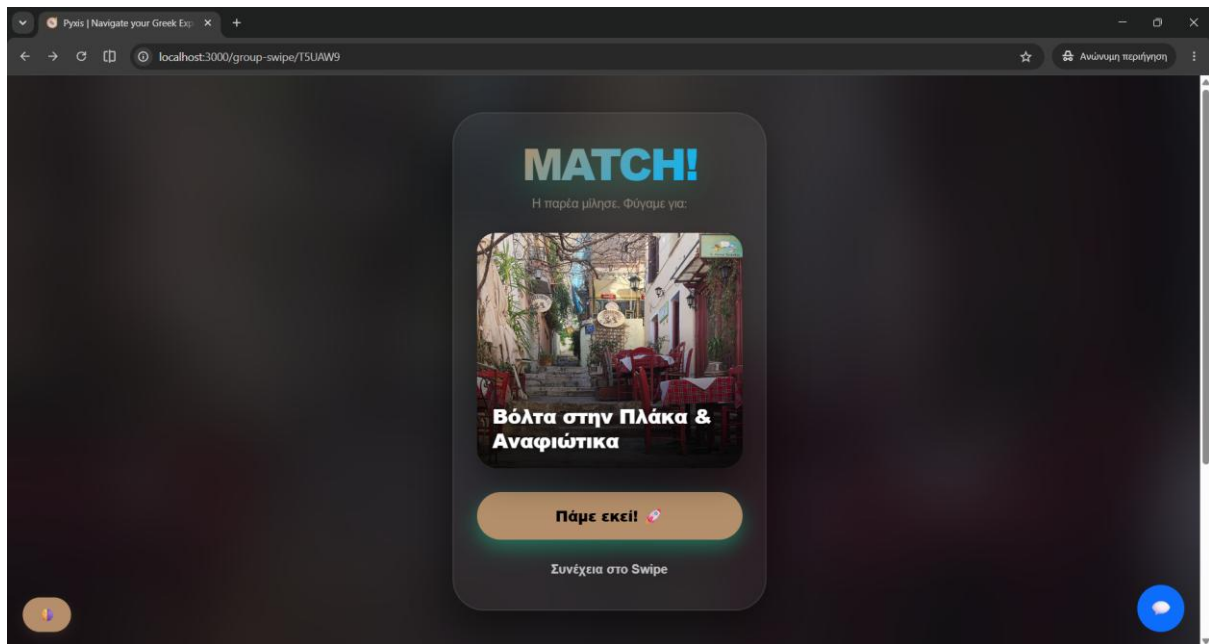


Εικόνα 5.5: Στιγμιότυπο οθόνης από την κάρτα δραστηριότητας και τον μηχανισμό σάρωσης (Swiping).

5.2.6 Οθόνη Επιτυχούς Συμφωνίας (Match Screen)

Το αποκορύφωμα της εμπειρίας χρήστη επέρχεται όταν επιτευχθεί ομοφωνία (όλα τα ενεργά μέλη του δωματίου εγκρίνουν την ίδια δραστηριότητα). Εκείνη τη στιγμή, το Backend ενημερώνει ακαριαία (μέσω WebSockets) [26] τις συσκευές των χρηστών. Η εφαρμογή διακόπτει την οθόνη σάρωσης και προβάλλει μια εντυπωσιακή, full-screen οθόνη "Match".

Για την ανάδειξη αυτής της στιγμής, συνδυάστηκαν CSS Keyframe Animations υψηλής απόδοσης (popIn και fadeIn) με την υποδομή οπτικών εφέ Framer Motion [33] της εφαρμογής, εμφανίζοντας με καθηλωτικό τρόπο την τελική επιλογή της ομάδας, συνοδευόμενη από την τοποθεσία, τη φωτογραφία και ένα άμεσο κουμπί καθοδήγησης ("Πάμε εκεί!").

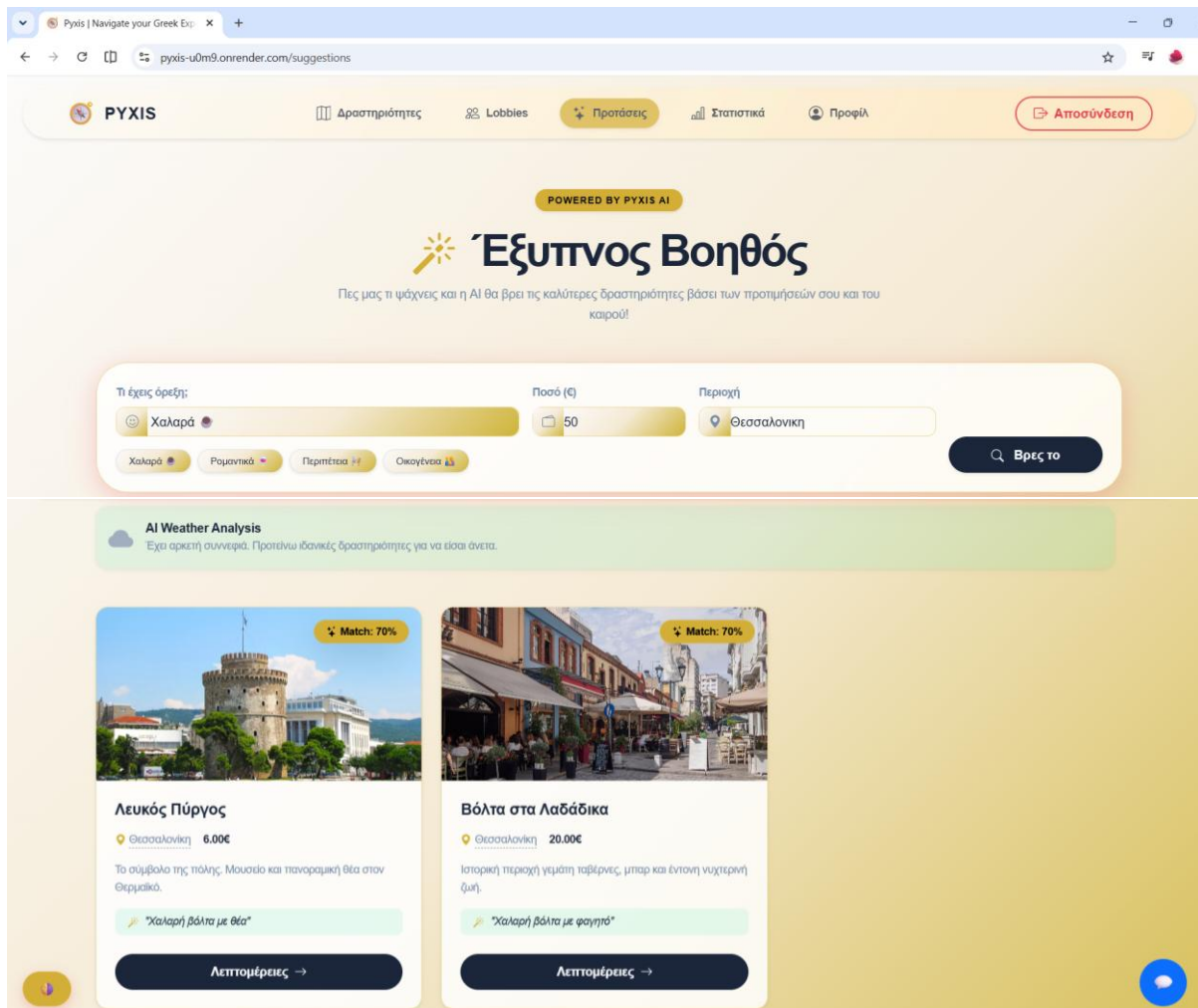


Εικόνα 5.6: Στιγμιότυπο οθόνης από την εμφάνιση επιτυχούς συμφωνίας (Match) στο σύνολο της ομάδας.

5.2.7 Εξερεύνηση και Πολυκριτηριακή Αναζήτηση Δραστηριοτήτων

Η κεντρική οθόνη του συστήματος Ryxis, η οποία αποτελεί και τον βασικό πυρήνα εξερεύνησης, προσφέρει στους χρήστες έναν δυναμικό κατάλογο των διαθέσιμων επιλογών αναψυχής. Μέσω μιας εύχρηστης διεπαφής, παρέχεται η δυνατότητα πολυκριτηριακής αναζήτησης και φιλτραρίσματος (filtering) των δραστηριοτήτων σε πραγματικό χρόνο. Ειδικότερα, οι χρήστες μπορούν να αναζητήσουν στοχευμένα μέσω κειμένου ή να περιορίσουν τα αποτελέσματα βάσει συγκεκριμένων κριτηρίων, όπως η κατηγορία της δραστηριότητας, η τοποθεσία και το επίπεδο κόστους.

Ο σχεδιασμός της σελίδας ακολουθεί τη λογική του δυναμικού πλέγματος (grid layout) με τη χρήση εξατομικευμένων καρτών (Activity Cards). Αυτή η προσέγγιση επιτρέπει την άμεση επισκόπηση των βασικών πληροφοριών κάθε επιλογής, όπως ο τίτλος, η βαθμολογία (rating) και η αντιπροσωπευτική εικόνα, εξασφαλίζοντας μια ομαλή και διαισθητική εμπειρία πλοήγησης (UX). Παράλληλα, από αυτή την οθόνη, οι χρήστες μπορούν με ένα κλικ να μεταβούν στη σελίδα των αναλυτικών λεπτομερειών.

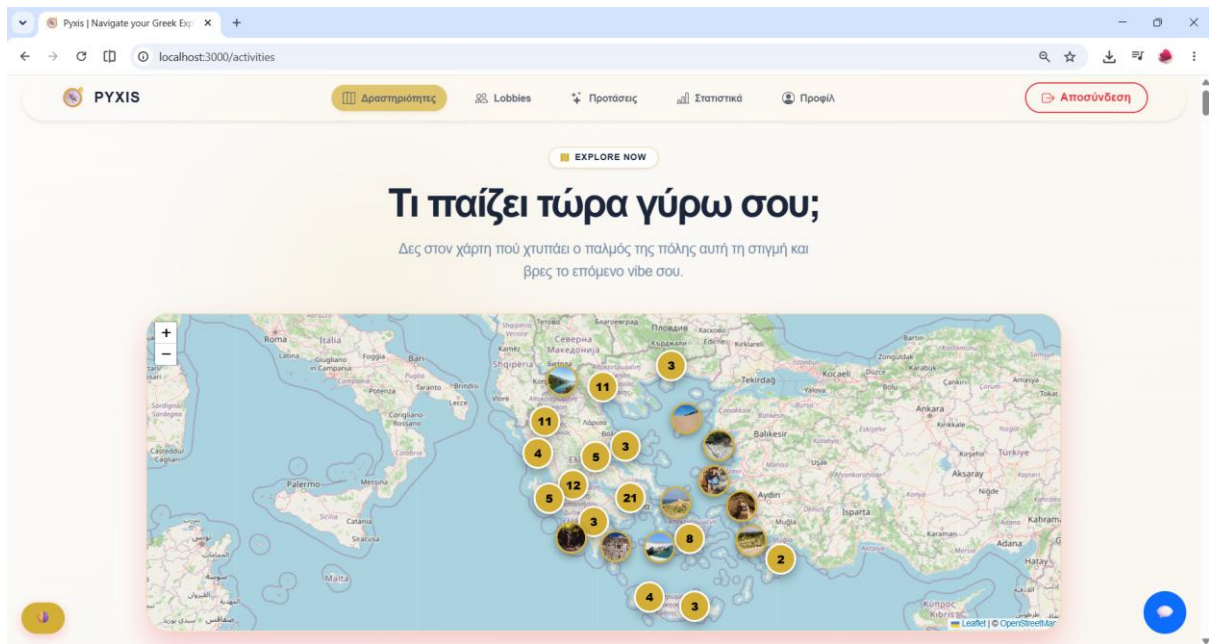


Εικόνα 5.7: Η κεντρική σελίδα εξερεύνησης δραστηριοτήτων και δυναμικών προτάσεων.

5.2.8 Διαδραστικός Χάρτης Δραστηριοτήτων (Interactive Map)

Για την καλύτερη χωρική αντίληψη, η εφαρμογή ενσωματώνει διαδραστικούς χάρτες μέσω των βιβλιοθηκών Leaflet και React-Leaflet [31]. Η υλοποίηση δεν περιορίζεται στην απλή απεικόνιση σημείων, αλλά ενσωματώνει προηγμένη διαχείριση συντεταγμένων.

Ειδικότερα, έχει αναπτυχθεί ένας ελεγκτής χάρτη (Map Controller) ο οποίος υπολογίζει δυναμικά το οριακό πλαίσιο (Bounding Box) των διαθέσιμων δραστηριοτήτων και προσαρμόζει αυτόματα την εστίαση (Auto-zoom / Fit Bounds), ώστε όλα τα σημεία ενδιαφέροντος να είναι ορατά στην οθόνη του χρήστη. Παράλληλα, εφαρμόζεται ένας αλγόριθμος μαθηματικής μετατόμισης (coordinate offsetting) ο οποίος αποτρέπει την πλήρη επικάλυψη των δεικτών (markers) όταν πολλές δραστηριότητες ανήκουν στην ίδια πόλη. Τέλος, με τη χρήση του React-Leaflet-Cluster, τα κοντινά σημεία ομαδοποιούνται δυναμικά διατηρώντας τη διεπαφή καθαρή (decluttered).

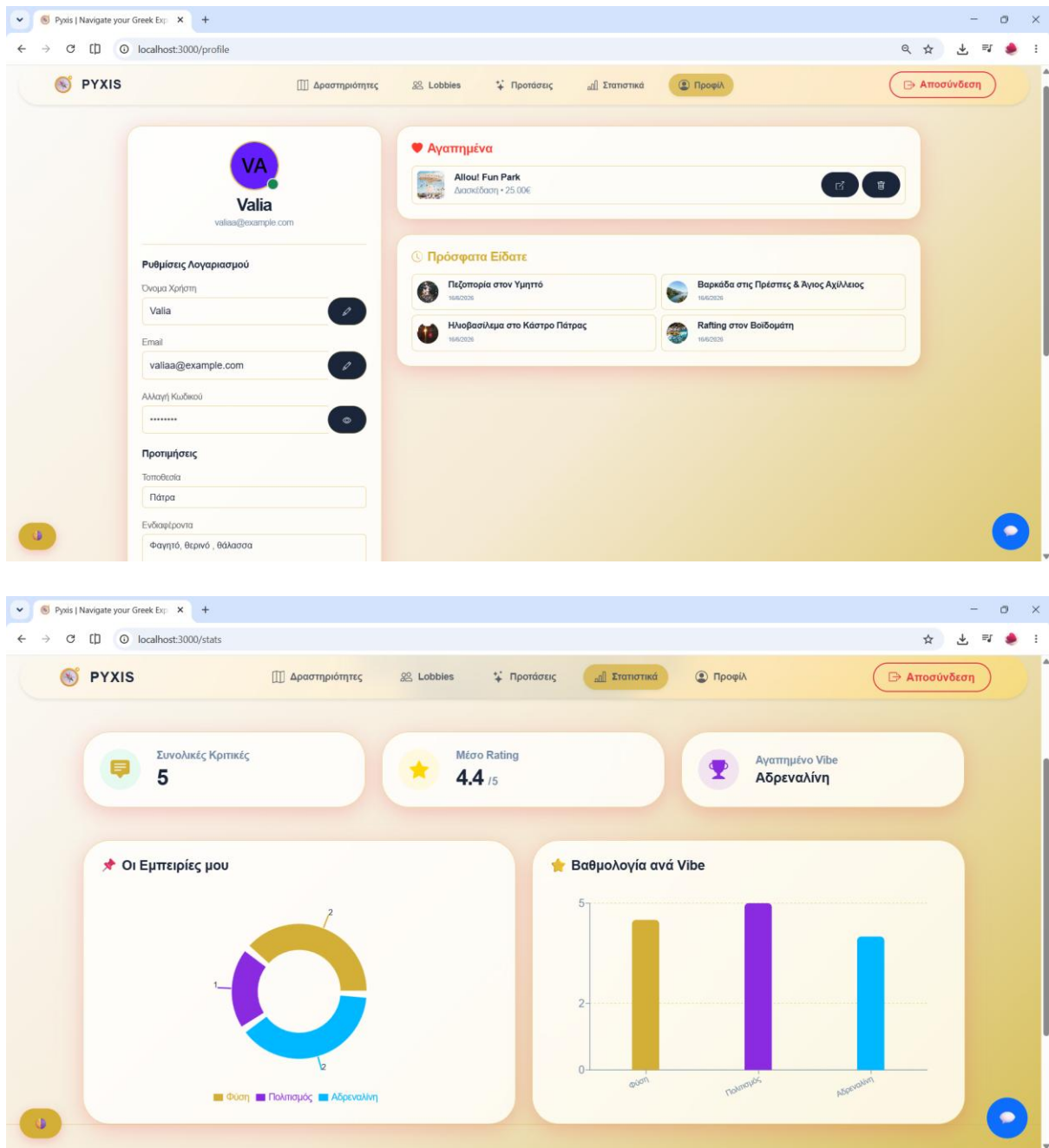


Εικόνα 5.8: Απεικόνιση του διαδραστικού χάρτη δραστηριοτήτων με δυναμική ομαδοποίηση.

5.2.9 Προφίλ Χρήστη, Στατιστικά και Προσωπικές Επιλογές (Profile, Recharts & Personal Choice)

Το σύστημα παρέχει σε κάθε ταυτοποιημένο χρήστη πρόσβαση στο προσωπικό του προφίλ. Μέσα από αυτή την ενότητα, ο χρήστης μπορεί να πλοηγηθεί στο ιστορικό των συμμετοχών του σε ομαδικές συνεδρίες, καθώς και να διαχειριστεί τις "αγαπημένες" του δραστηριότητες. Για την ομαλή εμπειρία χρήστη (UX) και την αποφυγή ακούσιων διαγραφών κατά τη διαχείριση των δεδομένων αυτών, έχουν υλοποιηθεί διαδραστικοί μηχανισμοί επιβεβαίωσης (custom confirmation toasts) κάνοντας χρήση της βιβλιοθήκης React Hot Toast.

Επιπλέον, βασικό στοιχείο της διεπαφής του προφίλ αποτελεί η οπτικοποίηση των δεδομένων. Για τον σκοπό αυτό, αξιοποιήθηκε η βιβλιοθήκη Recharts, η οποία αναλαμβάνει την παραγωγή διαδραστικών γραφημάτων (charts). Το σύστημα αντλεί τα δεδομένα του χρήστη και αναπαριστά γραφικά την κατανομή των δραστηριοτήτων του ανά κατηγορία [32], προσφέροντάς του μια άμεση και κατανοητή εικόνα των προσωπικών του τάσεων.

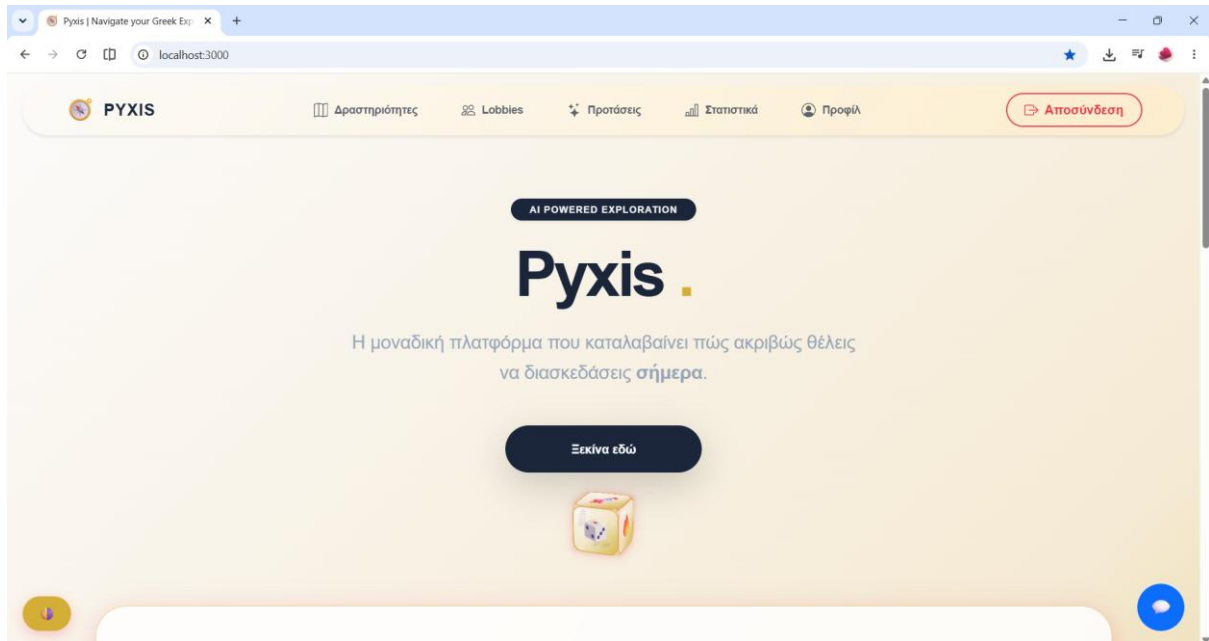


Εικόνα 5.9: Προβολή του προφίλ χρήστη και οπτικοποίηση στατιστικών μέσω Recharts.

5.2.10 Μηχανισμός Τυχαίας Επιλογής (Surprise Dice)

Για την περαιτέρω ενίσχυση της παιχνιδοποίησης (gamification) [12] και την άμεση αντιμετώπιση της αναποφασιστικότητας που συχνά χαρακτηρίζει τις ομάδες, η διεπαφή της εφαρμογής ενσωματώνει μια επιπλέον διαδραστική λειτουργία, το "Surprise Dice" (Ζάρι Έκπληξη) στην αρχική σελίδα. Μέσω αυτού του μηχανισμού, το σύστημα εκτελεί ένα τρισδιάστατο CSS animation κύλισης και ανασύρει εντελώς τυχαία μια δραστηριότητα από τη

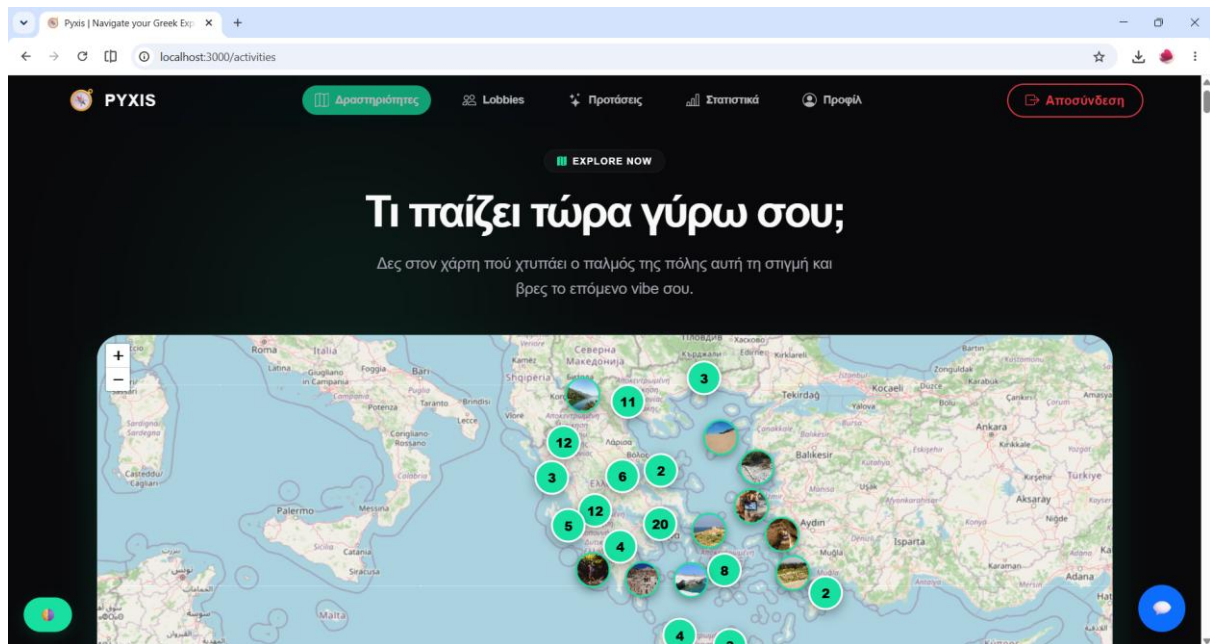
βάση δεδομένων. Η συγκεκριμένη λειτουργία προσφέρει μια εναλλακτική, άμεση και ευχάριστη διέξοδο σε περιπτώσεις όπου η παρέα δυσκολεύεται να καταλήξει σε συμφωνία.



Εικόνα 5.10: Μηχανισμός τυχαίας επιλογής (Surprise Dice).

5.2.11 Δυναμική Προσαρμογή Διεπαφής (Time-based Theming)

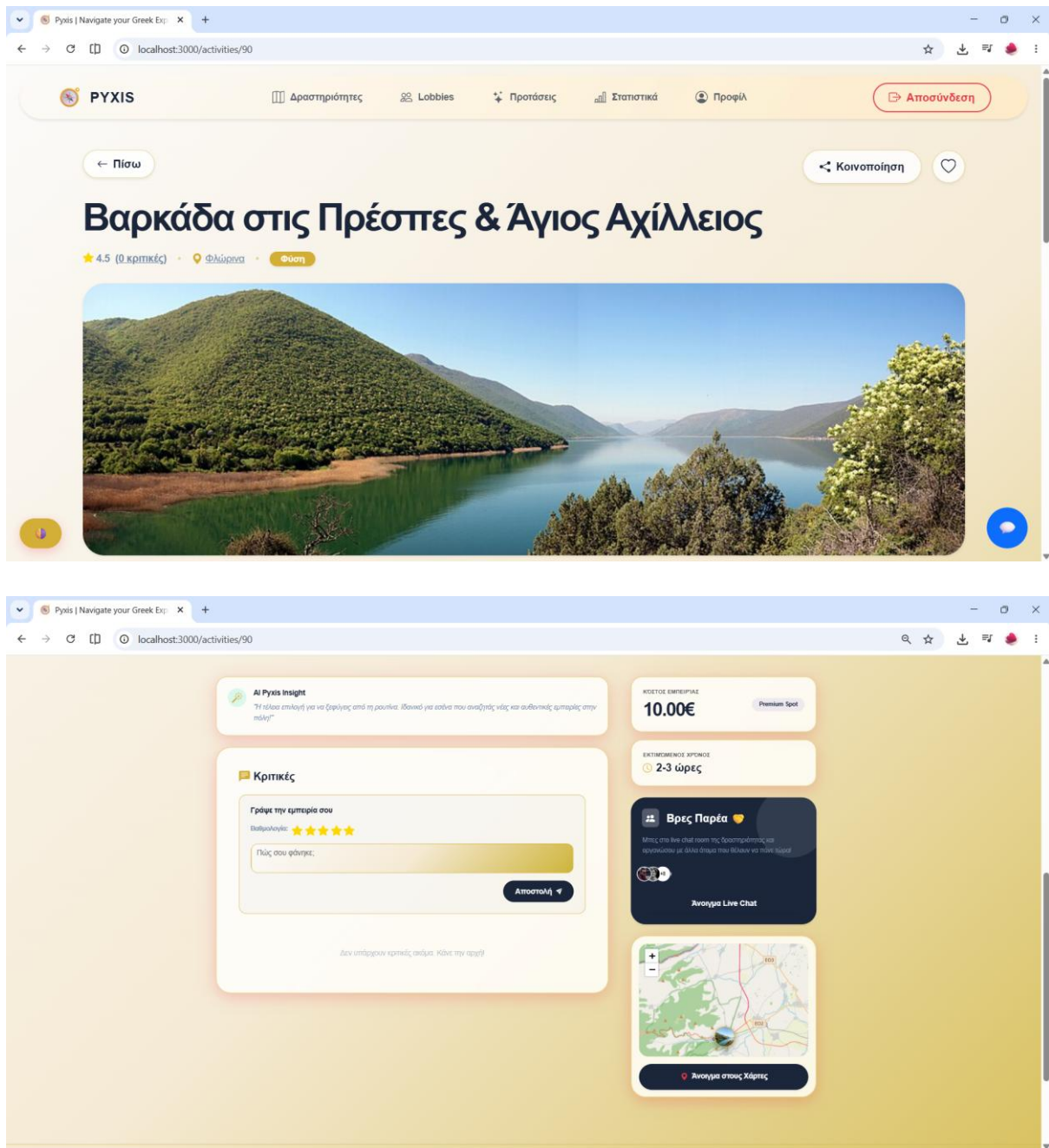
Μια σημαντική καινοτομία ως προς την εμπειρία χρήστη (UX) αποτελεί η δυναμική προσαρμογή του οπτικού θέματος της εφαρμογής βάσει του πραγματικού χρόνου. Το Frontend (App.jsx) αξιοποιεί συναρτήσεις JavaScript για τον έλεγχο της τοπικής ώρας της συσκευής του χρήστη κατά την εκκίνηση. Βάσει αυτού, εφαρμόζεται αυτόματα το κατάλληλο χρωματικό θέμα: «Πρωινό» (06:00 - 17:00), «Απογευματινό» (17:00 - 20:00) ή «Σκοτεινό / Βραδινό» (Dark Mode, 20:00 - 06:00). Η προσέγγιση αυτή βελτιώνει την αναγνώσιμότητα και μειώνει την καταπόνηση των ματιών, ενώ παρέχεται και ειδικό αιωρούμενο πλήκτρο για χειροκίνητη εναλλαγή (toggle).



Εικόνα 5.11: Το Σκοτεινό Θέμα (Dark Mode) της διεπαφής.

5.2.12 Κοινωνικός Διαμοιρασμός και Ενσωμάτωση AI (Social & AI Insights)

Στη σελίδα λεπτομερειών της εκάστοτε δραστηριότητας, προβάλλεται σε περίοπτη θέση το "AI Pyxis Insight", ένα δυναμικό κείμενο που παράγεται από το AI [6] και εξηγεί στους χρήστες με φιλικό τρόπο γιατί η συγκεκριμένη επιλογή ταιριάζει στα ενδιαφέροντά τους. Αφετέρου, ενσωματώνεται μηχανισμός άμεσου διαμοιρασμού (Share / Copy Link). Με τη χρήση του Web Clipboard API (navigator.clipboard), ο χρήστης μπορεί με ένα κλικ να αντιγράψει τις βασικές πληροφορίες της δραστηριότητας (Τίτλος, Τοποθεσία, Κόστος) μαζί με τον σύνδεσμο, διευκολύνοντας την πρόσκληση φίλων.

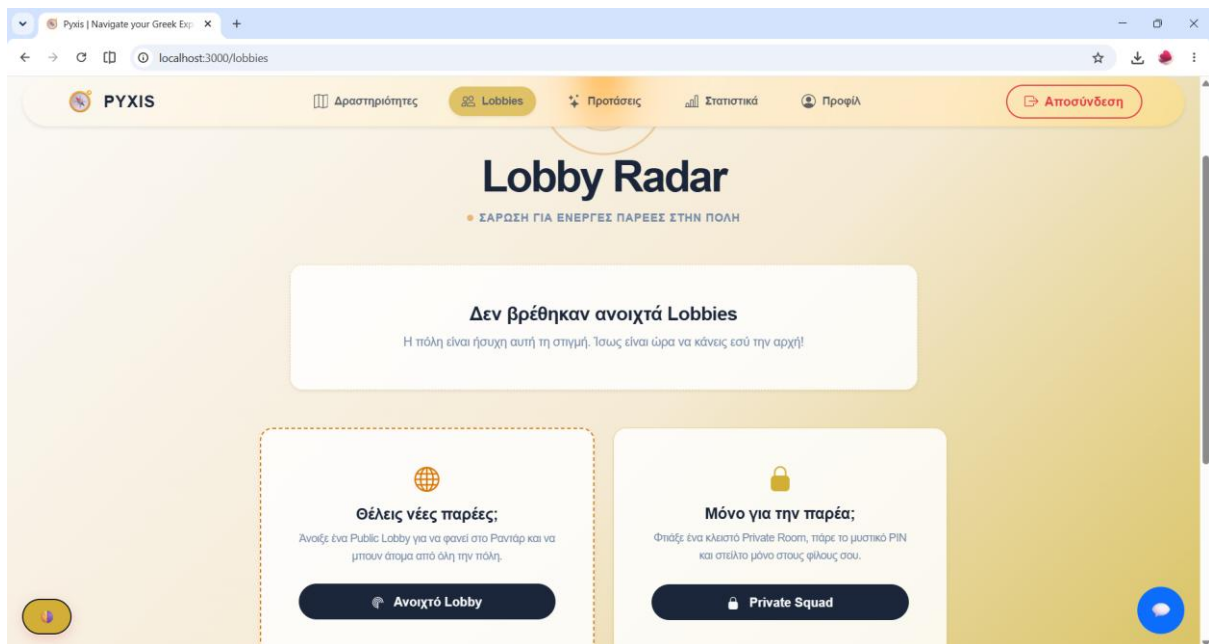


Εικόνα 5.12: Λεπτομέρειες δραστηριότητας, κοινωνικός διαμοιρασμός και AI Pyxis Insight.

5.2.13 Ανακάλυψη Ομάδων (Lobby Radar) και Πρόσβαση (PIN Access)

Η εφαρμογή διευκολύνει την κοινωνικοποίηση παρέχοντας ένα καινοτόμο **"Lobby Radar"** (Ραντάρ Ομάδων). Σε αυτή την οθόνη, το σύστημα σαρώνει την περιοχή και οι χρήστες μπορούν να εντοπίσουν σε πραγματικό χρόνο δημόσιες ομάδες (Public Lobbies) που είναι ενεργές στην πόλη τους, προκειμένου να ενταχθούν σε αυτές.

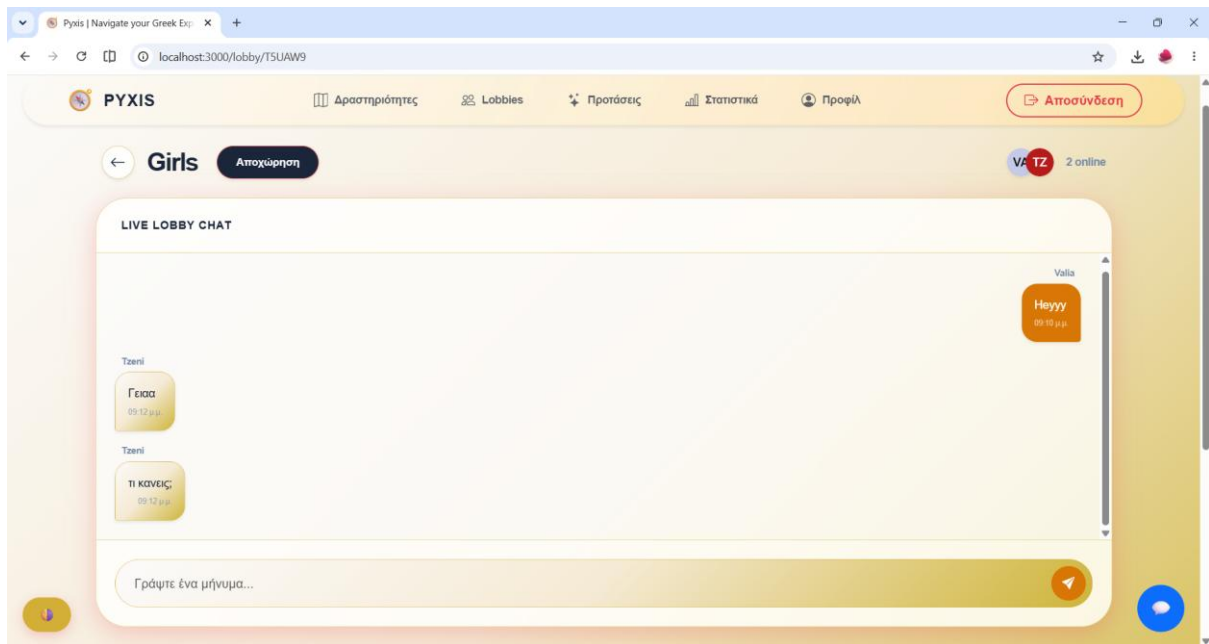
Παράλληλα, μέσα από την ίδια οθόνη διαχειρίζεται πλέον και η δημιουργία νέων δωματίων. Παρέχονται δύο διακριτές επιλογές: η δημιουργία ενός **"Ανοιχτού Lobby"** (το οποίο θα είναι ορατό στο ραντάρ για νέες παρέες) ή η δημιουργία ενός **"Private Squad"**. Στην περίπτωση του κλειστού δωματίου, η πρόσβαση των μελών επιτυγχάνεται αποκλειστικά μέσω του μυστικού 6-ψηφίου κωδικού (PIN) από την αρχική σελίδα, διασφαλίζοντας την πλήρη ιδιωτικότητα της ομάδας.



Εικόνα 5.13: Ανακάλυψη ενεργών ομάδων μέσω του Lobby Radar.

5.2.14 Ζωντανή Συνομιλία (Live Chat) σε Πραγματικό Χρόνο

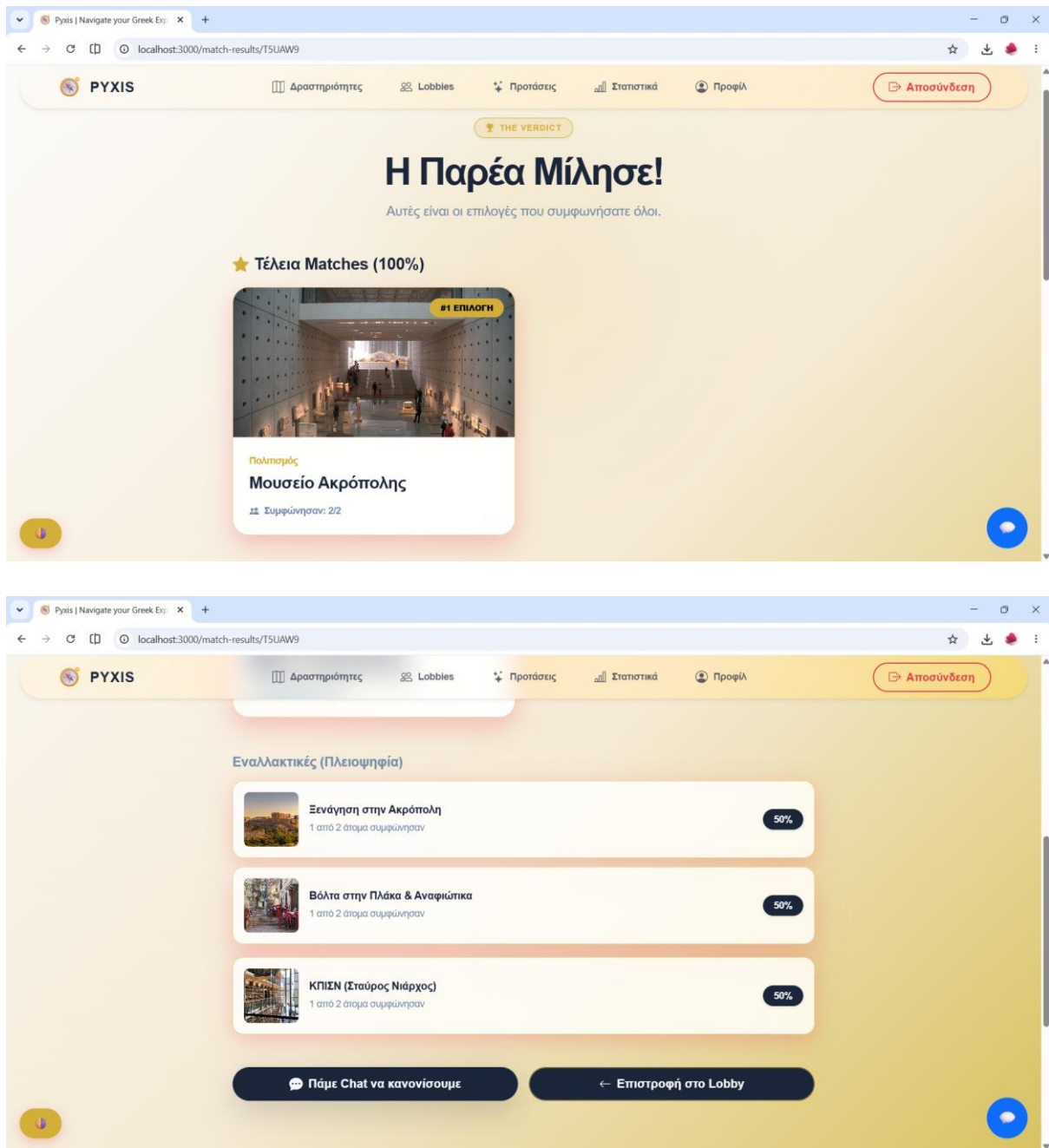
Για τη διευκόλυνση της οργάνωσης πριν την τελική επιλογή, το σύστημα ενσωματώνει δωμάτια ζωντανής συνομιλίας. Ένας εγγεγραμμένος χρήστης μπορεί να μεταβεί στο Live Chat του ιδιωτικού του Lobby ή σε ένα δημόσιο Chat Room που αφορά μια συγκεκριμένη δραστηριότητα (Review Section). Μέσω της τεχνολογίας Socket.IO, οι χρήστες βλέπουν ακαριαία ποιοι είναι συνδεδεμένοι (online status) και ανταλλάσσουν μηνύματα χωρίς καμία καθυστέρηση (Real-time socializing), επιλύοντας οργανωτικά ζητήματα πριν μεταβούν στη φάση της ψηφοφορίας.



Εικόνα 5.14: Ζωντανή ανταλλαγή μηνυμάτων της ομάδας.

5.2.15 Διαβαθμισμένα Αποτελέσματα Ομοφωνίας (Match Results)

Η διαδικασία της ψηφοφορίας (Swiping) καταλήγει σε μια εξελεγμένη οθόνη αποτελεσμάτων. Ο αλγόριθμος ταξινομεί τις επιλογές της ομάδας και προβάλλει την επιλογή που συγκέντρωσε απόλυτη ομοφωνία (100% Match). Σε περίπτωση που η ομάδα δεν καταφέρει να συμφωνήσει απόλυτα σε μία δραστηριότητα, το σύστημα δεν οδηγεί σε αδιέξοδο. Αντίθετα, αξιολογεί τα δεδομένα και προβάλλει εναλλακτικές επιλογές βάσει πλειοψηφίας (όπως δραστηριότητες που εγκρίθηκαν από το 80% ή 50% των μελών), προσφέροντας ευελιξία στη λήψη της τελικής απόφασης.

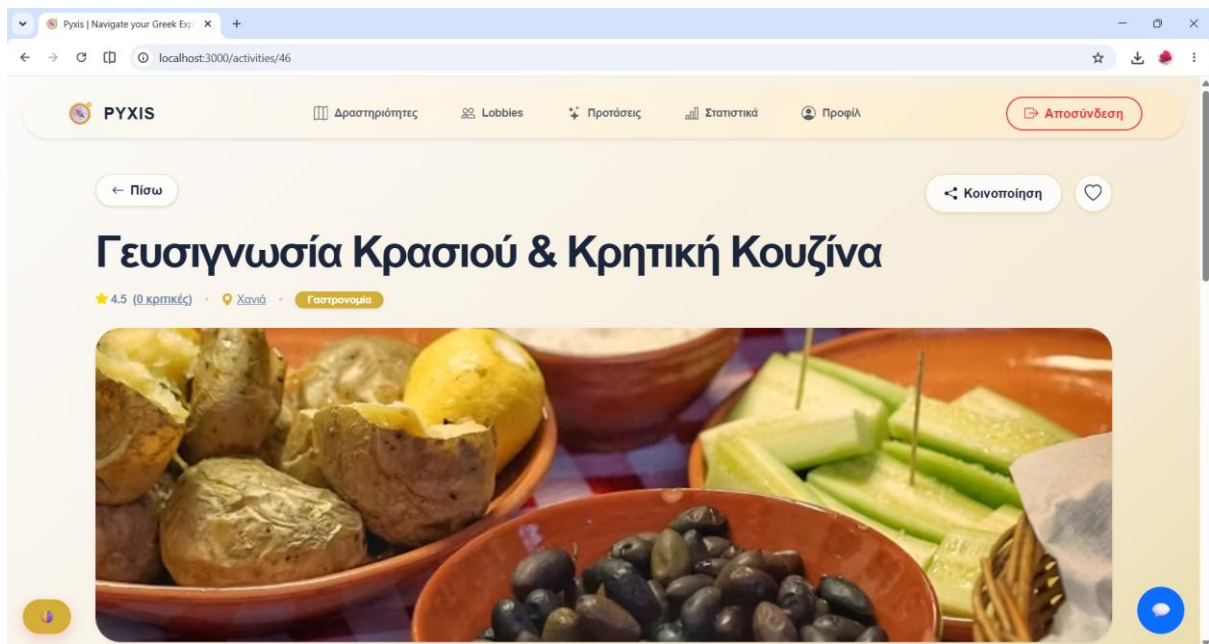


Εικόνα 5.15: Προβολή αποτελεσμάτων ομοφωνίας ή πλειοψηφίας.

5.2.16 Έξυπνη Διαχείριση Πολυμέσων (Smart Image Fallbacks)

Για να διασφαλιστεί η υψηλή αισθητική της διεπαφής ακόμα και όταν απουσιάζουν φωτογραφίες από τη βάση δεδομένων για κάποιες δραστηριότητες, αναπτύχθηκε ένας αλγόριθμος «έξυπνης» αναπλήρωσης εικόνων στο στοιχείο <ActivityCard>. Το σύστημα αναλύει δυναμικά λέξεις-κλειδιά από τον τίτλο (π.χ. «πλάκα», «βουνό», «λίμνη», «σινεμά») ή την κατηγορία της εκάστοτε δραστηριότητας χρησιμοποιώντας τη μέθοδο .find(), και της

αντιστοιχίζει αυτόματα μια θεματικά σχετική, υψηλής ανάλυσης εικόνα από εξωτερικές βιβλιοθήκες, διατηρώντας το οπτικό αποτέλεσμα άρτιο.



Εικόνα 5.16: Εφαρμογή του αλγορίθμου έξυπνης αναπλήρωσης εικόνων (Smart Fallbacks).

5.3 Διάθεση της Εφαρμογής στο Νέφος (Cloud Deployment)

Η μετάβαση μιας εφαρμογής από το τοπικό περιβάλλον ανάπτυξης (localhost) σε περιβάλλον παραγωγής (production) αποτελεί ένα κρίσιμο στάδιο στη μηχανική λογισμικού.

Για τη φιλοξενία του συστήματος Pyxis επιλέχθηκε η πλατφόρμα παροχής υπηρεσιών νέφους (Platform as a Service - PaaS) **Render** [25]. Η επιλογή της συγκεκριμένης πλατφόρμας βασίστηκε στην ευκολία διασύνδεσης (Continuous Integration / Continuous Deployment - CI/CD) με αποθετήρια κώδικα (όπως το GitHub), επιτρέποντας την αυτοματοποιημένη ενσωμάτωση και μεταγλώττιση αλλαγών σε κάθε push.

Το Backend (Node.js/Express) διαμορφώθηκε ως αυτόνομη διαδικτυακή υπηρεσία (Web Service), λαμβάνοντας τα απαραίτητα περιβαλλοντικά μεταβλητά (Environment Variables) μέσω αρχείου .env, όπως τα κλειδιά του Groq API, τα JWT μυστικά κλειδιά και τα διαπιστευτήρια της βάσης δεδομένων. Το Frontend (React.js) μεταγλωττίστηκε στατικά (npm run build) και εξυπηρετείται ως βελτιστοποιημένη εφαρμογή μίας σελίδας (Static Site). Η Βάση Δεδομένων φιλοξενείται σε απομακρυσμένο (cloud-based) διακομιστή MySQL, διασφαλίζοντας την αδιάλειπτη πρόσβαση (uptime) και τη διατήρηση των δεδομένων (persistence) των χρηστών.

Η επιτυχής ανάρτηση της εφαρμογής καθιστά το Pyxis ένα λειτουργικό προϊόν (Minimum Viable Product - MVP), άμεσα προσβάσιμο από το ευρύ κοινό μέσω διαδικτύου.

5.4 Σύνοψη

Στο τρέχον κεφάλαιο τεκμηριώθηκε η πρακτική εφαρμογή των θεωρητικών και σχεδιαστικών αρχών που τέθηκαν στα προηγούμενα κεφάλαια. Η προσεγμένη διεπαφή χρήστη, σε συνδυασμό με την ορθή λειτουργία της σε πραγματικό χρόνο και την επιτυχή της διάθεση στο cloud, αποδεικνύουν ότι η πλατφόρμα ανταποκρίνεται πλήρως στους στόχους που τέθηκαν κατά την έναρξη της παρούσας διπλωματικής.

Κεφάλαιο 6: Συμπεράσματα και Μελλοντικές Επεκτάσεις

6.1 Σύνοψη και Συμπεράσματα

Ο πρωταρχικός στόχος της παρούσας διπλωματικής εργασίας ήταν η μελέτη, ο σχεδιασμός και η ανάπτυξη ενός καινοτόμου πληροφοριακού συστήματος, ικανού να επιλύσει το πρακτικό πρόβλημα της ομαδικής λήψης αποφάσεων. Μέσα από την εξ ολοκλήρου υλοποίηση της διαδικτυακής εφαρμογής "Pyxis", αποδείχθηκε ότι η ενοποίηση σύγχρονων διαδικτυακών τεχνολογιών, μεθόδων παιχνιδοποίησης και Τεχνητής Νοημοσύνης μπορεί να μετασχηματίσει μια χρονοβόρα, και συχνά χαοτική, κοινωνική διαδικασία σε μια ευχάριστη, άμεση και διαδραστική εμπειρία.

Σε τεχνικό επίπεδο, το έργο ολοκληρώθηκε επιτυχώς, ικανοποιώντας πλήρως και υπερβαίνοντας τις αρχικές απαιτήσεις ενός ελάχιστα βιώσιμου προϊόντος (MVP - Minimum Viable Product). Η αρχιτεκτονική Πελάτη-Εξυπηρετητή (React.js [22] στο Frontend και Node.js [14] στο Backend) εξασφάλισε υψηλή απόκριση και επεκτασιμότητα, ενώ η επιτυχής ανάρτηση της εφαρμογής στο περιβάλλον υπολογιστικού νέφους (Render) [25] κατέστησε το σύστημα άμεσα διαθέσιμο. Κατά την ανάπτυξη της εφαρμογής Pyxis, αντιμετωπίστηκαν με

επιτυχία πολλαπλές τεχνικές προκλήσεις, αποδεικνύοντας τη δυναμική των σύγχρονων εργαλείων λογισμικού:

- **Αμφίδρομος Συγχρονισμός:** Η ενσωμάτωση του πρωτοκόλλου WebSockets (Socket.IO) [26] μεταμόρφωσε την εφαρμογή από ένα στατικό εργαλείο ψηφοφορίας σε μια ζωντανή πλατφόρμα (Live Chat & Real-time Swiping), γεφυρώνοντας το χάσμα μεταξύ της ψηφιακής διεπαφής και της πραγματικής ανθρώπινης αλληλεπίδρασης.
- **Πρακτική Εφαρμογή της AI:** Η διασύνδεση του συστήματος με το γλωσσικό μοντέλο LLaMA 3.3 μέσω του Groq API [27] δεν λειτούργησε απλώς ως ένα εργαλείο ελεύθερης συζήτησης, αλλά ως ένας ταχύτατος μηχανισμός παραγωγής δομημένων δεδομένων (JSON).
- **Υβριδική Βελτιστοποίηση και Επίγνωση Πλαισίου:** Η χρήση ενός τοπικού ευρετικού αλγορίθμου αξιολόγησης (Frontend Heuristic Scoring) σε συνδυασμό με την επεξεργαστική ισχύ του Backend, βελτιστοποίησε τους πόρους του συστήματος. Παράλληλα, η προσθήκη περιβαλλοντικού πλαισίου (καιρικές συνθήκες μέσω OpenWeather API) [28] απέδειξε πώς ένα μοντέλο AI μπορεί να γίνει πρακτικά ωφέλιμο, αποφεύγοντας τις γενικόλογες προτάσεις.

Συμπερασματικά, το Rychis αποτελεί μια ολοκληρωμένη λύση λογισμικού που γεφυρώνει το κενό μεταξύ των ατομικών συστημάτων συστάσεων και των εφαρμογών κοινωνικής δικτύωσης, ελαχιστοποιώντας τον κοινωνικό δισταγμό και προάγοντας την ομοφωνία.

6.2 Μελλοντικές Επεκτάσεις

Παρότι το σύστημα Rychis είναι πλήρως λειτουργικό και έτοιμο για παραγωγή, η αρχιτεκτονική του σχεδιάστηκε με τρόπο που επιτρέπει τη συνεχή εξέλιξη και προσθήκη νέων δυνατοτήτων. Ορισμένες προτάσεις για τη μελλοντική επέκταση της εφαρμογής περιλαμβάνουν:

1. **Ανάπτυξη Εγγενούς Εφαρμογής Κινητών Συσκευών (Native Mobile App):** Η μετάβαση από μια διαδικτυακή εφαρμογή (Web App) σε μια εγγενή εφαρμογή για λειτουργικά συστήματα iOS και Android (π.χ. με χρήση React Native ή Flutter) θα βελτιώσει περαιτέρω την εμπειρία χρήστη, αξιοποιώντας άμεσα τις ειδοποιήσεις ώθησης (push notifications) και την τοποθεσία GPS της συσκευής.

2. **Διασύνδεση με Εξωτερικά APIs Υπηρεσιών:** Η επέκταση του συστήματος ώστε να αντλεί δεδομένα σε πραγματικό χρόνο από πλατφόρμες όπως το Google Maps (για ακριβή τοποθεσία και ωράρια), το TripAdvisor (για πραγματικές κριτικές) και το OpenTable ή το e-food (για απευθείας κρατήσεις και παραγγελίες μέσα από το app, αμέσως μόλις επιτευχθεί Match).
3. **Προηγμένη Εξατομίκευση μέσω Μηχανικής Μάθησης (Machine Learning):** Η εφαρμογή αλγορίθμων Συνεργατικού Φιλτραρίσματος (Collaborative Filtering) στο ιστορικό των επιλογών (swipes) των χρηστών, ώστε το σύστημα να προτείνει προληπτικά (proactively) δραστηριότητες που ταιριάζουν στο προφίλ της εκάστοτε ομάδας, πριν καν αυτές ζητηθούν από το Chatbot.
4. **Υποστήριξη Πολυγλωσσικότητας και Προσβασιμότητας:** Προσθήκη δυνατοτήτων διεθνοποίησης (Internationalization - i18n) για την υποστήριξη πολλαπλών γλωσσών, καθώς και βελτιστοποίηση της διεπαφής για άτομα με προβλήματα όρασης (Screen Readers, High Contrast Modes), καθιστώντας την εφαρμογή πλήρως συμπεριληπτική και προσβάσιμη σε ένα ευρύτερο κοινό.

7. Βιβλιογραφία

Επιστημονικά Άρθρα, Βιβλία και Έρευνες

- [1] J. Masthoff, "Group Recommender Systems: Combining Individual Models," in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston, MA: Springer US, 2011, pp. 677–702.
- [2] A. Jameson and B. Smyth, "Recommendation to Groups," in The Adaptive Web: Methods and Strategies of Web Personalization, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 596–627.
- [3] L. Boratto and S. Carta, "State-of-the-Art in Group Recommendation and New Approaches for Automatic Identification of Groups," in Information Retrieval and Mining in Distributed Environments, A. Soro, E. Vargiu, G. Armano, and G. Paddeu, Eds. Berlin, Heidelberg: Springer, 2010, pp. 1-20.
- [4] M. O'Connor, D. Cosley, J. A. Konstan, and J. Riedl, "PolyLens: A recommender system for groups of users," in Proceedings of the Seventh European Conference on Computer Supported Cooperative Work (ECSCW), 2001, pp. 199-218.
- [5] P. Resnick and H. R. Varian, "Recommender systems," Communications of the ACM, vol. 40, no. 3, pp. 56-58, Mar. 1997.
- [6] W. Fan et al., "Recommender Systems in the Era of Large Language Models (LLMs)," arXiv preprint arXiv:2307.02046, 2023.
- [7] D. Jannach, A. Manzoor, W. Cai, and L. Chen, "A Survey on Conversational Recommender Systems," ACM Comput. Surv., vol. 54, no. 5, pp. 1–36, May 2021.
- [8] T. Brown et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877-1901, 2020.
- [9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Blog, 2019.
- [10] G. David and C. Cambre, "Screened Intimacies: Tinder and the Swipe Logic," Social Media + Society, vol. 2, no. 2, Apr. 2016.

- [11] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: defining 'gamification'," in Proceedings of the 15th International Academic MindTrek Conference, Sep. 2011, pp. 9–15.
- [12] J. Hamari, J. Koivisto, and H. Sarsa, "Does Gamification Work? - A Literature Review of Empirical Studies on Gamification," in 2014 47th Hawaii International Conference on System Sciences, 2014, pp. 3025-3034.
- [13] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [14] D. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, Nov. 2010.
- [15] M. Haverbeke, Eloquent JavaScript: A Modern Introduction to Programming, 3rd ed. San Francisco, CA: No Starch Press, 2018.
- [16] S. Newman, Building Microservices: Designing Fine-Grained Systems, 1st ed. Sebastopol, CA: O'Reilly Media, 2015.
- [17] M. Richards, Software Architecture Patterns. Sebastopol, CA: O'Reilly Media, 2015.
- [18] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," RFC 3986, Jan. 2005.
- [19] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY: McGraw-Hill Education, 2019.
- [20] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA: Addison-Wesley, 2003.

Τεχνολογίες, Πλαίσια Ανάπτυξης και Εργαλεία

- [21] Mozilla Developer Network (MDN), "Web Technology for Developers," 2024. [Online]. Available: <https://developer.mozilla.org/>
- [22] Meta Platforms Inc., "React – A JavaScript library for building user interfaces," 2024. [Online]. Available: <https://react.dev/>

- [23] OpenJS Foundation, "Express - Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>
- [24] JWT.IO, "JSON Web Tokens - Introduction," 2024. [Online]. Available: <https://jwt.io/introduction>
- [25] Render, "Cloud Application Hosting for Developers," 2024. [Online]. Available: <https://render.com/>
- [26] Socket.IO, "Real-time bidirectional event-based communication," 2024. [Online]. Available: <https://socket.io/>
- [27] Groq Inc., "Groq - The LPU Inference Engine," 2024. [Online]. Available: <https://groq.com/>
- [28] OpenWeather Ltd., "OpenWeather API - Weather data and forecast API," 2024. [Online]. Available: <https://openweathermap.org/api>
- [29] Postman Inc., "Postman API Platform," 2024. [Online]. Available: <https://www.postman.com/>
- [30] PremiumSoft CyberTech Ltd., "Navicat - Database Management," 2024. [Online]. Available: <https://www.navicat.com/>
- [31] V. Agafonkin, "Leaflet - an open-source JavaScript library for mobile-friendly interactive maps," 2024. [Online]. Available: <https://leafletjs.com/>
- [32] Recharts, "A composable charting library built on React components," 2024. [Online]. Available: <https://recharts.org/>
- [33] Framer B.V., "Framer Motion - A production-ready motion library for React," 2024. [Online]. Available: <https://www.framer.com/motion/>

Πηγές Πολυμέσων και Φωτογραφικού Υλικού

Το σύνολο του φωτογραφικού υλικού που χρησιμοποιήθηκε για την αναπαράσταση των δραστηριοτήτων και των τοποθεσιών προέρχεται αρχικά από αποθετήρια ελεύθερης χρήσης. Διανέμεται υπό άδειες Creative Commons (όπως CC BY-SA 4.0, CC BY 2.0) ή ανήκει στον δημόσιο τομέα (CC0). Ενδεικτικά αξιοποιήθηκαν τα αποθετήρια Wikimedia Commons και Unsplash.

Ωστόσο, για τη βέλτιστη διαχείριση των πόρων (assets) και τη διατήρηση της Σχεσιακής Βάσης Δεδομένων (MySQL) σε χαμηλό μέγεθος, οι εικόνες δεν αποθηκεύονται τοπικά. Αντιθέτως, αναρτήθηκαν και φιλοξενούνται στη δωρεάν υπηρεσία Image Hosting ImgBB (στο δημόσιο άλμπουμ: <https://ibb.co/album/8gXZKC>). Από εκεί εξάγονται οι απευθείας σύνδεσμοι (direct URLs), οι οποίοι και αποθηκεύονται στο πεδίο `image_url` του πίνακα `activities` της βάσης δεδομένων, εξασφαλίζοντας ταχύτατη φόρτωση (CDN) κατά την προβολή των καρτών στο Frontend.

- **Wikimedia Commons** (<https://commons.wikimedia.org/>): Επιλεγμένες φωτογραφίες ιστορικών και φυσικών μνημείων της Ελλάδας (π.χ. Υμηττός, Εθνικός Κήπος, Μουσείο Ακρόπολης, Κάστρο Πάτρας, Λίμνη Βουλιαγμένης).
- **Unsplash** (<https://unsplash.com/>): Εικόνες υψηλής ανάλυσης για θεματικές κατηγορίες δραστηριοτήτων, με χρήση έξυπνων αλγορίθμων αναπλήρωσης (image fallbacks) στο Frontend.

Παράρτημα Α: Σημαντικά Αποσπάσματα

Πηγαίου Κώδικα

Στο παρόν παράρτημα παρατίθενται τα πιο αξιοσημείωτα αποσπάσματα του πηγαίου κώδικα της εφαρμογής Ryxis. Διαχωρίζονται σε τέσσερις βασικούς πυλώνες: Τεχνητή Νοημοσύνη, Διαχείριση Βάσης και Ομοφωνίας, Real-Time Διεπαφή Χρήστη, και Διαδραστική Χαρτογράφηση.

Ενότητα Α.1: Υλοποίηση Τεχνητής Νοημοσύνης (Backend - ai.js)

Απόσπασμα 1: Αλγόριθμος Καθαρισμού και Κανονικοποίησης Γεωγραφικών

Δεδομένων

Πριν την αποστολή των δεδομένων στο LLM, εκτελείται τοπική συνάρτηση κανονικοποίησης (normalization) που αφαιρεί τόνους, ειδικούς χαρακτήρες και κεφαλαία γράμματα, επιτρέποντας την ορθή σύγκριση τοποθεσιών.

```
// φίλτρο για τοποθεσία (αγνοούμε τόνους και κεφαλαία)
if (location) {
  // συνάρτηση που αφαιρεί τόνους και κάνει τα γράμματα πεζά
  const normalize = (text) => {
    return text ? text.normalize("NFD").replace(/[\u0300-\u036f]/g, "").toLowerCase() : "";
  };

  const locClean = normalize(location).trim();

  filteredActs = filteredActs.filter(a => {
    const dbLocClean = normalize(a.location);
    return dbLocClean.includes(locClean) || locClean.includes(dbLocClean);
  });
}
```

Απόσπασμα 2: Παραμετροποίηση Μοντέλου (System Prompt Engineering) για το AI Suggestion

Το System Prompt καθορίζει τον ρόλο του μοντέλου ως ταξιδιωτικού συμβούλου και επιβάλλει αυστηρούς ευρετικούς κανόνες βαθμολόγησης βάσει καιρού και σημασιολογικής συνάφειας.

```
// οδηγίες προς την AI
const systemPrompt = `
Είσαι ο "Pyxis AI", ένας κορυφαίος ταξιδιωτικός σύμβουλος για την Ελλάδα.
ΣΗΜΑΝΤΙΚΟ: Τα δεδομένα που λαμβάνεις έχουν ήδη περάσει από αυστηρό έλεγχο budget και τοποθεσίας. Μην ασχολείσαι με αυτούς τους παράγοντες.

Ο ΡΟΛΟΣ ΣΟΥ:
Βαθμολόγησε (ai_score 50-100) τις διαθέσιμες δραστηριότητες με βάση ΜΟΝΟ τον Καιρό και τα Ενδιαφέροντα του χρήστη.

ΚΑΝΟΝΕΣ ΒΑΘΜΟΛΟΓΗΣΗΣ:
1. ΚΑΙΡΟΣ: Αν ο καιρός είναι "Rain", "Snow" ή "Thunderstorm", δώσε μεγάλο bonus (+30) σε δραστηριότητες εσωτερικού χώρου (outdoor=0) και ποινή (-30) σε εξωτερικού (outdoor=1).
2. ΕΝΔΙΑΦΕΡΟΝΤΑ: Ταίριαξε σημασιολογικά τα ενδιαφέροντα του χρήστη με τα "tags" ή τον τίτλο της δραστηριότητας (+40 πόντους για τέλειο ταίριασμα).

ΚΑΝΟΝΕΣ ΕΞΟΔΟΥ:
- Πρέπει να επιστρέφεις ΑΥΣΗΤΑ ένα JSON object.
- Για κάθε πρόταση, γράψε έναν ελκυστικό, φιλικό λόγο (reason) στα Ελληνικά (max 15 λέξεις) που να εξηγεί ΓΙΑΤΙ ταιριάζει στον χρήστη.
- Επέστρεψε ΜΟΝΟ όσες δραστηριότητες έχουν ai_score >= 50.

ΜΟΡΦΗ JSON ΠΟΥ ΑΠΑΙΤΕΙΤΑΙ:
{
  "matches": [
    { "id": 1, "ai_score": 95, "reason": "Ιδανικό για βροχερή μέρα και ταιριάζει τέλεια στην αγάπη σου για την ιστορία!" }
  ]
}
`;

const userPrompt = `
ΔΕΔΟΜΕΝΑ ΧΡΗΣΤΗ:
- Καιρός: "${weather || 'Clear'}"
- Ενδιαφέροντα / Διάθεση: "${interests || 'Οτιδήποτε ενδιαφέρον'}"

ΔΙΑΘΕΣΙΜΕΣ ΔΡΑΣΤΗΡΙΟΤΗΤΕΣ (JSON):
${JSON.stringify(simpleActs)}
`;
```

Απόσπασμα 3: Ασφαλής Σύνδεση με το Groq SDK και Επιβολή Δομημένης Εξόδου (Structured JSON)

Η κλήση προς το μοντέλο LLaMA 3.3 χρησιμοποιεί την ιδιότητα response_format για την εξασφάλιση έγκυρης δομής JSON, αποτρέποντας σφάλματα κατά τη μετατροπή (parsing).

```
const completion = await groq.chat.completions.create({
  messages: [
    { role: "system", content: systemPrompt },
    { role: "user", content: userPrompt }
  ],
  model: "llama-3.3-70b-versatile",
  temperature: 0.1,
  response_format: { type: "json_object" }
});

const parsed = JSON.parse(completion.choices[0]?.message?.content || "{}");
```

Απόσπασμα 4: Περιορισμός Ψευδαισθήσεων (Hallucination Control) στη Λειτουργία του Chatbot

Για την αποτροπή προβολής ανύπαρκτων δεδομένων, το Chatbot τροφοδοτείται με τη λίστα των πραγματικών δραστηριοτήτων της MySQL και δεσμεύεται από αυστηρούς κανόνες συμπεριφοράς.

```
// λειτουργία chatbot
router.post("/chatbot", async (req, res) => {
  const { message } = req.body;

  try {
    const [activities] = await db.query("SELECT id, title, location FROM activities");

    // φτιάχνουμε μια λίστα με τα links για το prompt
    const activitiesList = activities.map(a => `-${a.title} (Περιοχή: ${a.location}) -> LINK: {{LINK:/activities/${a.id}}}`).join("\n");

    const systemPrompt = `
Είσαι ο Pyxis, ένας ταξιδιωτικός βοηθός.
ΜΙΑΣ ΑΥΣΤΗΡΑ ΚΑΙ ΜΟΝΟ ΕΛΛΗΝΙΚΑ. ΑΠΑΓΟΡΕΥΕΤΑΙ ΝΑ ΧΡΗΣΙΜΟΠΟΙΗΣΕΙΣ ΑΓΓΛΙΚΑ, ΙΤΑΛΙΚΑ Ή ΑΛΛΕΣ ΓΛΩΣΣΕΣ.

ΛΙΣΤΑ ΔΙΑΘΕΣΙΜΩΝ ΔΡΑΣΤΗΡΙΟΤΗΤΩΝ (ΑΠΑΓΟΡΕΥΕΤΑΙ ΝΑ ΠΡΟΤΕΙΝΕΙΣ ΚΑΤΙ ΠΟΥ ΔΕΝ ΕΙΝΑΙ ΕΔΩ):
${activitiesList}

ΚΑΝΟΝΕΣ:
1. ΑΝ ο χρήστης ζητήσει μια περιοχή (π.χ. Καλαμάτα) και δεν βλέπεις τη λέξη αυτή στην παραπάνω λίστα, ΠΕΣ ΜΟΝΟ: "Δυστυχώς δεν έχω προτάσεις για αυτή την περιοχή."
2. ΑΠΑΓΟΡΕΥΕΤΑΙ να προτείνεις μέρη από άλλες πόλεις αν δεν βρεις αυτό που ζητάει.
3. ΠΟΤΕ μην γράφεις τη λέξη "ID" ή νούμερα όπως "ID: 11".
4. ΟΤΑΝ προτείνεις κάτι, αντέγραψε ακριβώς το LINK δίπλα στο όνομα. Παράδειγμα: "Σου προτείνω το Κάστρο {{LINK:/activities/11}}"
5. ΟΤΑΝ προτείνεις 2 ή περισσότερες δραστηριότητες, να τις παρουσιάζεις ΠΑΝΤΑ σε μορφή λίστας (με μια παύλα "-" στην αρχή). Πρέπει ΑΥΣΤΗΡΑ να βάζεις την κάθε δραστηριότητα σε νέα γραμμή.
`;

    const response = await groq.chat.completions.create({
      messages: [
        { role: "system", content: systemPrompt },
        { role: "user", content: message }
      ],
      model: "llama-3.3-70b-versatile",
      temperature: 0.0 // απόλυτη υπακοή στους κανόνες, καθόλου φαντασία
    });

    res.json({ reply: response.choices[0].message?.content });
  } catch (e) {
    res.json({ reply: "Παρουσιάστηκε πρόβλημα στη σύνδεση." });
  }
});

module.exports = router;
```

Ενότητα Α.2: Επιχειρησιακή Λογική και SQL Queries (Backend - group.js)

Απόσπασμα 5: Καταγραφή Ψήφων με Αποτροπή Διπλοεγγραφών (Upsert Logic)

Η χρήση της εντολής ON DUPLICATE KEY UPDATE διασφαλίζει ότι αν ένας χρήστης αλλάξει γνώμη κατά τη σάρωση (swipe), η προηγούμενη ψήφος του θα ενημερωθεί χωρίς να δημιουργηθεί πλεονάζουσα εγγραφή στη βάση.

```
await db.query(`
  INSERT INTO group_votes (session_id, user_id, activity_id, vote_type)
  VALUES (?, ?, ?, ?)
  ON DUPLICATE KEY UPDATE vote_type = ?
`, [sessionId, userId, activityId, voteType, voteType]);
```

Απόσπασμα 6: Αλγόριθμος Real-Time Υπολογισμού Απόλυτης Ομοφωνίας (Match)

Κάθε φορά που καταγράφεται ένα θετικό Vibe (like), το σύστημα ελέγχει δυναμικά αν ο αριθμός των συγκεντρωμένων Likes για τη συγκεκριμένη δραστηριότητα ταυτίζεται με το σύνολο των ενεργών μελών του δωματίου.

```
if (voteType === 'like') {
  // μετράμε πόσοι είμαστε στο δωμάτιο
  const [totalUsersRes] = await db.query(`
    SELECT COUNT(DISTINCT user_id) as totalUsers
    FROM group_votes
    WHERE session_id = ?
  `, [sessionId]);
  const totalUsers = totalUsersRes[0].totalUsers;

  // μετράμε πόσα likes έχει αυτό το activity
  const [likesRes] = await db.query(`
    SELECT COUNT(*) as totalLikes
    FROM group_votes
    WHERE session_id = ? AND activity_id = ? AND vote_type = 'like'
  `, [sessionId, activityId]);
  const totalLikes = likesRes[0].totalLikes;

  // αν όλοι πάτησαν like, έχουμε match!
  if (totalUsers > 1 && totalLikes === totalUsers) {
    isMatch = true;
  }
}
```

Απόσπασμα 7: Σύνθετο SQL Query για τον Υπολογισμό των Match Results (Ranking)

Σε περίπτωση που δεν επιτευχθεί απόλυτη ομοφωνία, το ερώτημα αυτό εκτελεί ομαδοποίηση (aggregation) των ψήφων ανά δραστηριότητα, ανακτώντας τον συνολικό αριθμό ανά τύπο ψήφου (like/dislike). Τα πρωτογενή αυτά δεδομένα αξιοποιούνται στη συνέχεια από το επίπεδο της εφαρμογής (Backend) για την τελική ταξινόμηση των δραστηριοτήτων και τον υπολογισμό του ποσοστού αποδοχής.

```
const [summary] = await db.query(`
  SELECT a.title, v.vote_type, COUNT(v.user_id) as vote_count
  FROM group_votes v
  JOIN activities a ON v.activity_id = a.id
  WHERE v.session_id = ?
  GROUP BY v.activity_id, v.vote_type
`, [sessionId]);
```

Ενότητα Α.3: Σύνδεση WebSockets και Mobile Gestures (Frontend - GroupSwipePage.jsx)

Απόσπασμα 8: Διαχείριση Real-Time Γεγονότων μέσω Socket.IO Client

Η εφαρμογή (μέσω της React) ακούει διαρκώς το κανάλι επικοινωνίας του δωματίου. Μόλις ληφθεί σήμα επίτευξης ομοφωνίας (match) από τον διακομιστή, διακόπτεται η τρέχουσα ροή αξιολόγησης (swiping) και εμφανίζεται δυναμικά το αναδυόμενο παράθυρο επιτυχίας (Match Pop-up).

```
// αρχική φόρτωση και σύνδεση στο socket
useEffect(() => {
  socket.emit('join_group', sessionId);

  const loadActivities = async () => {
    try {
      const token = localStorage.getItem("token");
      const res = await axios.get(`/api/group/activities/${sessionId}`, {
        headers: { Authorization: `Bearer ${token}` }
      });
      setActivities(res.data);
    } catch (err) {
      console.error("Σφάλμα κατά τη φόρτωση καρτών:", err);
    } finally {
      // Τελείωσε το ψάξιμο
      setLoading(false);
    }
  };
  loadActivities();

  // ακούμε για live matches από άλλους χρήστες
  socket.on('receive_swipe', (data) => {
    if (data.match) {
      setActivities(prev => {
        const matchedAct = prev.find(a => Number(a.id) === Number(data.activityId));
        if (matchedAct) {
          setMatches(prevMatches => {
            if (prevMatches.find(m => m.id === matchedAct.id)) return prevMatches;
            setLatestMatch(matchedAct);
            setShowMatch(true);
            return [...prevMatches, matchedAct];
          });
        }
        return prev;
      });
    }
  });

  return () => socket.off('receive_swipe');
}, [sessionId]);
```

Απόσπασμα 9: Μαθηματικός Υπολογισμός Κίνησης για Custom Swipe Χειρονομίες

Ανάπτυξη αλγορίθμου μέτρησης γεωγραφικού διανύσματος αφής (Touch Events) σε συσκευές smartphone, επιτρέποντας την αναγνώριση Swipe Left και Swipe Right χωρίς εξωτερικές βιβλιοθήκες.

```
const startSwipe = (e) => {
  touchStartX.current = e.clientX || (e.touches && e.touches[0].clientX);
};

const endSwipe = (e) => {
  touchEndX.current = e.clientX || (e.changedTouches && e.changedTouches[0].clientX);
  const swipeDistance = touchStartX.current - touchEndX.current;

  if (swipeDistance > 60) {
    vote('dislike');
  } else if (swipeDistance < -60) {
    vote('like');
  }
};
```

Ενότητα A.4: Διαδραστική Χαρτογράφηση και Γεωμετρία (Frontend - Map.js)

Απόσπασμα 10: Αλγόριθμος Πρόληψης Επικάλυψης Pins (Marker Displacement Logic)

Όταν πολλαπλές δραστηριότητες αντιστοιχίζονται στην ίδια γεωγραφική περιοχή (π.χ. ίδια πόλη), οι συντεταγμένες τους συμπίπτουν, καθιστώντας τα επιμέρους στοιχεία μη προσβάσιμα στον χάρτη. Για την επίλυση αυτού του προβλήματος, αναπτύχθηκε ένας αλγόριθμος διακριτής διασποράς, ο οποίος εφαρμόζει ελεγχόμενες αποκλίσεις (offsets) στις συντεταγμένες των Marker στοιχείων με βάση το μοναδικό αναγνωριστικό (id) της κάθε δραστηριότητας.

```
// μαθηματικό offset για να μη στοιβάζονται τα pins
const offsetLat = (Number(activity.id) % 5) * OFFSET_LAT_STEP - OFFSET_LAT_BASE;
const offsetLng = (Number(activity.id) % 3) * OFFSET_LNG_STEP - OFFSET_LNG_BASE;
const finalPos = [position[0] + offsetLat, position[1] + offsetLng];

return (
  <Marker key={activity.id} position={finalPos} icon={createZenlyIcon(activity)}>
    <Popup closeButton={false}>
      <div style={{ textAlign: "center" }}>
        <div style={{ position: "relative" }}>
          <img
            src={getImage(activity)}
            alt={activity.title}
            style={{
              width: "100%",
              height: "130px",
              objectFit: "cover",
              display: "block",
            }}
          />
          <span className="badge bg-dark bg-opacity-75 position-absolute bottom-0 end-0 m-2">
            {Number(activity.cost) === 0 ? "Free" : `${activity.cost}€`}
          </span>
        </div>
      </div>
    </Popup>
  </Marker>
)
```

Ενότητα Α.5: Ασφάλεια και Αρχιτεκτονική Υποδομή

Απόσπασμα 11: Προστασία Διαδρομών (Middleware Αυθεντικοποίησης)

Το παρακάτω τμήμα κώδικα αποτελεί το ενδιάμεσο λογισμικό (middleware) που αναλαμβάνει την προστασία των ιδιωτικών διαδρομών (routes) του συστήματος (π.χ. Προφίλ Χρήστη, Δημιουργία Ομάδας). Ελέγχει την ύπαρξη και την εγκυρότητα του JSON Web Token (JWT) μέσω του μυστικού κλειδιού (JWT_SECRET), αποτρέποντας την πρόσβαση σε μη εξουσιοδοτημένους χρήστες με την επιστροφή κατάλληλων HTTP Status Codes (401 και 403).

```

const jwt = require("jsonwebtoken");
const JWT_SECRET = process.env.JWT_SECRET;

// middleware που προστατεύει τα routes, ελέγχοντας αν ο χρήστης είναι συνδεδεμένος
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];

  if (!token) {
    return res.status(401).json({ error: "Δεν παρέχεται token πρόσβασης (Unauthorized)" });
  }

  // επαλήθευση του token
  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: "Μη έγκυρο ή ληγμένο token (Forbidden)" });
    }
    // περνάμε τον χρήστη στα επόμενα routes μέσω του req
    req.user = user;
    next();
  });
}

module.exports = authenticateToken;

```

Απόσπασμα 12: Κρυπτογράφηση Κωδικών (bcrypt)

Για τη διασφάλιση της ιδιωτικότητας και την προστασία των δεδομένων των χρηστών, οι κωδικοί πρόσβασης δεν αποθηκεύονται σε απλή μορφή κειμένου (plain text) στη βάση δεδομένων. Αντίθετα, εφαρμόζεται η βιβλιοθήκη bcrypt, η οποία εκτελεί κρυπτογράφηση τύπου hashing με χρήση salt, καθιστώντας αδύνατη την ανάκτηση του αρχικού κωδικού ακόμα και σε περίπτωση διαρροής της βάσης.

```

const hashedPassword = await bcrypt.hash(password, 10);

await db.query(
  `INSERT INTO users (username, email, password)
  VALUES (?, ?, ?)`,
  [username, email, hashedPassword]
);

```


Απόσπασμα 13: Κεντρική Διαχείριση Σύνδεσης Βάσης (Database Connection Pooling)

Για τη βελτιστοποίηση της απόδοσης και την εξασφάλιση της υψηλής διαθεσιμότητας του συστήματος, εφαρμόστηκε η τεχνική του Database Connection Pooling. Αντί για τη δημιουργία μιας μεμονωμένης σύνδεσης ανά αίτημα, το σύστημα διατηρεί μια δεξαμενή (pool) ενεργών συνδέσεων, οι οποίες επαναχρησιμοποιούνται. Αυτή η προσέγγιση επιτρέπει την αποτελεσματική διαχείριση ταυτόχρονων αιτημάτων, αποτρέποντας την εξάντληση των πόρων της βάσης δεδομένων...

```
const mysql = require('mysql2/promise');
require('dotenv').config();

// βασικές ρυθμίσεις της βάσης από το .env
const dbConfig = {
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
};

// προσθήκη ssl μόνο όταν η εφαρμογή τρέχει σε production περιβάλλον
if (process.env.NODE_ENV === 'production') {
  dbConfig.ssl = {
    rejectUnauthorized: false
  };
}

// δημιουργία connection pool αντί για απλό connection για καλύτερο performance
const db = mysql.createPool(dbConfig);

// αρχικό τεστ για να βεβαιωθούμε ότι η βάση είναι προσβάσιμη
db.getConnection()
  .then(connection => {
    console.log("Επιτυχής σύνδεση με τη βάση (db.js)");
    connection.release(); // Την αφήνουμε ελεύθερη
  })
  .catch(err => {
    console.error("Σφάλμα σύνδεσης στο db.js:", err.message);
  });

module.exports = db;
```

Απόσπασμα 14: Προστασία από SQL Injection

Για την αποτροπή κακόβουλων παρεμβάσεων στη βάση δεδομένων και την προστασία έναντι επιθέσεων τύπου SQL Injection, το σύστημα υλοποιεί παραμετροποιημένα ερωτήματα (parameterized queries). Αντί της άμεσης ενσωμάτωσης των δεδομένων εισόδου του χρήστη στο σώμα του ερωτήματος SQL, χρησιμοποιούνται σύμβολα κράτησης θέσης (placeholders, ?). Με αυτή την πρακτική, το επίπεδο διασύνδεσης με τη βάση δεδομένων αναλαμβάνει την αυτόματη και ασφαλή διαφυγή (escaping) των χαρακτήρων, εξασφαλίζοντας την ακεραιότητα των συναλλαγών.

```
await db.query(
  "UPDATE users SET location = ?, interests = ?, budget = ? WHERE id = ?",
  [location, interests, budget, req.user.id] // Το ID από το token
);
```

Απόσπασμα 15: Συνολική Διαχείριση Σφαλμάτων (Error Handling)

Η εφαρμογή διασφαλίζει ότι σε περίπτωση απροσδόκητου σφάλματος (π.χ. αδυναμία επεξεργασίας αιτήματος από τη βάση δεδομένων κατά την αναζήτηση συγκεκριμένης δραστηριότητας), ο διακομιστής δεν καταρρέει. Το σφάλμα «συλλαμβάνεται» μέσω εντολών try-catch, καταγράφεται εσωτερικά (logging) και επιστρέφεται ένα σαφές μήνυμα σφάλματος στον χρήστη με HTTP Status Code 500.

```
// φέρνουμε λεπτομέρειες για μία συγκεκριμένη δραστηριότητα
router.get("/:id", async (req, res) => {
  try {
    const activityId = req.params.id;
    const [data] = await db.query("SELECT * FROM activities WHERE id = ?", [activityId]);

    // αν το id δεν υπάρχει στη βάση
    if (data.length === 0) {
      return res.status(404).json({ message: "Η δραστηριότητα δεν βρέθηκε" });
    }
    res.json(data[0]);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Σφάλμα στον server" });
  }
});

module.exports = router;
```

